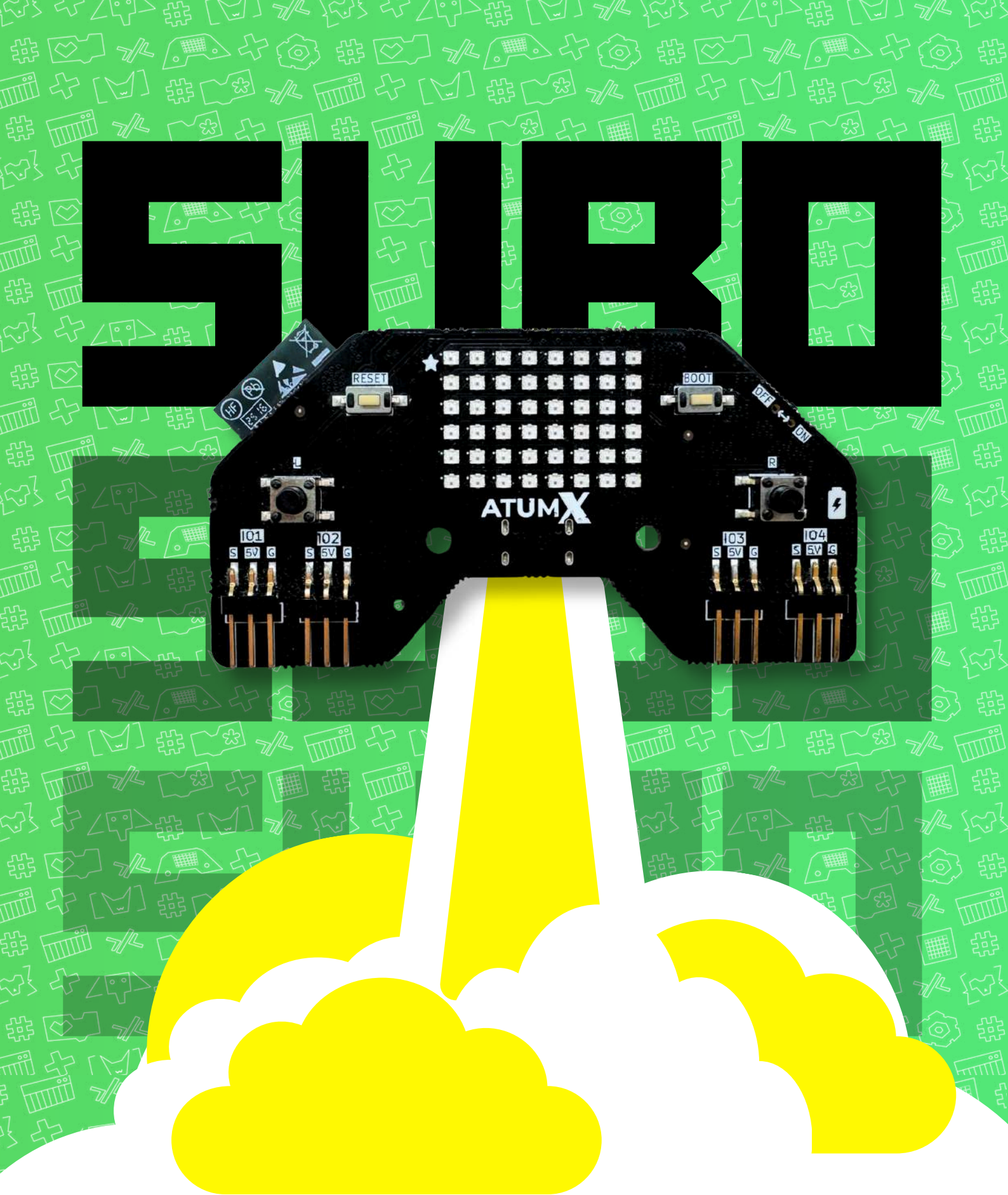




# User manual and Secrets



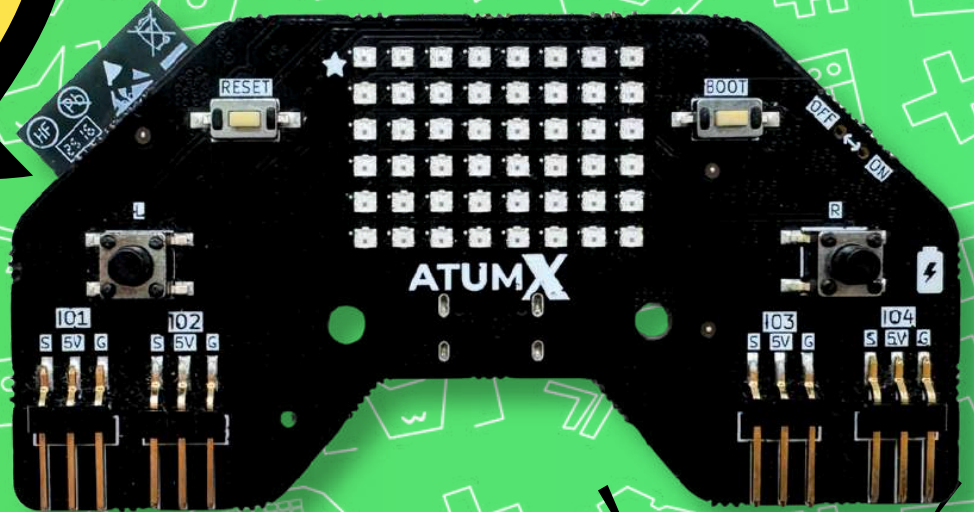
# Index

|                                                |           |
|------------------------------------------------|-----------|
| <b>Subo says Hi!</b>                           | <b>4</b>  |
| <b>Installing the App</b>                      | <b>5</b>  |
| <b>App features</b>                            | <b>6</b>  |
| <b>Entering Blocks</b>                         | <b>11</b> |
| <b>Connect Subo with App</b>                   | <b>12</b> |
| <b>Renaming SUBO</b>                           | <b>15</b> |
| <b>Theme</b>                                   | <b>17</b> |
| <b>How to make the code work?</b>              | <b>18</b> |
| <b>Recharging our battery using our board!</b> | <b>21</b> |
| <b>Board Intro</b>                             | <b>23</b> |
| <b>Variables</b>                               | <b>28</b> |
| <b>Ultrasonic Sensor</b>                       | <b>30</b> |
| <b>IR Sensor</b>                               | <b>33</b> |
| <b>LDR Sensor</b>                              | <b>36</b> |



|                                               |           |
|-----------------------------------------------|-----------|
| <b>Gas Sensor</b>                             | <b>39</b> |
| <b>Touch Sensor</b>                           | <b>42</b> |
| <b>Vibration Sensor</b>                       | <b>45</b> |
| <b>Sound Sensor</b>                           | <b>48</b> |
| <b>DHT-11 Temperature and Humidity Sensor</b> | <b>51</b> |
| <b>Relay Module</b>                           | <b>54</b> |
| <b>Soil moisture Sensor</b>                   | <b>57</b> |
| <b>LCD Display</b>                            | <b>60</b> |
| <b>Colour Sensor</b>                          | <b>63</b> |
| <b>OLED Display</b>                           | <b>68</b> |
| <b>180° servo motor</b>                       | <b>71</b> |
| <b>360° servo motor</b>                       | <b>74</b> |
| <b>Motor Magic</b>                            | <b>77</b> |
| <b>Blocky to python mode</b>                  | <b>81</b> |
| <b>Python to blockly mode</b>                 | <b>83</b> |
| <b>Blocks Glossary</b>                        | <b>85</b> |





**SUPPORTS  
MULTIPLE DIFFERENT  
EXPANSIONS**

**48 Jolly Lights**

**08 Pins of Power**

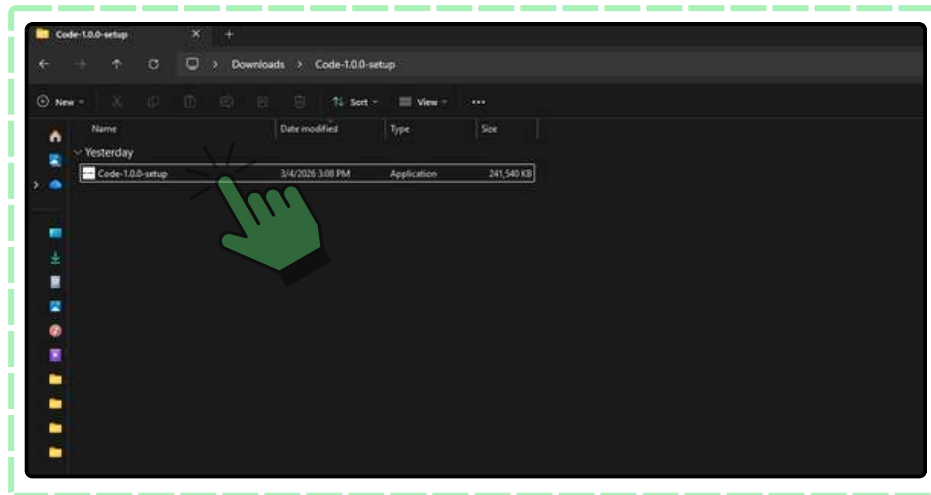
**01 Buzzer**

**02 Buttons**

**Infinite  Possibilities**

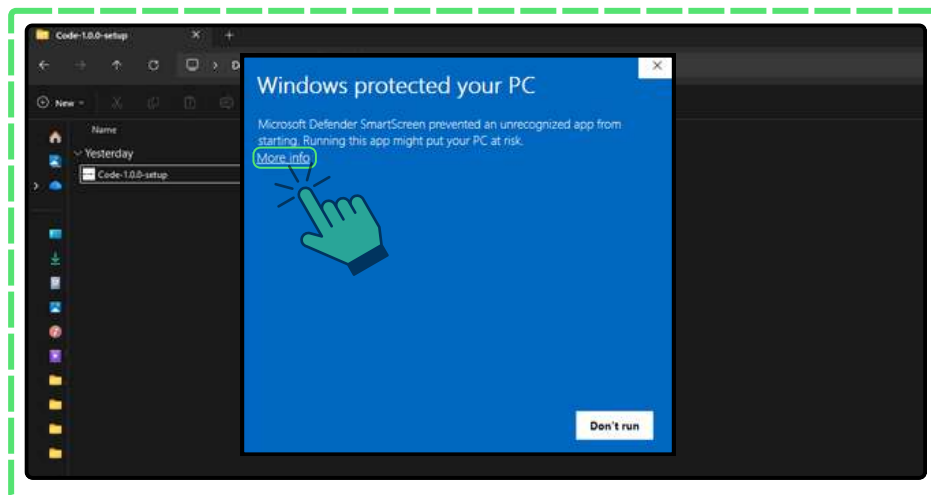






## Step 1:

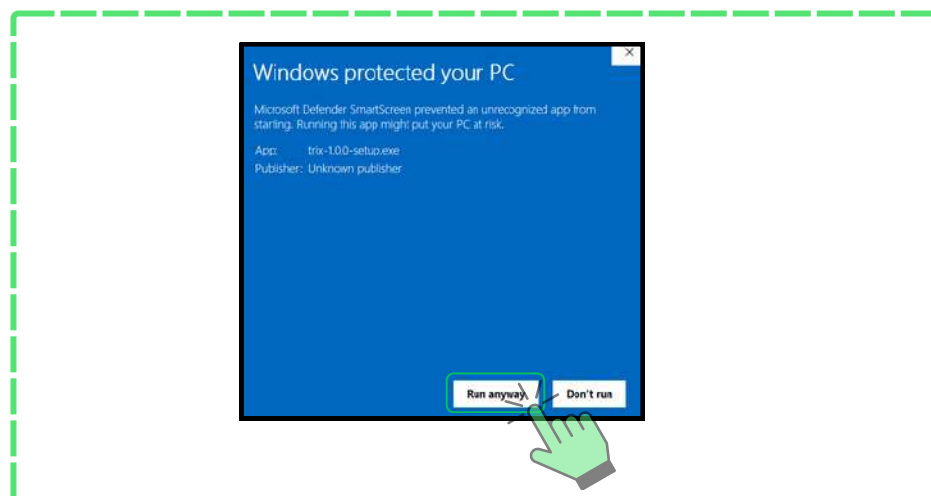
Click on the App you have downloaded from the QR code



## Step 2:

A Windows pop up will open. Don't panic, the App is safe.

Click "More info" as shown here.



## Step 3:

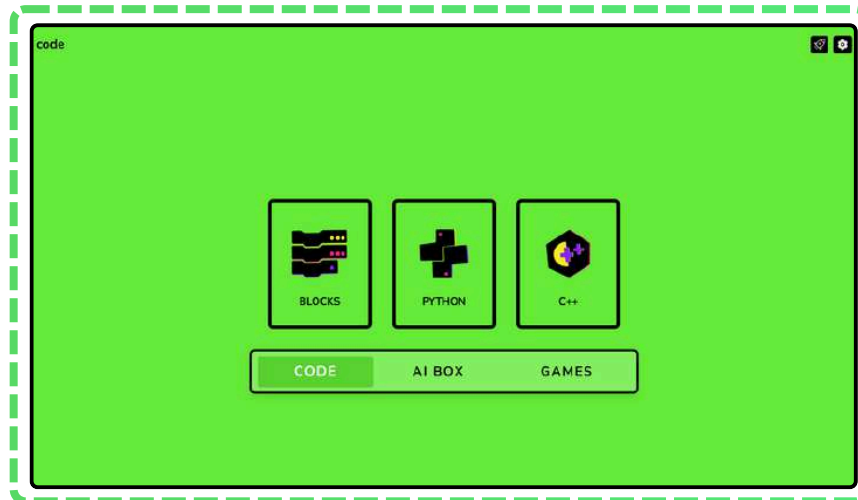
Click **Run** anyway and the app will be opened.

# App features

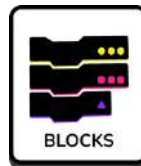
Meow...



## Home Screen



### Blocks Mode



A visual drag-and-drop coding system using blocks.

### Projects

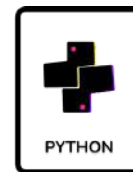


Opens all the programs and activities you have created or saved.

Icons in code



### Python



Text-based coding using the Python programming language.

### Settings



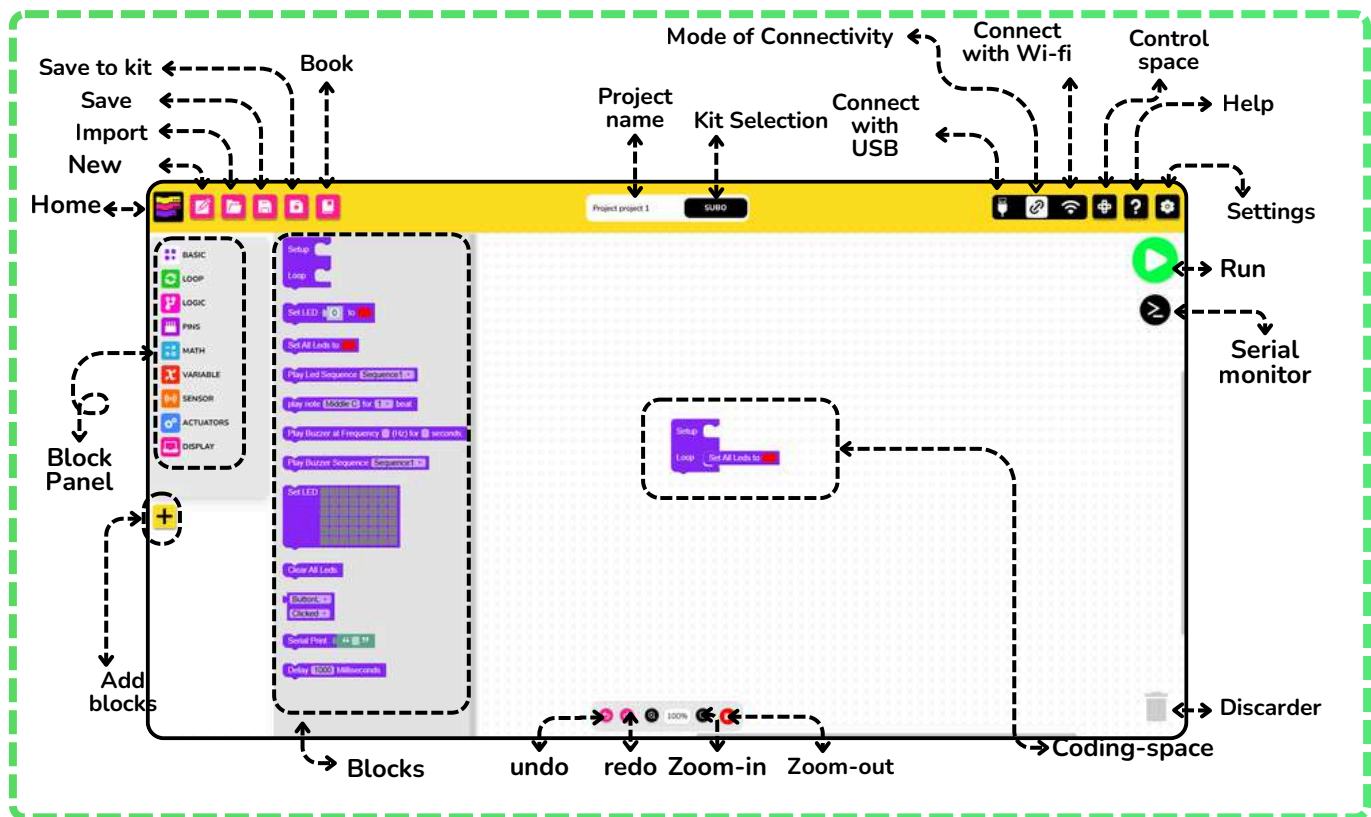
Used to control Sound, Theme and board options

### C++



Coding in the C++ language for faster, performance-based programs.

# Parts



## Blocks available:

-  **BASIC** → Contains fundamental blocks to control our board.
-  **LOOP** → Repeats actions using loops like “repeat” or “forever.”
-  **LOGIC** → Controls program flow using conditions like “if...then.”
-  **PINS** → Contains blocks that are used to perform read and write values
-  **MATH** → Performs mathematical operations and calculations.
-  **VARIABLE** → Stores and retrieves data using named variables.
-  **SENSOR** → Reads data from various sensors like light or temperature.
-  **ACTUATORS** → Contains various blocks to control actuators like motors, etc.
-  **DISPLAYS** → Contains blocks that are used with display and its operations.





# Functions



## Import

Loads a saved project.



## Book

Opens learning resources or documentation.



## Save to Kit

Uploads and stores the program directly to our hardware kit



## Save

Saves the current project



## New

Creates a new project



## Home

Returns to the main screen.



## Connect with USB

Links the device for uploading code using USB.



## Connect with Wi-Fi

Links the device for uploading code using Wi-Fi.



## Zoom out

Adjusts block view size.



## Undo

Reverses the last change.



## Redo

Restores the last change.



## Blocks

Displays all available block categories.



## Block Panel

The sidebar containing categorized blocks.



## Add blocks

For adding any other block extensions.



## Help

Place to find instructions and help



## Mode of connectivity

Choose between Wi-Fi and USB mode of connecting our hardware



## Control space

Has a dedicated remote control space for our hardware kits



## Settings

For app related settings.



## Run

Executes the program.



## Discarder

Removes unwanted blocks.



## Serial monitor

Displays data from the connected device.



## Coding-space

The area where the program is built using blocks.



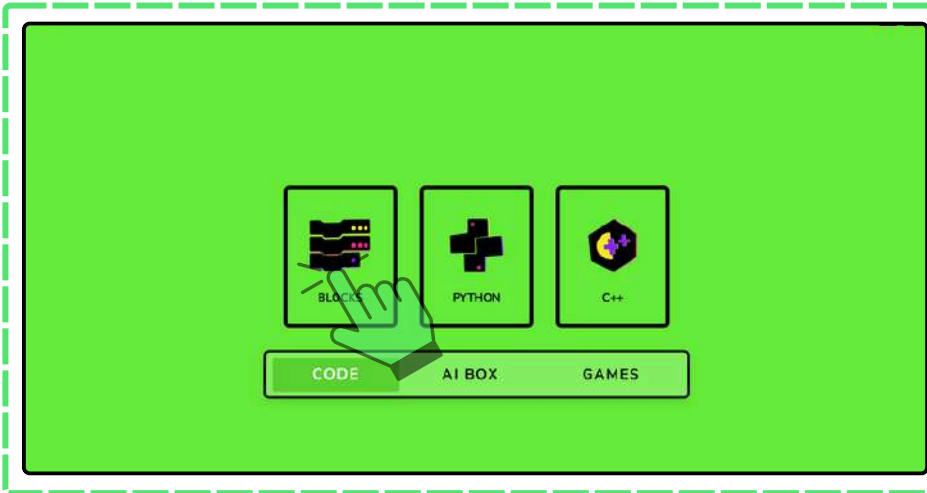
## Zoom in

Adjusts block view size.



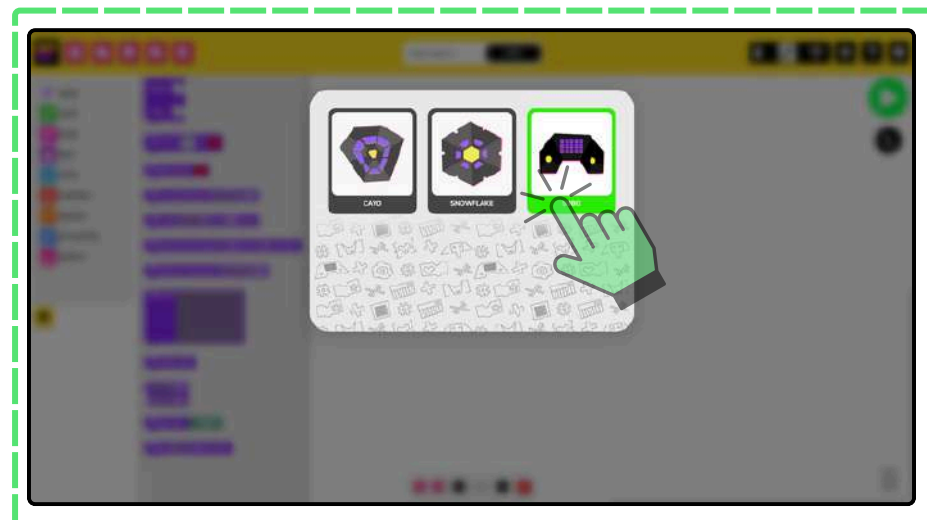
# Blocks:

## Entering blocks



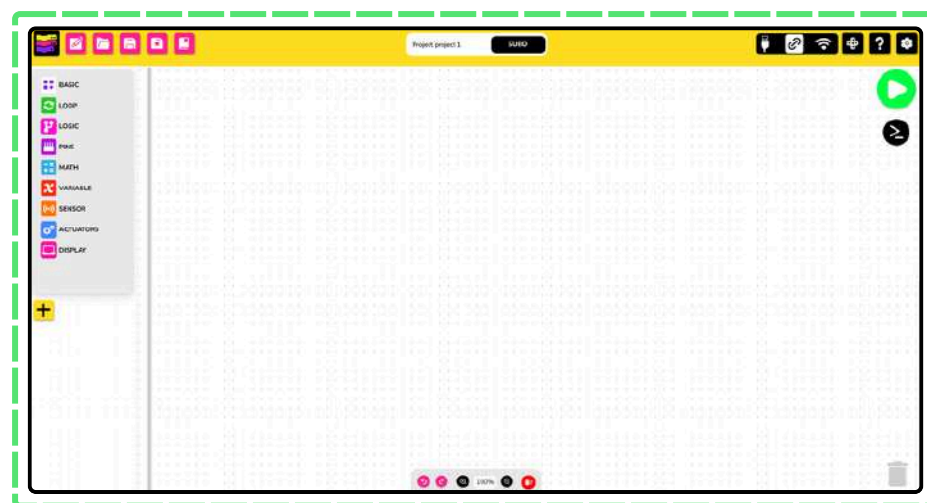
### Step 1:

Open the App you have downloaded and press **Blocks** in code section.



### Step 2:

Click on **Subo**.



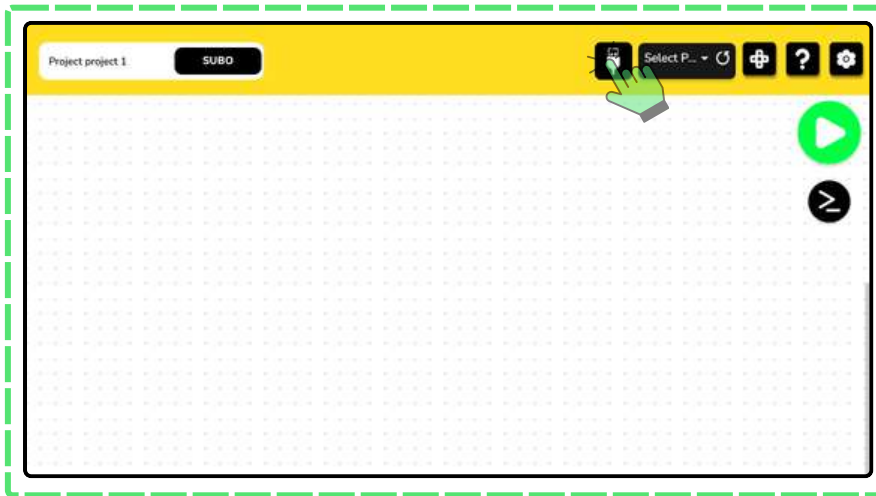
The app is now **open** and ready for coding.

Koneckt.

Connect!

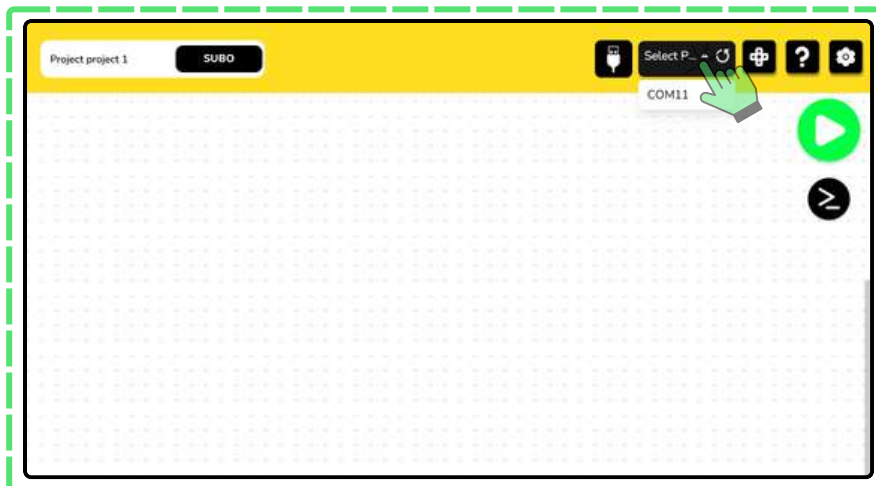
# Connect with the app

## Wired mode :



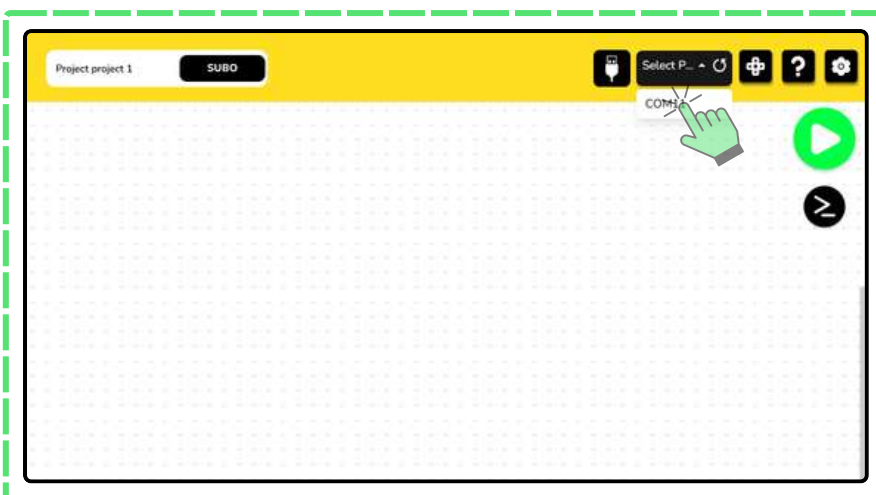
### Step 1:

Making sure the board is connected to the PC, **Click** on the **USB Connect** icon .



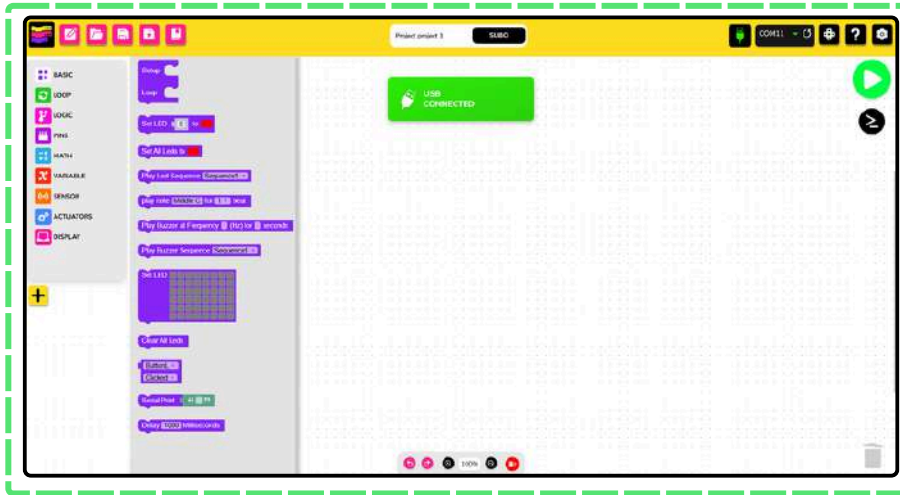
### Step 2:

Click on the **Drop-down box** as shown.

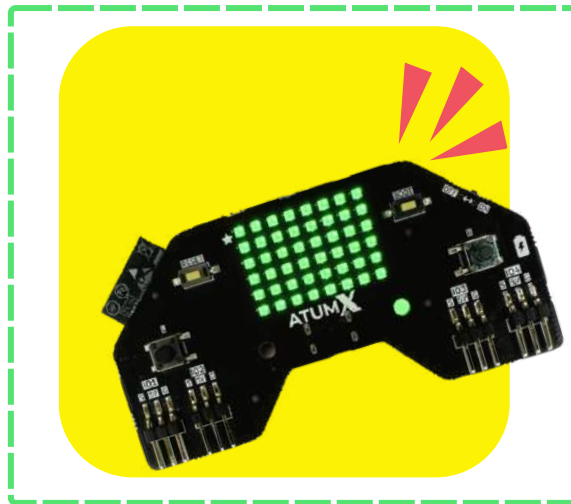


### Step 3:

Click on the **COM port** and wait for a popup.

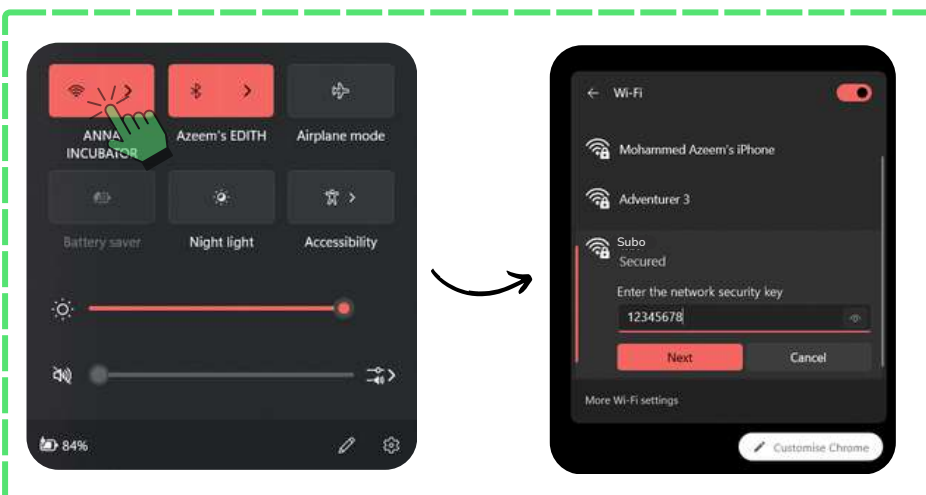


Once the popup appears, the wired icon changes to green, & the board will glow with **green LEDs**



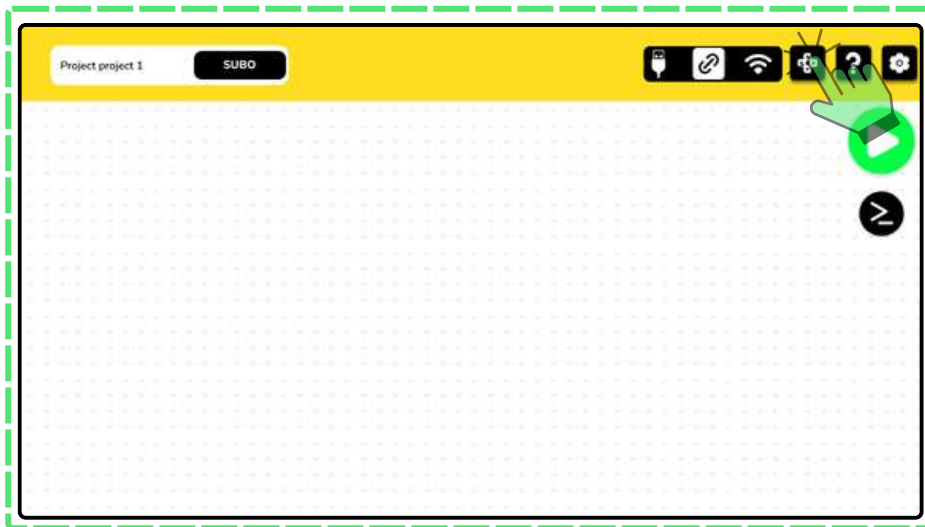
All the LEDs turn green, confirming that the board has been successfully connected “wirelessly”.

## Wireless mode :



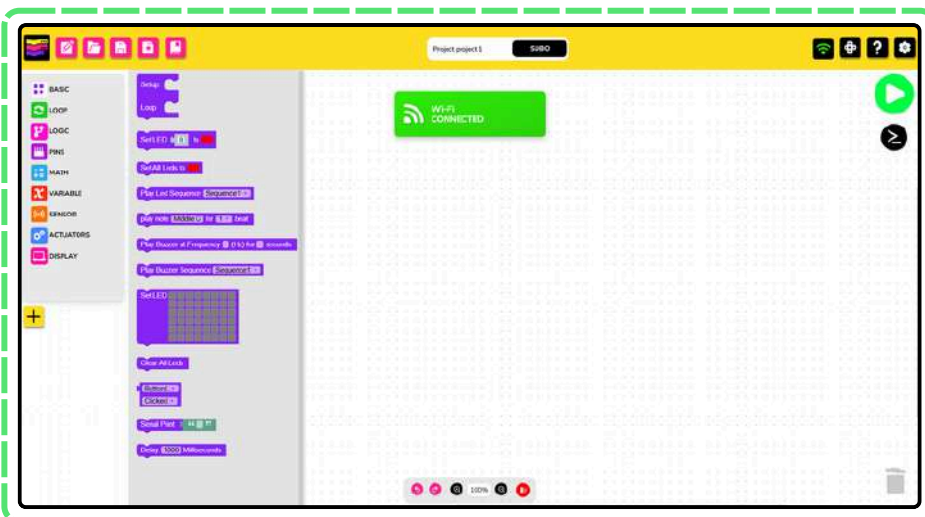
### Step 1:

Open Wi-Fi Settings. Then Select “**SUBO**”, Enter **password** (12345678) as shown and connect.



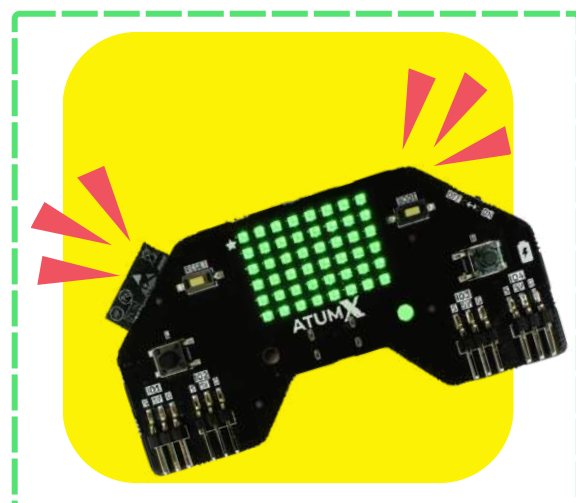
## Step 2:

Click on the **Wi-Fi icon** as shown.



Once the popup appears, the Wi-Fi icon changes to green, & the board will **glow with green LEDs**

All the LEDs turn green, confirming that the board has been successfully connected "wirelessly".



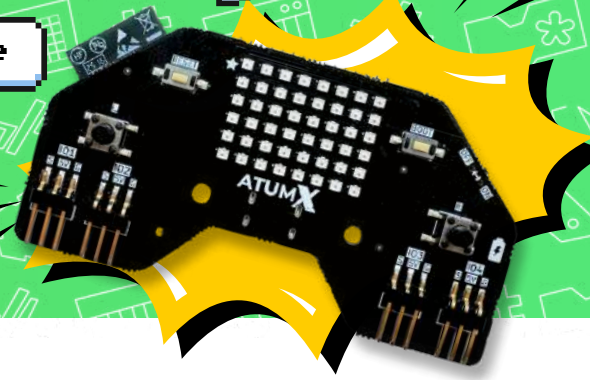


# Renaming SUBO



Rename

Huh?!



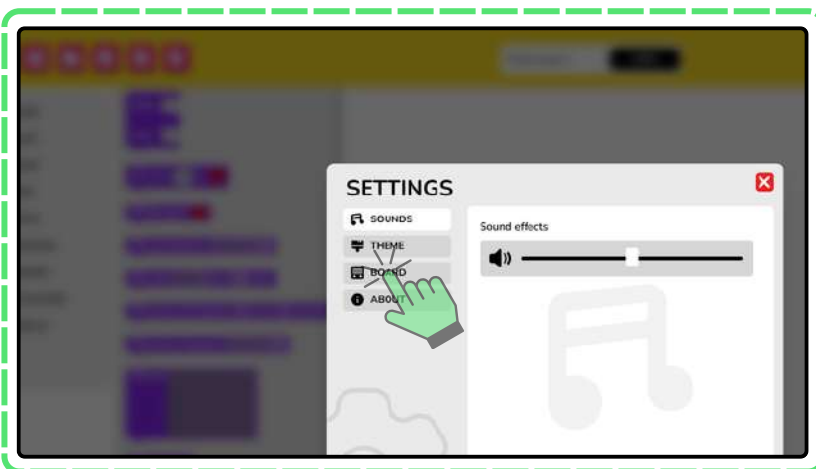
"SUBO" is fine. But fine isn't good enough for a board that's excellent. Give it a name that feels like you. Maybe it's your name, your team's name, or just something that sounds cool. Open the Code App, tap Rename, and make it official.

Before renaming lets ensure to keep the board connected Wirelessly via Wifi.



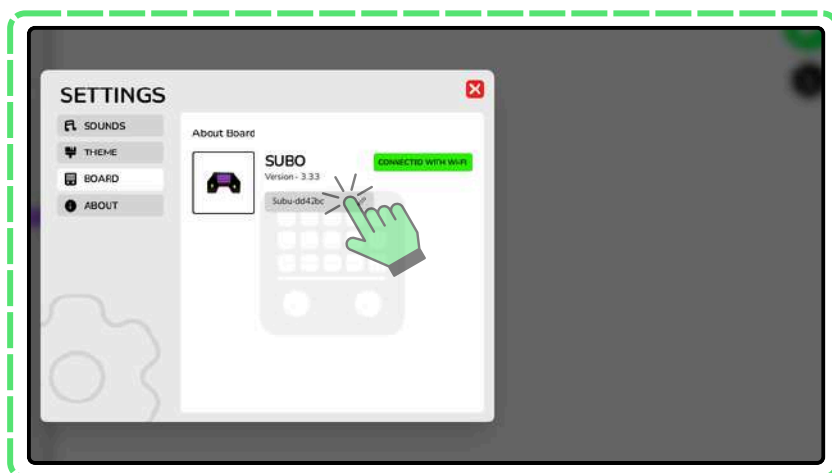
## Step 1:

Click on the **Settings** icon present on the top right corner.



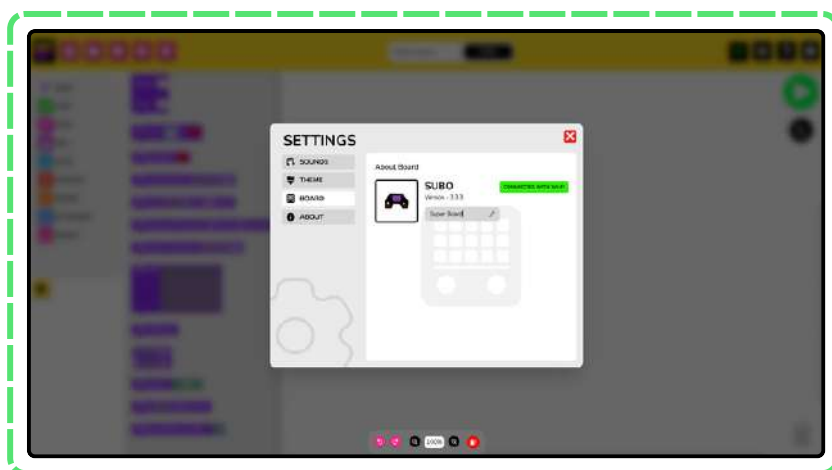
## Step 2:

Click on Board



### Step 3:

Click on the **rename** button



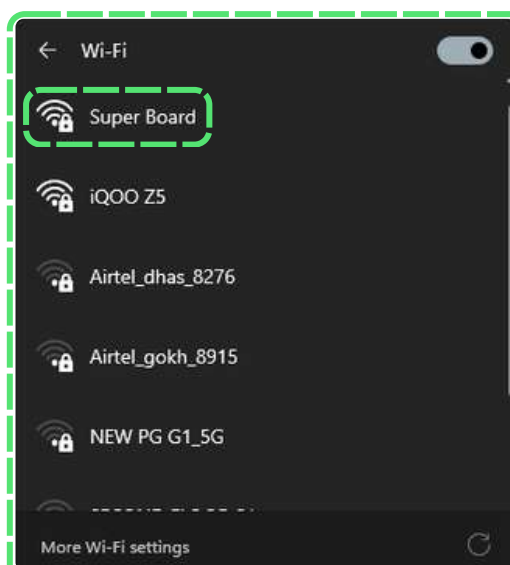
### Step 4:

Rename the board with any name of your choice and hit **Enter**.

Once you hit enter, the board disconnects.



Do not worry, your board can be accessed using the name assigned to it along with its default password “12345678”

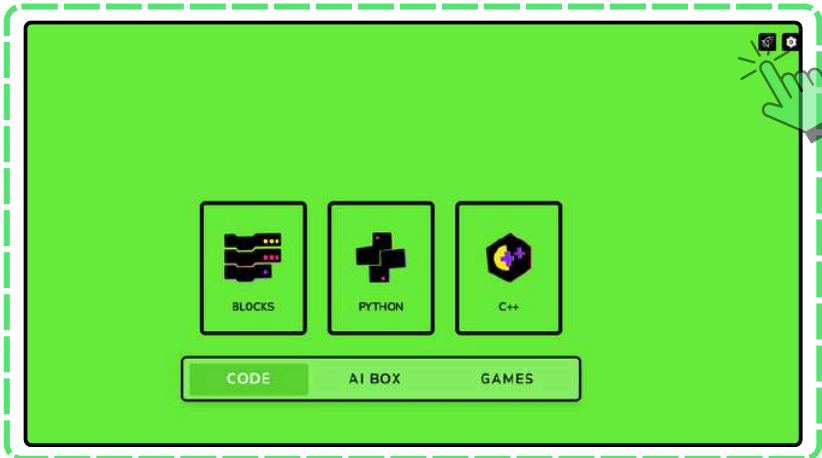


Here's how it looks after successfully renaming our Subo board.

# Theme (Light/Dark)

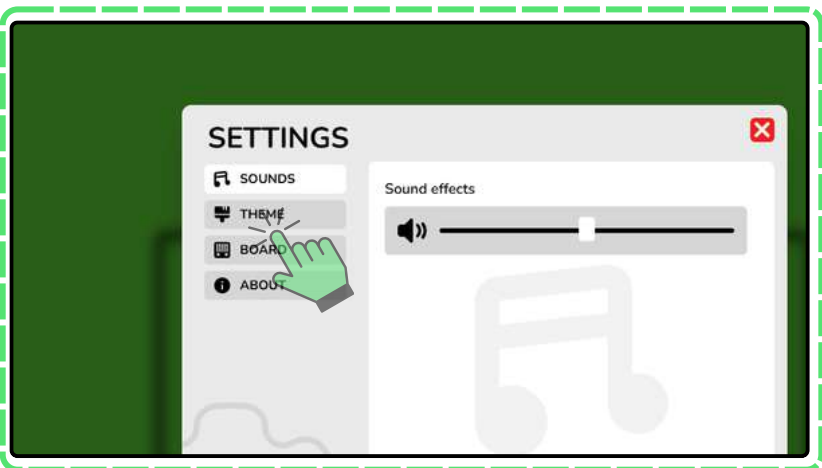


Customize your coding experience by switching between **Light** and **Dark mode** in the **Code App**.



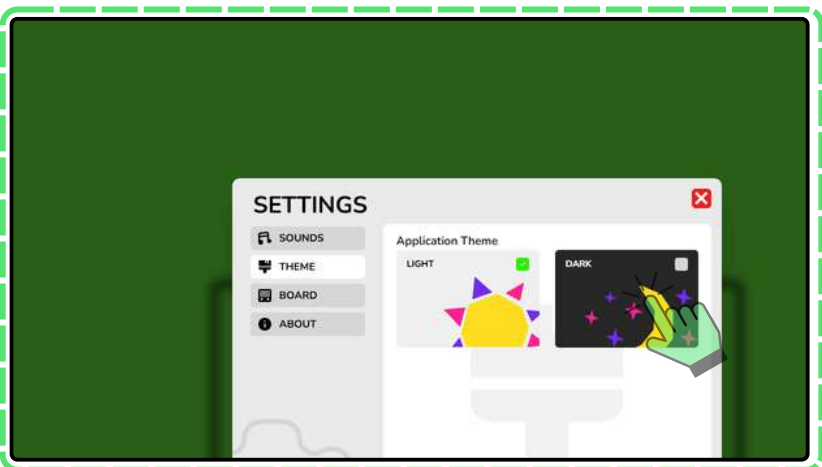
## Step 1:

Click on the **Settings** icon present on the top right corner.



## Step 2:

Click on **Theme**



## Step 3:

Click on **Dark** below Application theme to change the theme.

# How to make the code work?

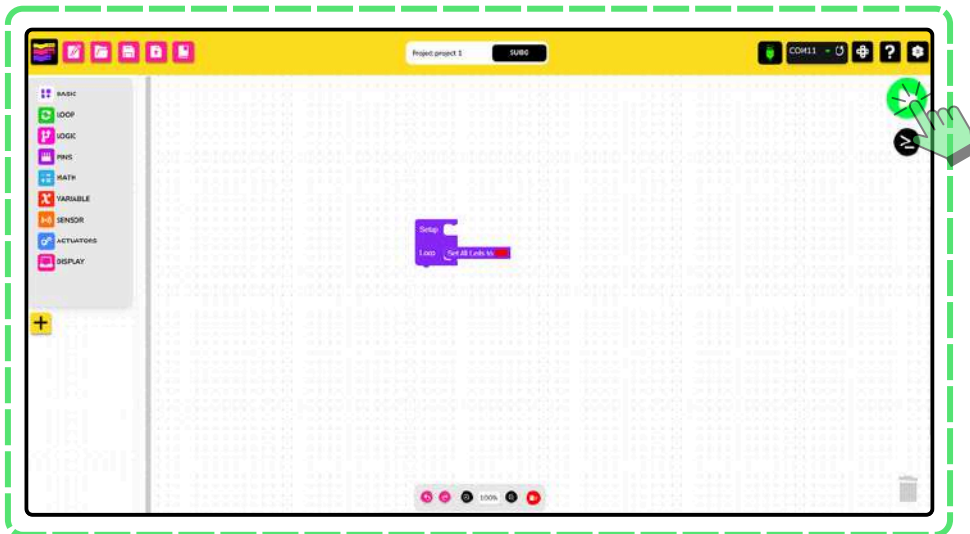


Before we start getting things done with **SUBO**, let's learn the two simple ways to program and make things happen. Take a look below



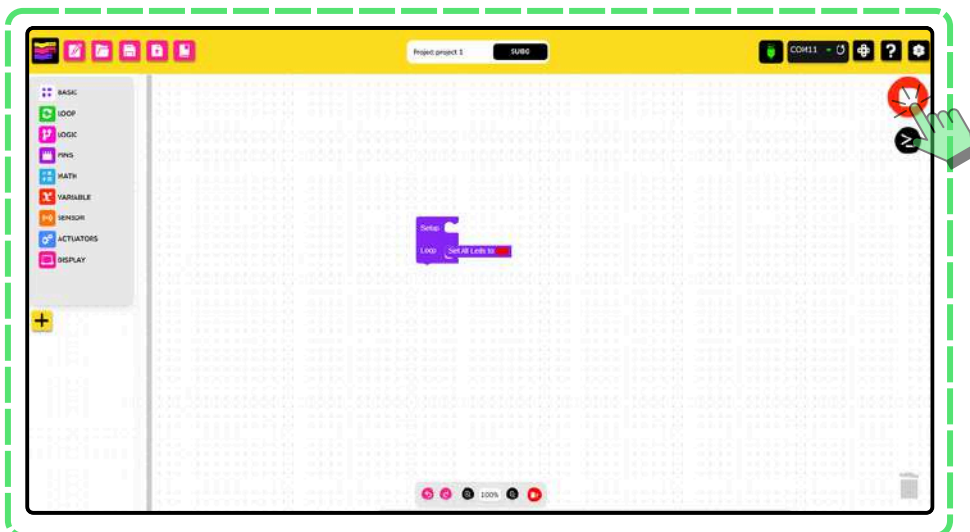
Make sure to keep the board connected with the Code App before running the code

## Using Run button



### Step 1:

Click this to instantly send your program to SUBO and watch it execute right away in real time



### Step 2:

Click the red square button to **stop** SUBO from **running** the code whenever you want



# Saving to kit

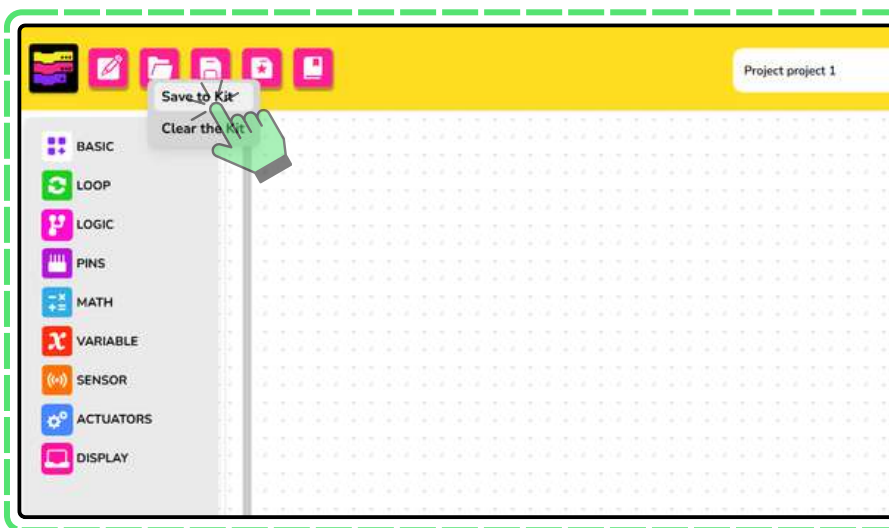


Make sure to keep the board connected with the Code App before running the code



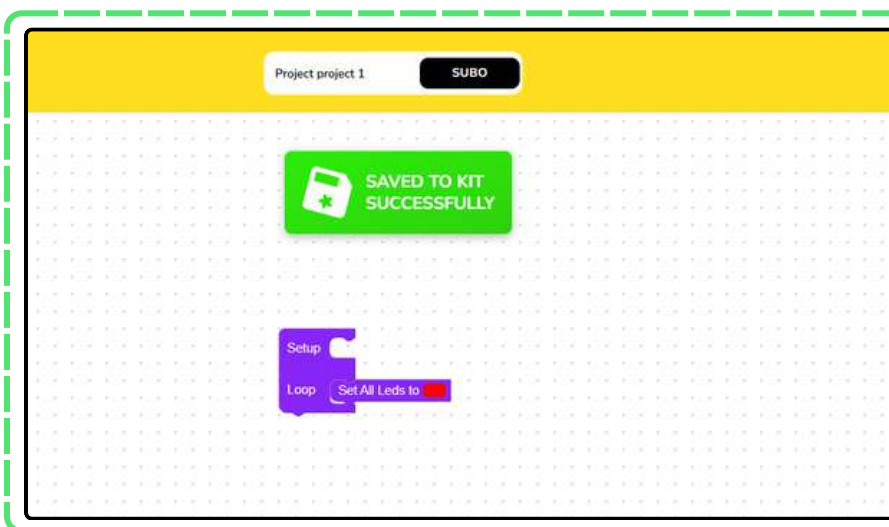
## Step 1:

To understand this better lets use **Save to kit** with a simple LED Program using these blocks.



## Step 2:

Click the **save to kit button** from the top bar



## Step 3:

The green color **saved to kit** pops up indicating the program is saved successfully.



After the program is saved, it runs automatically every time the board is turned on.

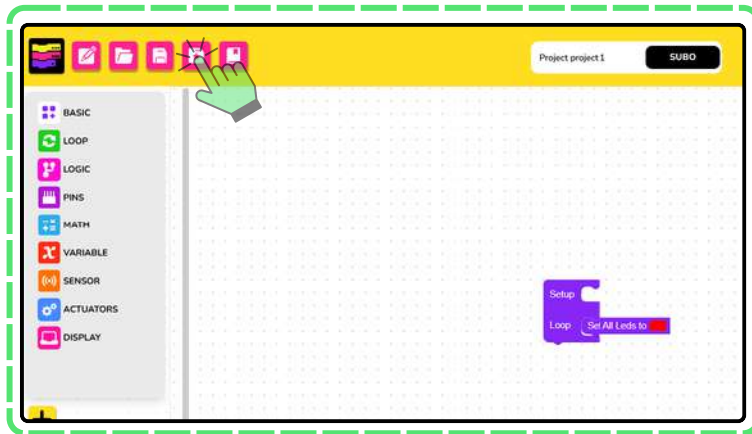
# Removing the saved programs



As we have mentioned earlier, the saved programs keep running when the device is plugged in or reset every time. To stop these programs from running automatically and to remove them from the kit we use the function **Clear the kit**.

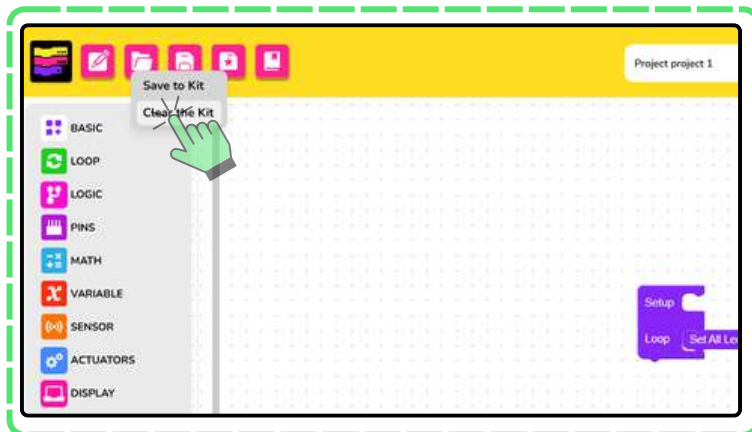


Make sure to keep the board connected with the Code App before running the code



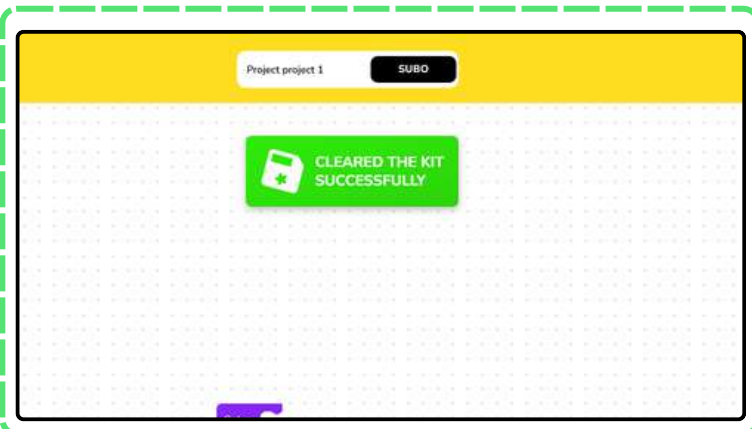
## Step 1:

Open the **Save to kit** Section by pressing the Save to kit icon



## Step 2:

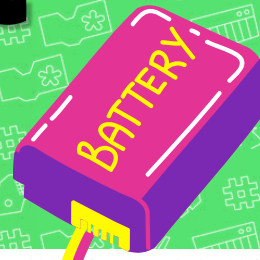
Click on **Clear the kit** option from the list



## Step 3:

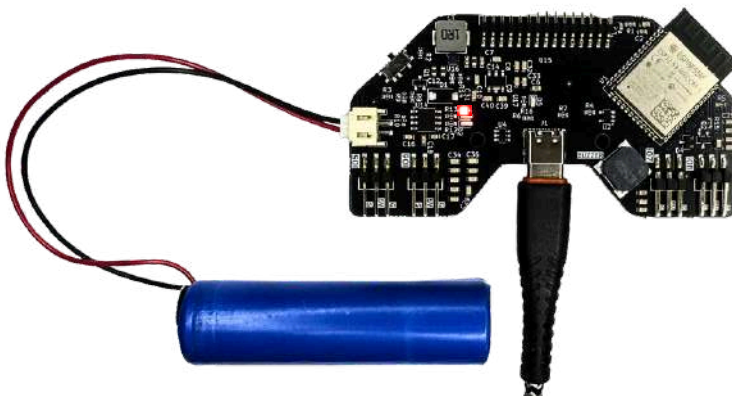
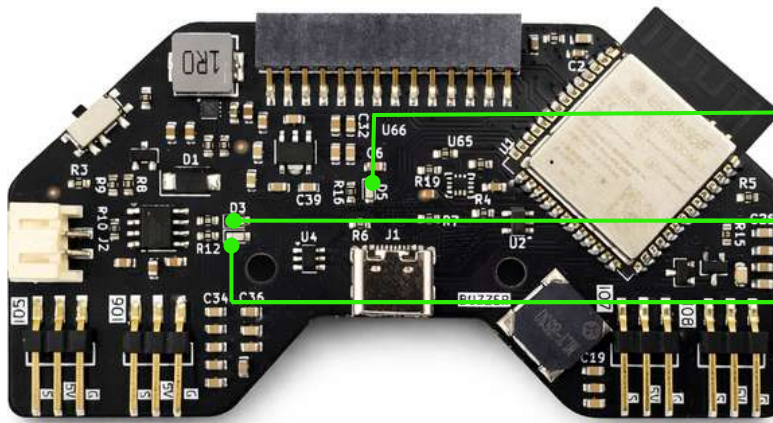
A green color “**Cleared the kit successfully**” pops up indicating the kit has been cleared successfully.

# Battery recharge with SUBO



## CAUTION

If you order the board by itself, the rechargeable battery isn't included - but don't worry, you can order one separately from us. The good news is that our Animal and Car kits already come with a battery included.

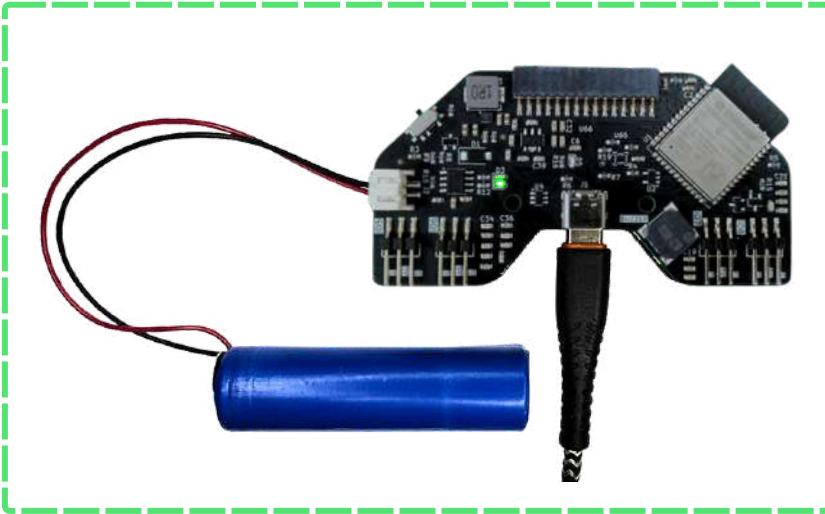
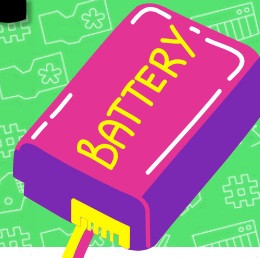


## Charging:

LED glows **RED** = Battery is **charging**



# Battery recharge with SUBO



## Battery full:

LED Glows **GREEN** = Battery is Full

Power LED - Glows in **Orange color** when the board is turned on

Red LED - Glows in **Red color** indicating the battery charge

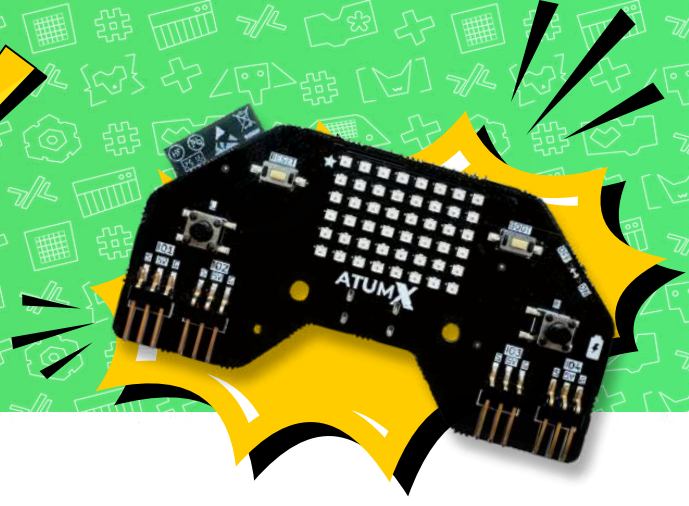
Green LED - Glows in **Green color** indicating a full charge of battery

OUR BATTERIES USUALLY TAKE  
45MINS TO 2HRS TO FULLY  
RECHARGE





# Board Intro



**BEFORE YOU START: RUNS  
ONCE & RUNS CONTINUOUSLY!**



Setup

Loop

## Setup

The code blocks inside setup() run only once at the beginning. They're used to prepare the system like setting pin modes, starting sensors.

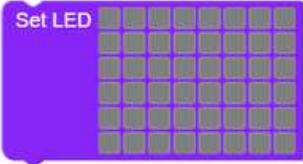
## Loop

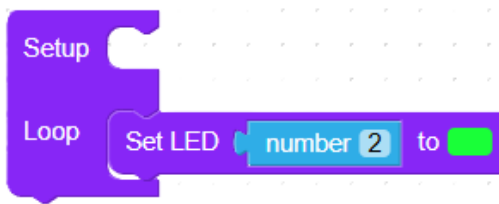
The blocks inside loop() run again and again in a continuous cycle. This is where the main program logic goes checking sensors, turning LEDs on/off, playing sounds, etc.

# Control the LEDs

| BLOCK PANEL | BLOCK | PURPOSE                                                                                                            |
|-------------|-------|--------------------------------------------------------------------------------------------------------------------|
|             |       | Turn on one led specified in the blank in particular color specified. (Can be used with number or variable blocks) |
|             |       | Turn on all leds in particular color specified                                                                     |
|             |       | Turns off all the LED's on SUBO                                                                                    |

# Create your own pattern!

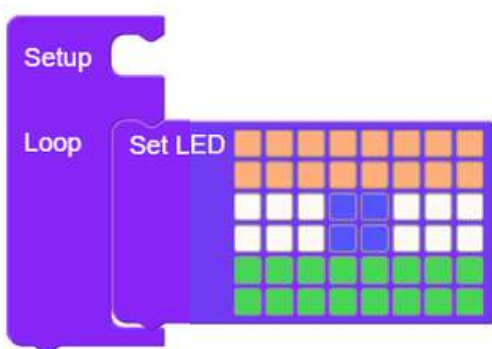
| BLOCK PANEL                                                                       | BLOCK                                                                             | PURPOSE                                                           |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------|
|  |  | Turn on LEDs selected in the blank in particular color specified. |
|  |  | Displays a custom in-built sequence for LEDs                      |



Turns on LED 2 in green color.



Turns on all LEDs in green color.



The top two rows are made orange , center with white and the bottom two rows are green.

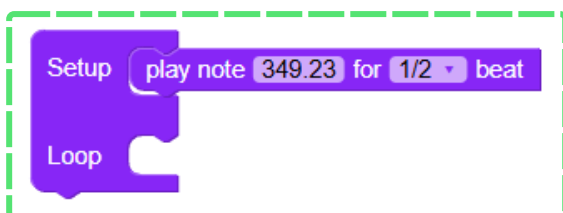


The LEDs glow as programmed

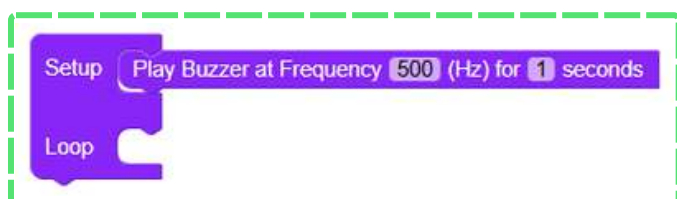
# Control the Buzzer

| BLOCK PANEL | BLOCK | PURPOSE                                                                 |
|-------------|-------|-------------------------------------------------------------------------|
|             |       | Play buzzer at particular frequency for specified time.                 |
|             |       | Play buzzer at particular frequency for specified beat that you select. |
|             |       | Play buzzer at particular sequence.                                     |

## Examples

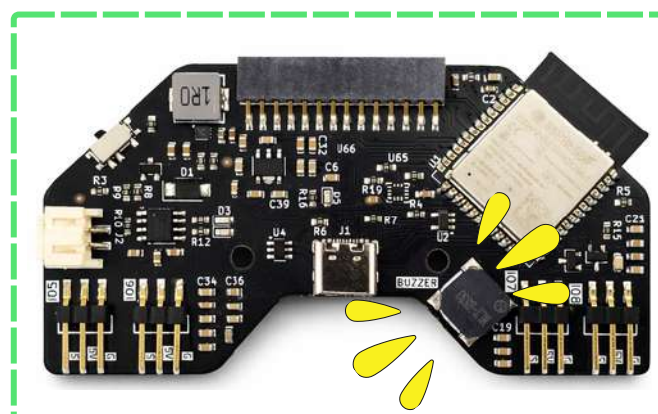
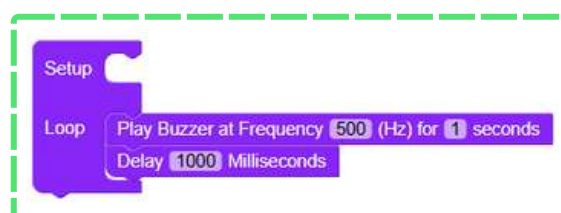


Play note middle F for 1/2 beat.



Play buzzer at 500 Hz for 1 second

Add a **delay** when using **Play Note** or **Play Buzzer** blocks in **Loop**, and keep the **delay duration** equal to the **buzzer tone duration** as shown below



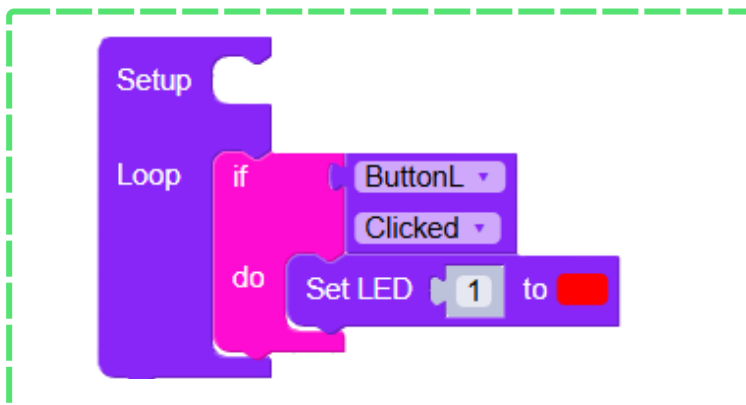
Voila! There goes your buzzer.



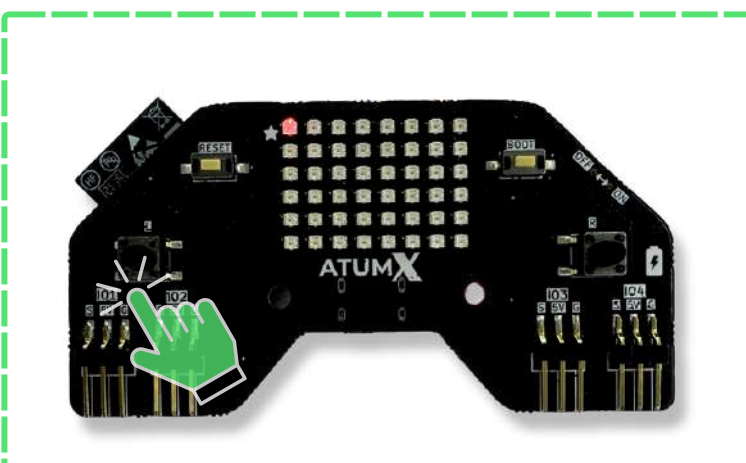
# Control Buttons

| BLOCK PANEL | BLOCK | PURPOSE                                  |
|-------------|-------|------------------------------------------|
|             |       | Detects whether left button is clicked.  |
|             |       | Detects whether right button is clicked. |

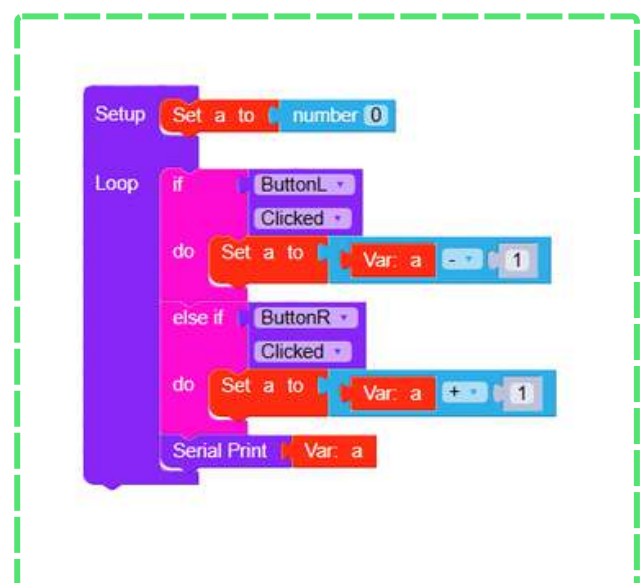
## Examples



Turn on LED 1 in red color if left button is clicked.



LED 1 glows red when left button is clicked.

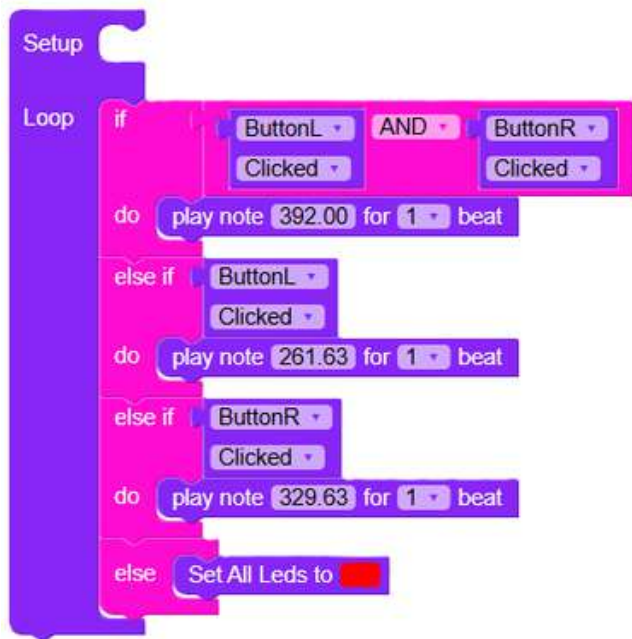


Setting variable values based on clicking left or right button.

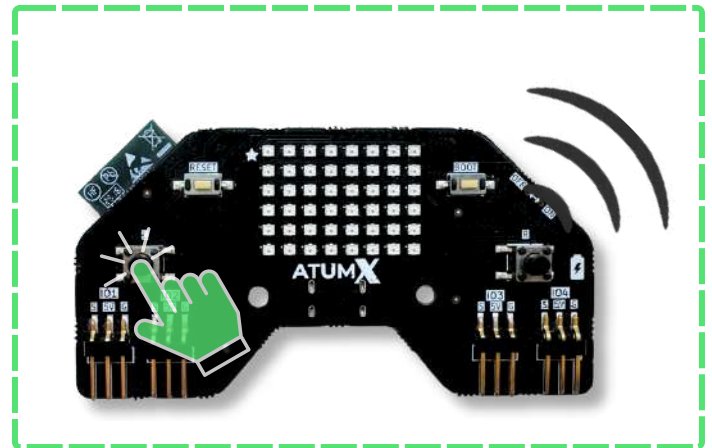


# Control Buttons

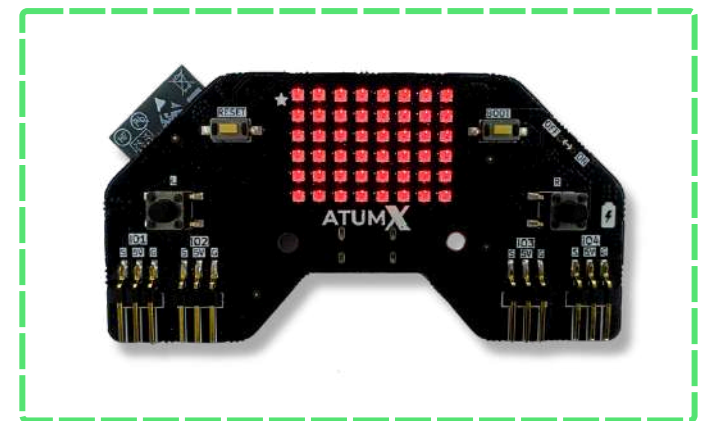
## Examples



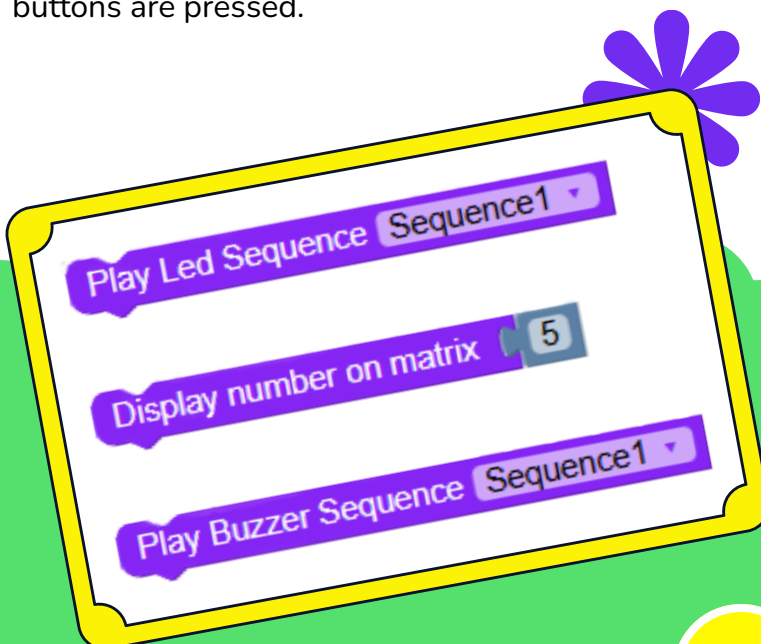
Play different notes for accessing left and right buttons and turn on all LEDs in red color if no buttons are pressed.



Plays a note when a button is pressed.



When no buttons are pressed.



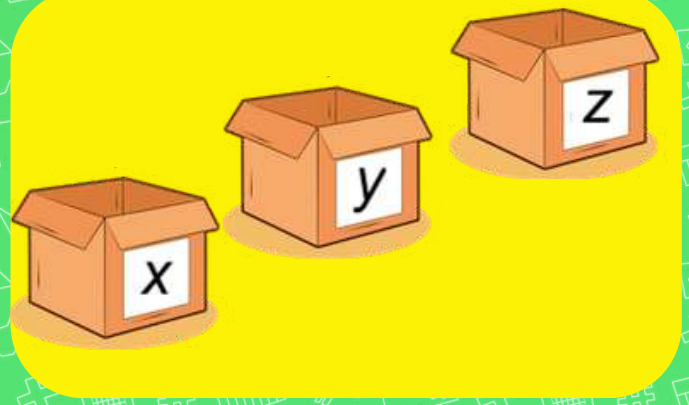
try!  
me!

Try these blocks in the code  
and have some extra fun!

# Variables



Before we get started  
lets get to know more  
about variables!



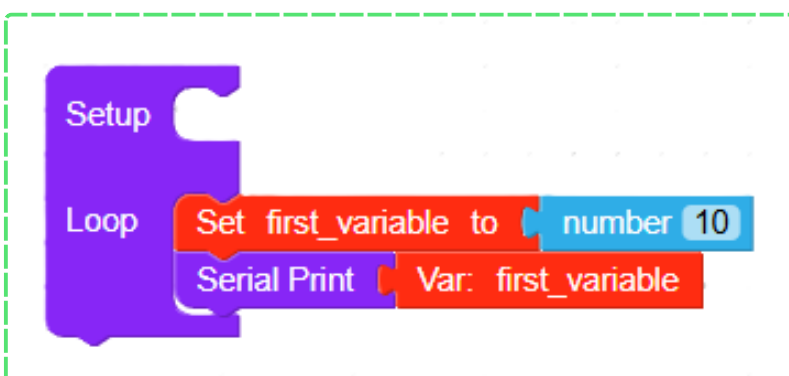
A variable is like a box that stores information. In Blockly, you can create a variable, give it a name, and put any value inside it like a number or a word. Your program can use that value anytime, and you can even change it while the program is running.

Variables are super useful you can use them to keep track of things in a loop, make decisions in your logic, or simply store information for later. Think of them as your program's memory.

## Our code app consists of totally 3 blocks for variables:

|                                                                                     |                                                                                   |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  | Used for creating a variable                                                      |
|  | Sets a value from a sensor, number or value from another variable to the variable |
|  | Used with print or logical statements to retrieve the values of the variable.     |

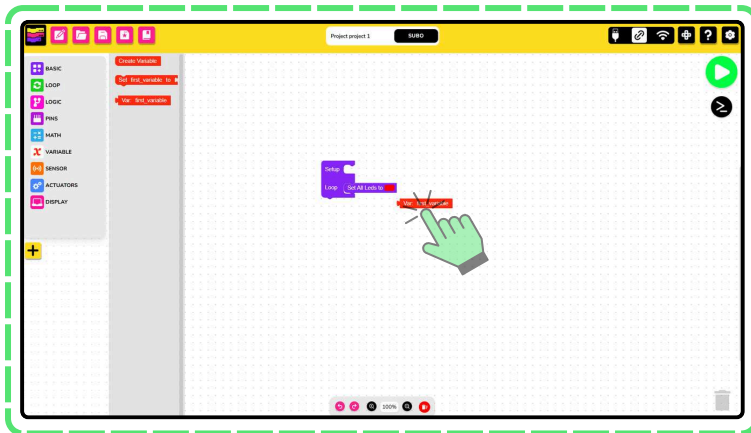
## Example:



The number **value 10** is **stored** in the variable name **first\_variable**

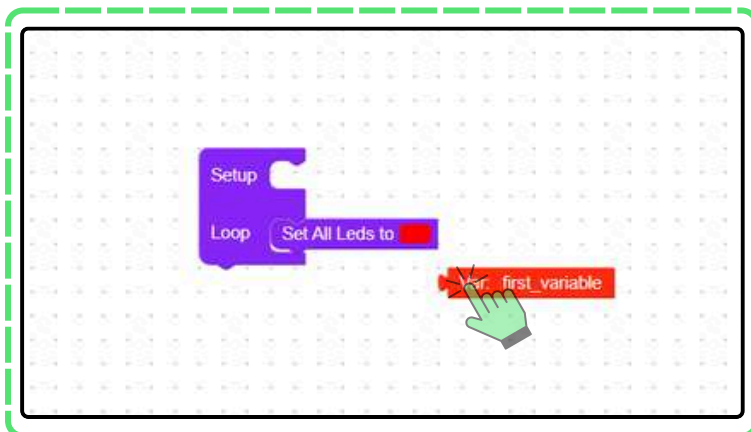
This program prints the number 10 continuously until manually stopped.

## To delete a variable permanently:



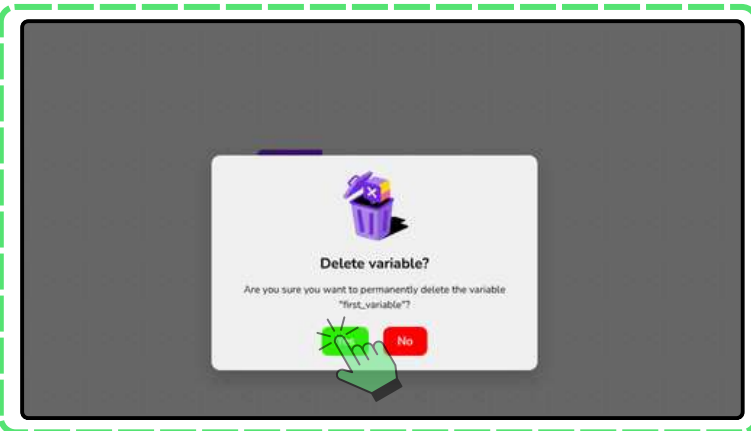
### Step 1:

Open the **Save to kit** Section by pressing the Save to kit icon



### Step 2:

Click on the variable until the delete variable pops up.



### Step 3:

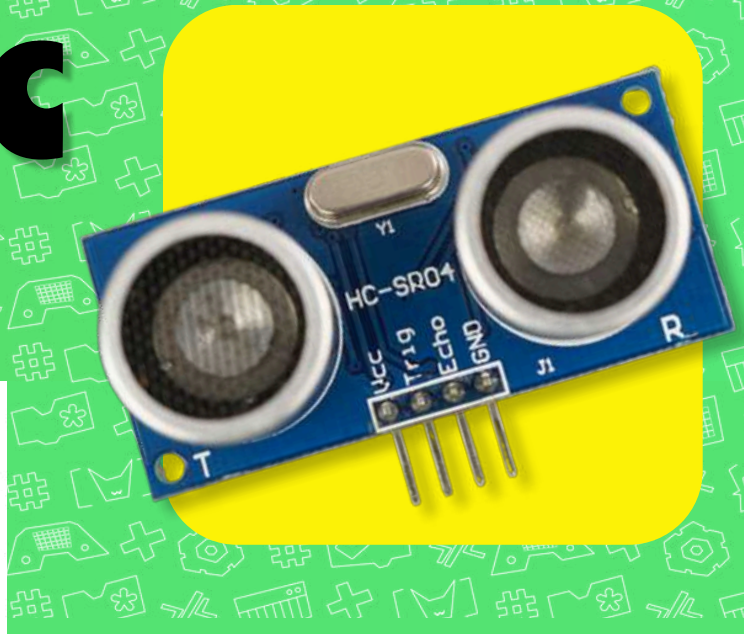
Click on **Yes**. It delete the variable permanently.



# Ultrasonic sensor

## Models Compatible

HC-SR04



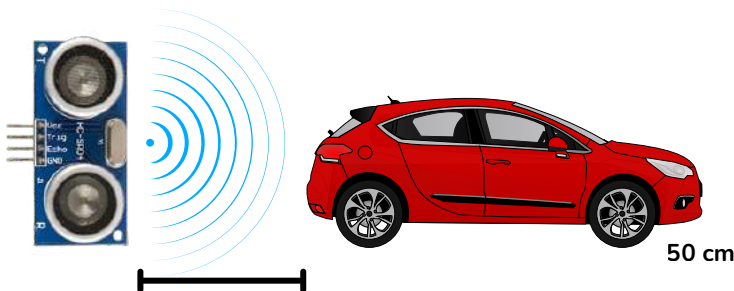
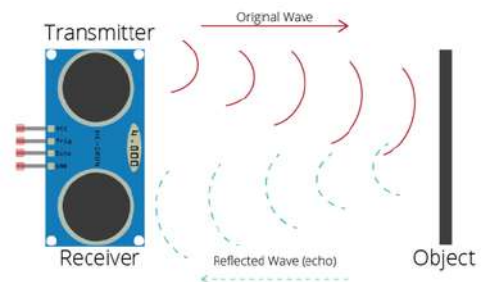
**Ultrasonic Sensor** sends out high pitched sound waves and waits for them to bounce back after hitting something. By measuring how long the echo takes to return, it can figure out how far away the object is. The sensor emits ultrasonic waves (usually around 40 kHz), which travel through the air, reflect off an object, and return to the sensor's receiver.

- The distance is calculated using the formula:

$$\text{Distance} = \frac{\text{Speed of Sound} \times \text{Time}}{2}$$

- The division by 2 is because the sound has to travel to the object and back.
- It has two main parts:
  - a. **Transmitter (TX)**: Sends out the ultrasonic pulse.
  - b. **Receiver (RX)**: Listens for the echo.

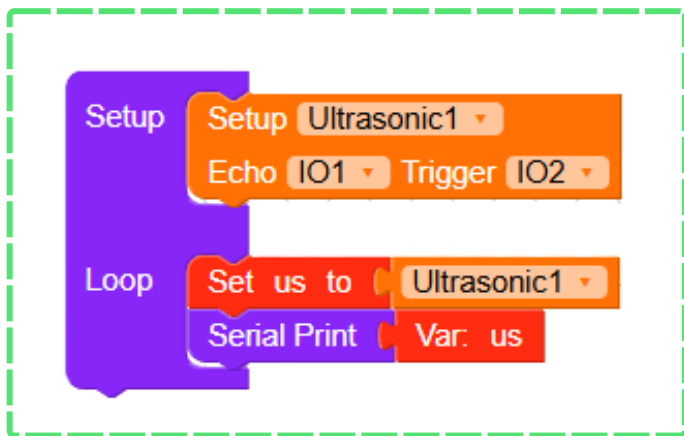
## Sensor Working



The ultrasonic sensor emits waves that bounce off an object. By measuring the return time, it calculates the distance. As the object moves farther, the return time increases, giving a higher distance reading.



# Block Code



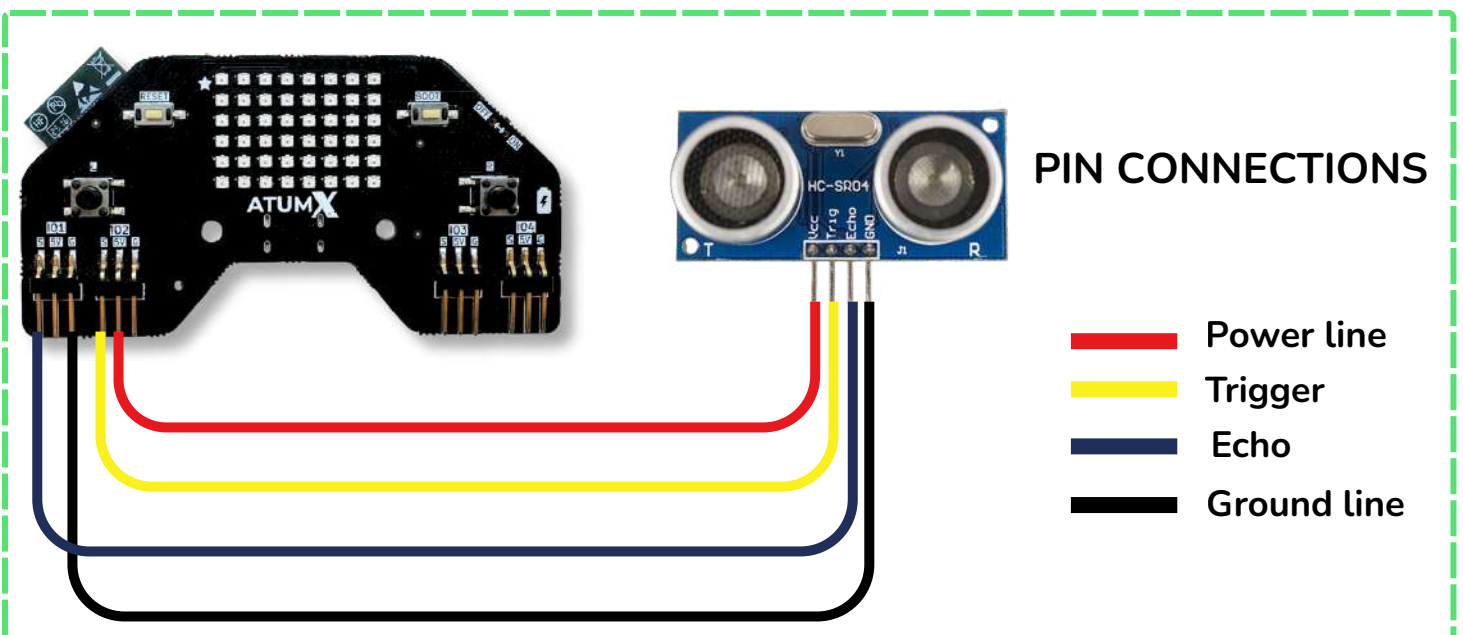
Video link on how to connect



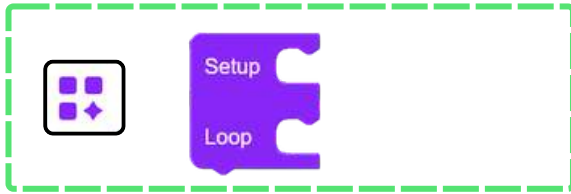
# Pin Connection

| Board                                                                                       | Sensor |
|---------------------------------------------------------------------------------------------|--------|
| 5V                                                                                          | VCC    |
| Gnd                                                                                         | Gnd    |
| Connect to <b>S(IO1)</b> or any other <b>available IO pin</b> defined in the snippet block. | ECHO   |
| Connect to <b>S(IO2)</b> or any other <b>available IO pin</b> defined in the snippet block. | TRIG   |

Check the IO pins you connected in the board and in the code snippet are same.

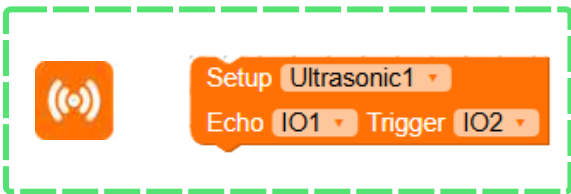


# Code explanation



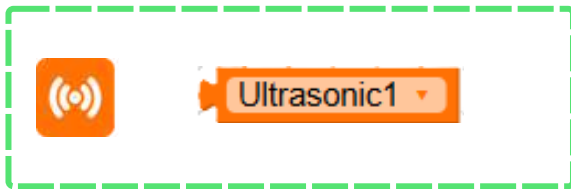
## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs once at the start, while code in **Loop** runs continuously.



## Block 2:

Configure the ultrasonic sensor pins. Select any available **GPIO** for **Trigger** and **Echo** using this block



## Block 3:

This block **triggers the ultrasonic sensor to measure the distance value based on sound wave reflection**



## Block 4:

Create a **variable** with variable name **“us”**



## Block 5:

This block **stores the value** of Ultrasonic Sensor in a variable that can be reused for printing and other logical operations.



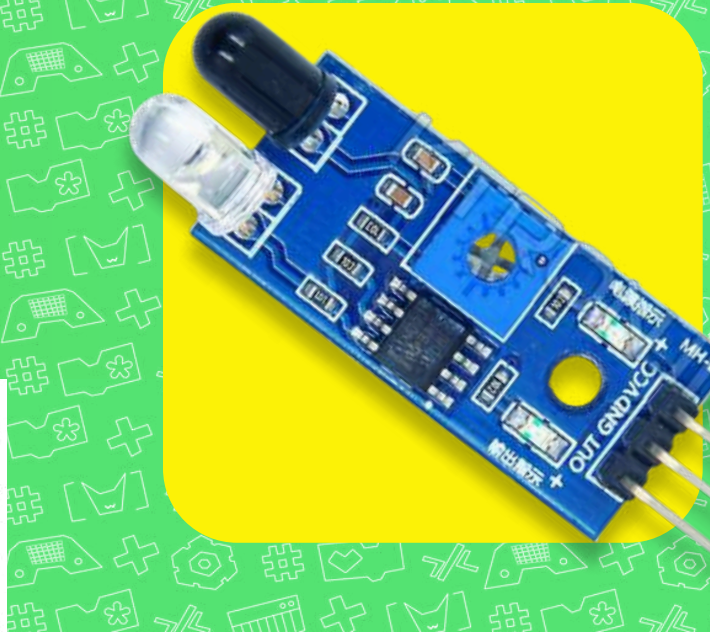
## Block 6:

Use **Serial Print block** with variable block to display Ultrasonic sensor values on the **Serial Monitor**.

# IR sensor

## Models Compatible

3 pins- LM393  
4 pins- TCRT5000



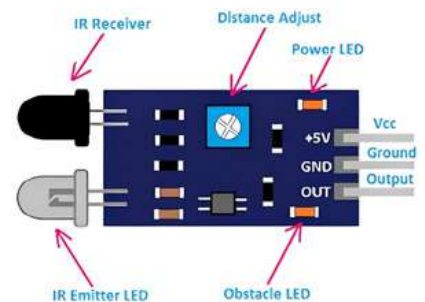
An **IR (Infrared) sensor** uses infrared light to detect objects or measure distance. It works by emitting IR light from an IR LED and checking how much of it is reflected back by an object using a photodiode.

- If more light is reflected, the object is closer.
- If little or no light is reflected, the object is far or absent.

It has two main parts:

- Emitter (IR LED): Sends out infrared light.
- Receiver (Photodiode): Detects the reflected IR light.

IR sensors are widely used for obstacle detection, proximity sensing, and line-following robot



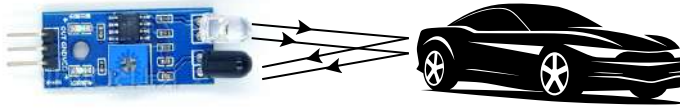
## Sensor Working

a)



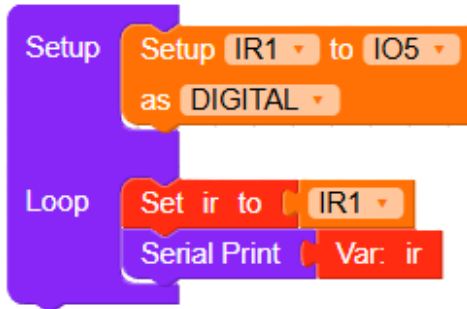
a) No object present – IR rays are not reflected back, so no detection.

b)



b) Object present – IR rays reflect back to the receiver, object detected.

## Block Code



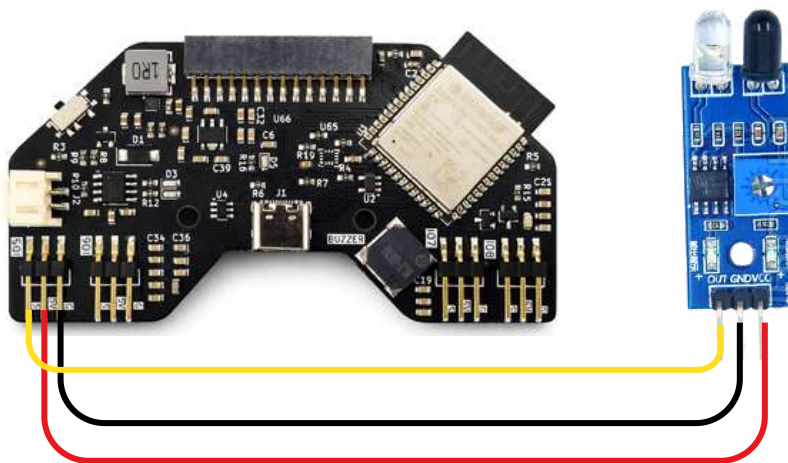
Video link on how to connect



## Pin Connection

| Board                                                                     | Sensor |
|---------------------------------------------------------------------------|--------|
| 5 V                                                                       | VCC    |
| G                                                                         | GND    |
| Select S(IO5) or any other available IO pin defined in the snippet block. | AO/DO  |

Check the IO pins you connected in the board and in the code snippet are same.

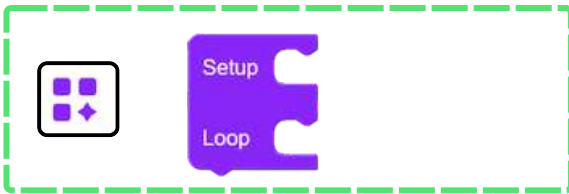


### PIN CONNECTIONS

- Power line
- DO
- Ground line



# Code explanation



## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



## Block 2:

Configure the IR sensor pin. Select any available GPIO code snippet and set it as DIGITAL using this block.



## Block 3:

This block **reads** the object detection status as 0's and 1's from the IR sensor.



## Block 4:

Create a **variable** with variable name “ir”



## Block 5:

This block **stores** the **value of IR sensor** that can be reused for printing and other logical operations.



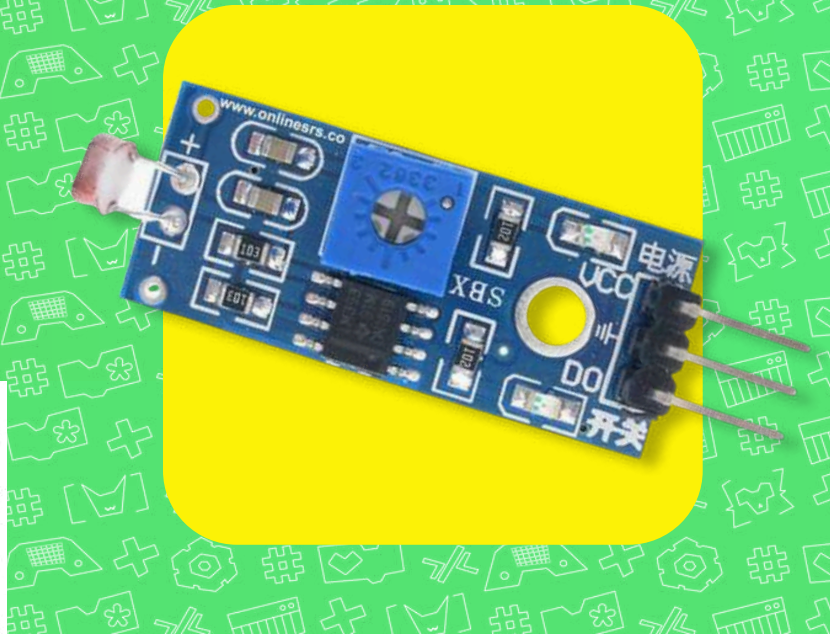
## Block 6:

Use **Serial Print** block with variable block to display IR sensor values on the **Serial Monitor**.

# LDR sensor

## Models Compatible

LM393 based



An **LDR (Light Dependent Resistor)** is a sensor whose resistance changes based on the amount of light falling on it. In bright light, its resistance becomes very low, allowing more current to pass. In darkness, its resistance becomes very high, restricting current flow. By measuring this change in resistance, a microcontroller can determine the light level .

### Working Principle:

The sensor works on the principle of photoconductivity the conductivity of certain materials increases when they are exposed to light..

**In Fig.1** When light falls on the LDR, its resistance decreases, allowing more current to flow.

**In Fig.2** In darkness, the LDR's resistance increases, reducing the current flow.

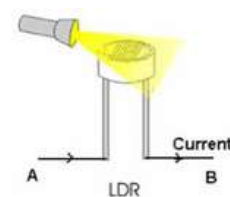
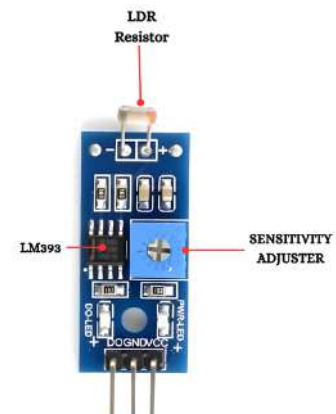


Fig.1

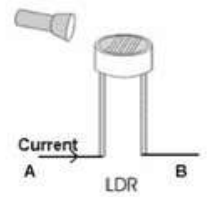


Fig.2

## Sensor Working

a)



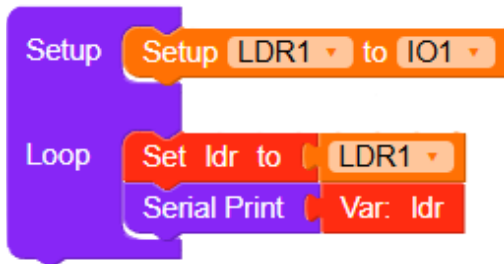
When exposed to light, the LDR output decreases (e.g., closer to 0)

b)



When exposed to darkness,  
the output increases (e.g.,  
closer to 4095).

## Block Code



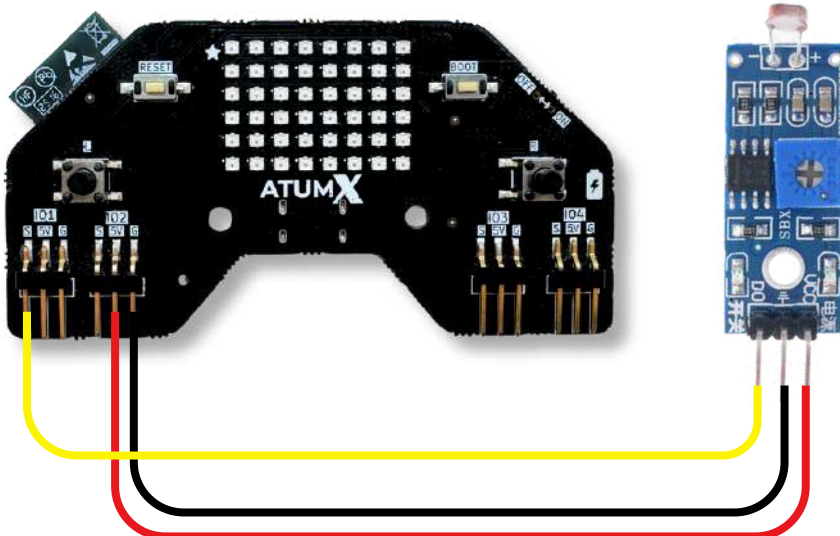
Video link on how  
to connect



## Pin Connection

| Board                                                                            | Sensor |
|----------------------------------------------------------------------------------|--------|
| In Board, connect to<br>any 5V Pins                                              | VCC    |
| Connect to any<br>Ground pins                                                    | GND    |
| Connect to IO1 or<br>any available GPIO<br>Pin defined in the<br>snippet block . | AO/DO  |

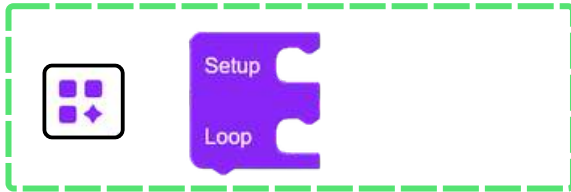
Check the IO pins you  
connected in the board and in  
the code snippet are same.



### PIN CONNECTIONS

- Power line
- DO
- Ground line

# Code explanation



## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



## Block 2:

Use the **Setup LDR** block to assign the LDR sensor to any **GPIO** pin on the board.



## Block 3:

Use the **LDR** block to read the light intensity detected by the Light Dependent Resistor



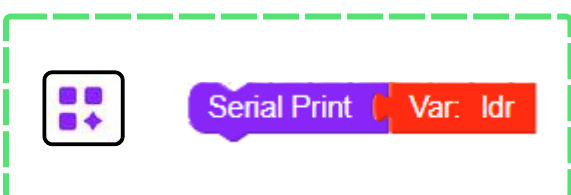
## Block 4:

Create a **variable** with variable name **“ldr”**



## Block 5:

This block **stores** the value of LDR sensor in the variable **“ldr”** that can be reused for printing and other logical operations.



## Block 6:

Use **Serial Print** block attached with variable block to print LDR sensor values on the **Serial Monitor**.



# Gas Sensor

## Models Compatible

MQ-2



A **Gas Sensor** detects the presence and concentration of gases in the air. It works by changing its electrical resistance when it comes in contact with certain gases. The sensor has a sensing element (usually made of metal oxide like  $\text{SnO}_2$ ) that reacts chemically with gas molecules. This reaction changes the current flowing through the sensor.

It has two main parts:

- Sensing Element: Detects the gas by reacting with its molecules.
- Circuit Module: Converts the change in resistance into an electrical signal that can be measured

### Working principle:

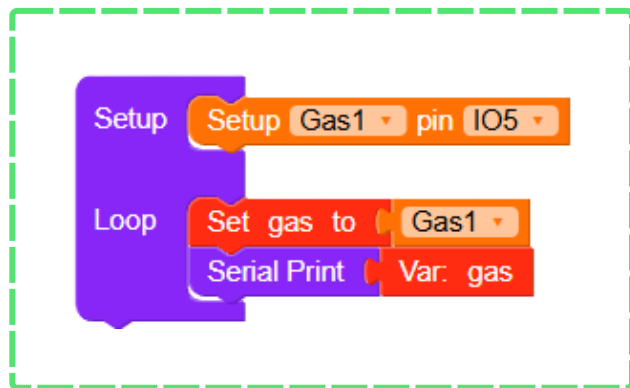
- When no gas – resistance of the sensing material remains normal.
- When gas is present – the gas molecules react with the sensing surface, changing its resistance.
- The circuit detects this change and sends a corresponding output signal.

## Sensor Working



When gas is present, sensor reacts and produces an electrical signal.

# Block Code



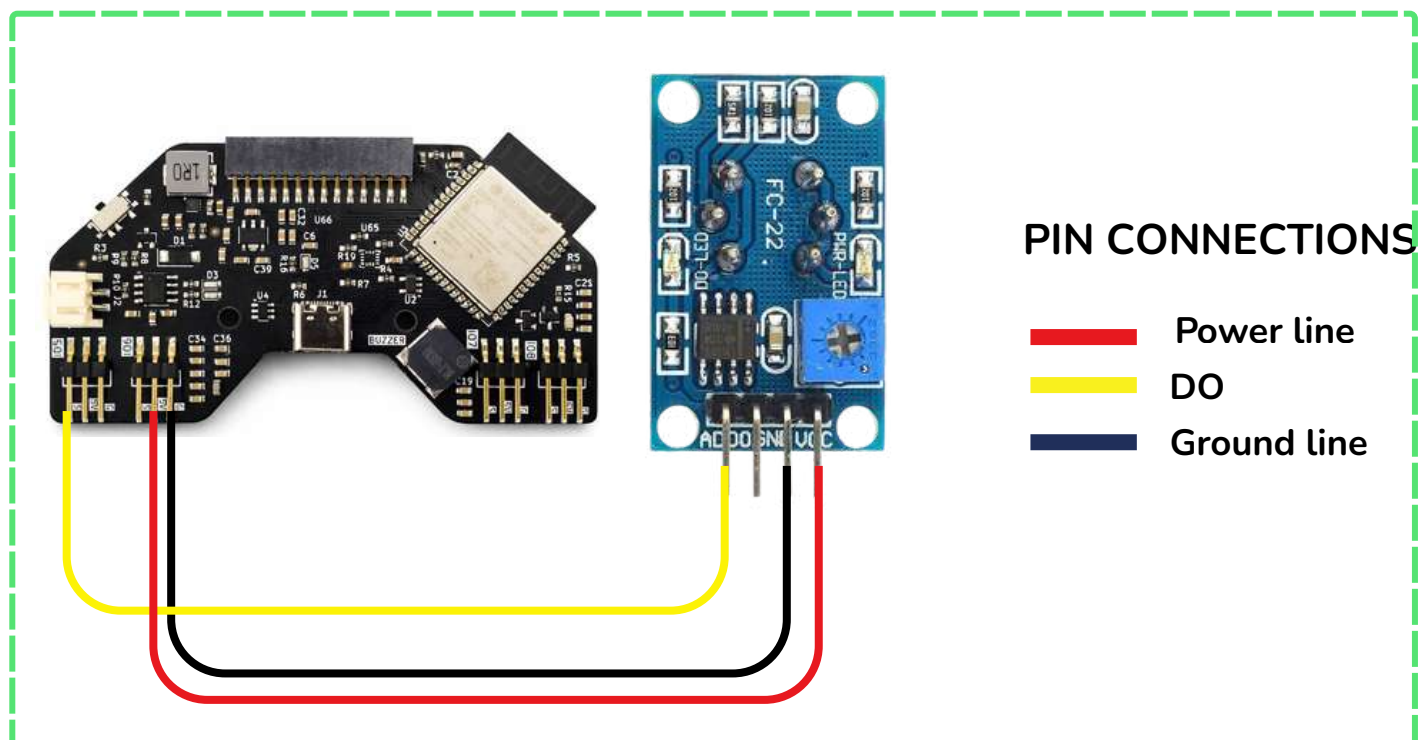
Video link on how to connect



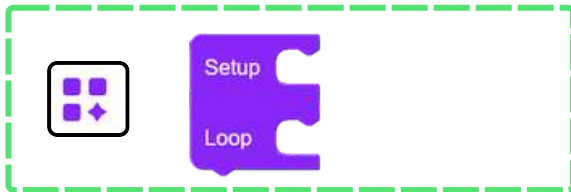
# Pin Connection

| Board                                                                   | Sensor |
|-------------------------------------------------------------------------|--------|
| In Board, connect to any 5V Pins                                        | VCC    |
| Connect to any Ground pins                                              | GND    |
| Connect to IO5 or any available GPIO Pin defined in the snippet block . | AO/DO  |

Check the IO pins you connected in the board and in the code snippet are same.



# Code explanation



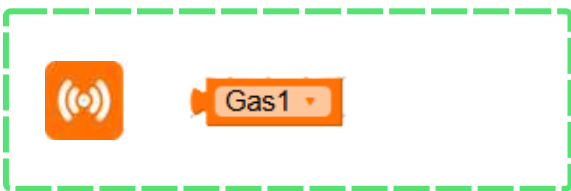
## Block 1:

From **Basics**, drag the **Setup Loop** block. Code in **Setup** runs once at the start, while code in **Loop** runs continuously.



## Block 2:

Use the **Set Gas** block to assign the gas sensor to any **GPIO pin** on the board.



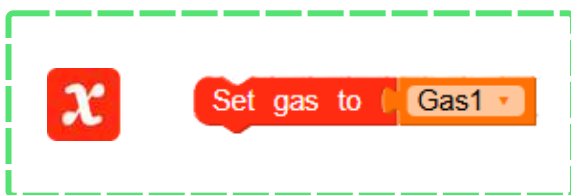
## Block 3:

This gas block is used to **read the output** from the gas sensor



## Block 4:

Create a **variable** with variable name **“gas”**



## Block 5:

This block **stores the measured** value in the variable **“gas”**, allowing it to be reused for other logical operations.



## Block 6:

Use **Serial Print** block attached with variable block to print gas sensor values on the **Serial Monitor**.



# Touch Sensor

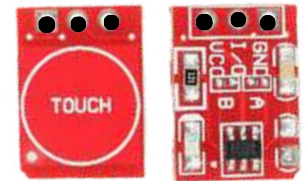
## Models Compatible

TTP223B : 1 Channel (blue)

TTP223R : 1 Channel (red)



A **Touch Sensor** detects physical contact or touch on its surface. It works by sensing a small change in capacitance or resistance when a human finger touches it. When you touch the sensor, it generates an electrical signal that can be used to trigger an output device like an LED or buzzer.



### Working principle:

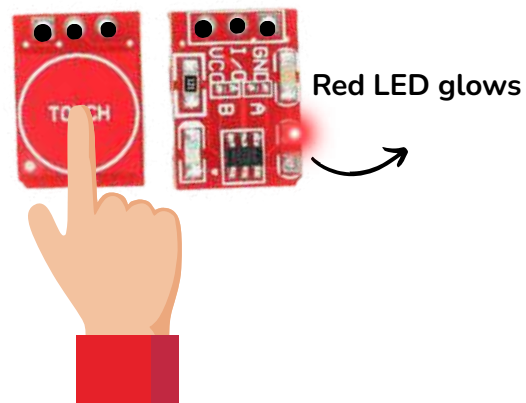
- When no touch – the circuit remains in its normal state.
- When touched – the human body (which conducts electricity slightly) changes the capacitance, and the circuit detects this change as a “touch.”

## Sensor Working

When no touch, LED does not glow.

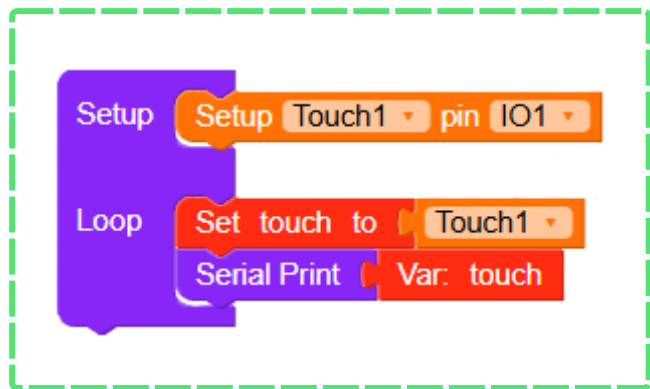


When touched, LED glows.





# Block Code



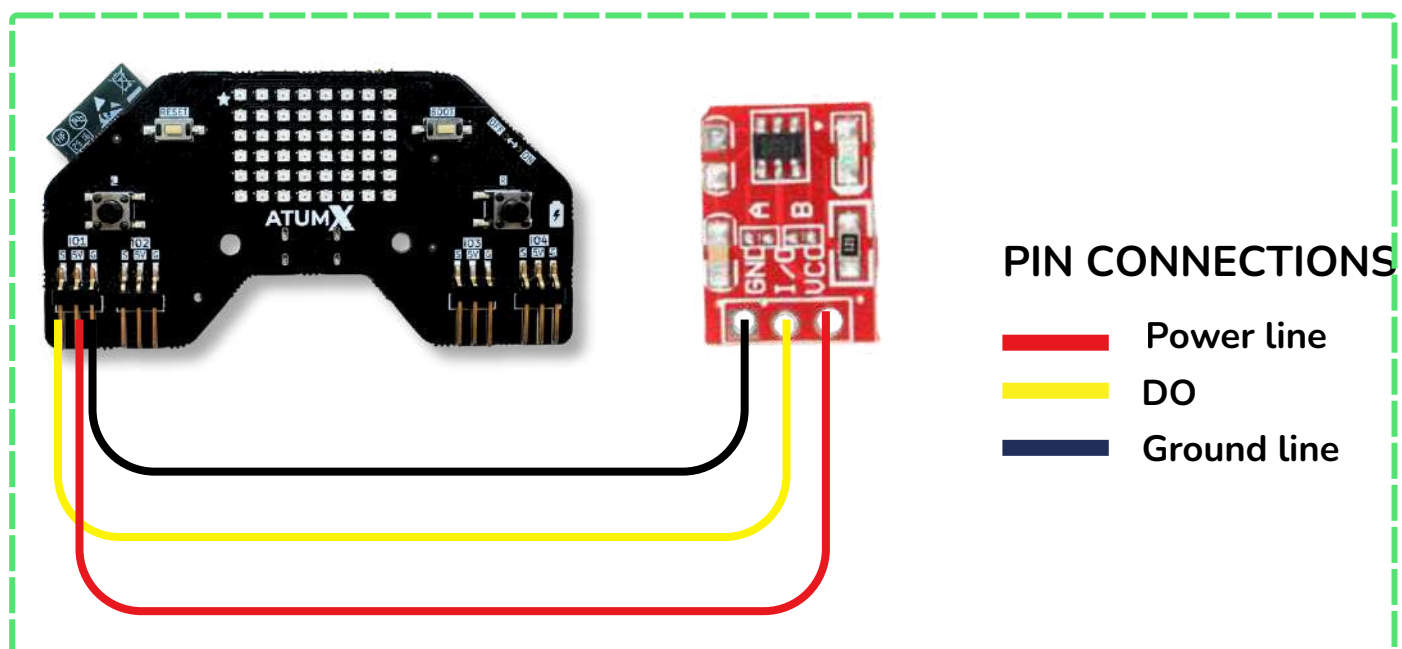
Video link on how to connect



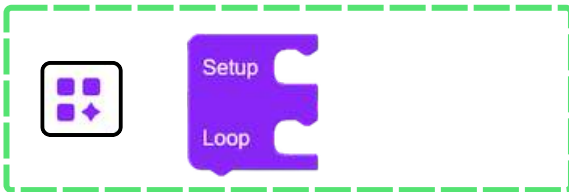
# Pin Connection

| Board                                                                   | Sensor |
|-------------------------------------------------------------------------|--------|
| In Board, connect to any 5V Pins                                        | VCC    |
| Connect to any Ground pins                                              | GND    |
| Connect to IO1 or any available GPIO Pin defined in the snippet block . | AO/DO  |

Check the IO pins you connected in the board and in the code snippet are same.



# Code explanation



## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs once at the start, while code in **Loop** runs continuously.



## Block 2:

Use the **Set touch pin** block to assign the touch sensor to any **GPIO pin** on the board.



## Block 3:

This block **reads** the value from the touch sensor.



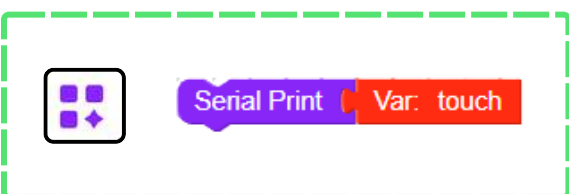
## Block 4:

Create a **variable** with variable name **“touch”**



## Block 5:

This block stores the touch value in the variable **“touch”** that can be used for further logical operations.



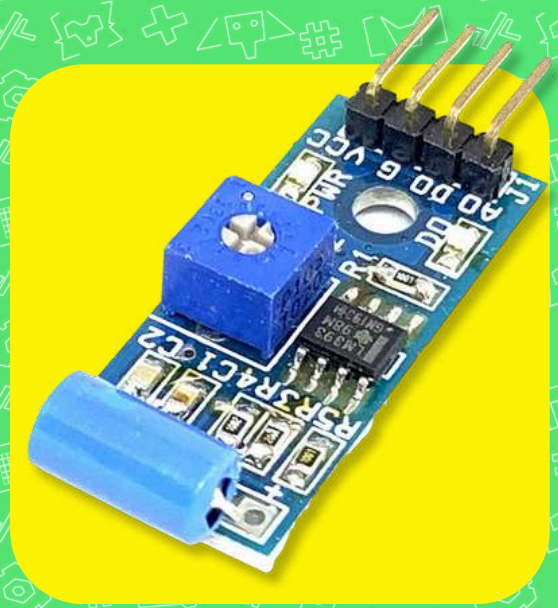
## Block 6:

Use **Serial Print** block attached with variable block to print gas sensor values on the **Serial Monitor**.

# Vibration Sensor

## Models Compatible

SW-420



A **Vibration Sensor** detects vibrations, shocks, or movements from surfaces or objects. It converts mechanical vibration into an electrical signal that can be measured by a microcontroller or circuit. It is often used to detect motion, impacts, or mechanical faults in machines.

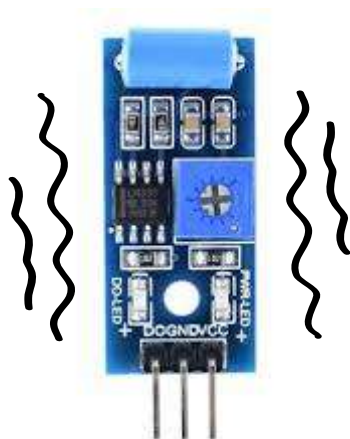
### Main Parts:

- Sensing Element (Spring or Piezoelectric Material): Detects vibration or movement.
- Signal Conditioning Circuit: Converts the vibration into a readable electrical signal.

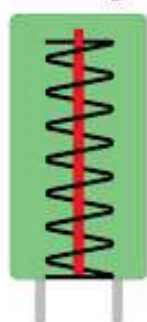
### Working Principle:

- When no vibration – the sensor remains in its normal state and output stays LOW.
- When vibration occurs – mechanical movement causes a change in voltage (in a piezoelectric type) or closes a contact (in a spring type).
- The circuit detects this change and sends an output signal (HIGH) to indicate vibration.

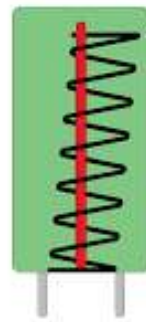
## Sensor Working



No vibration



Vibration occurs



Vibration causes spring to move and close the contact, generating a signal.

# Block Code

```
Setup
  Setup Vibration1 pin IO1

Loop
  Set vibe to Vibration1
  Serial Print Var: vibe
```

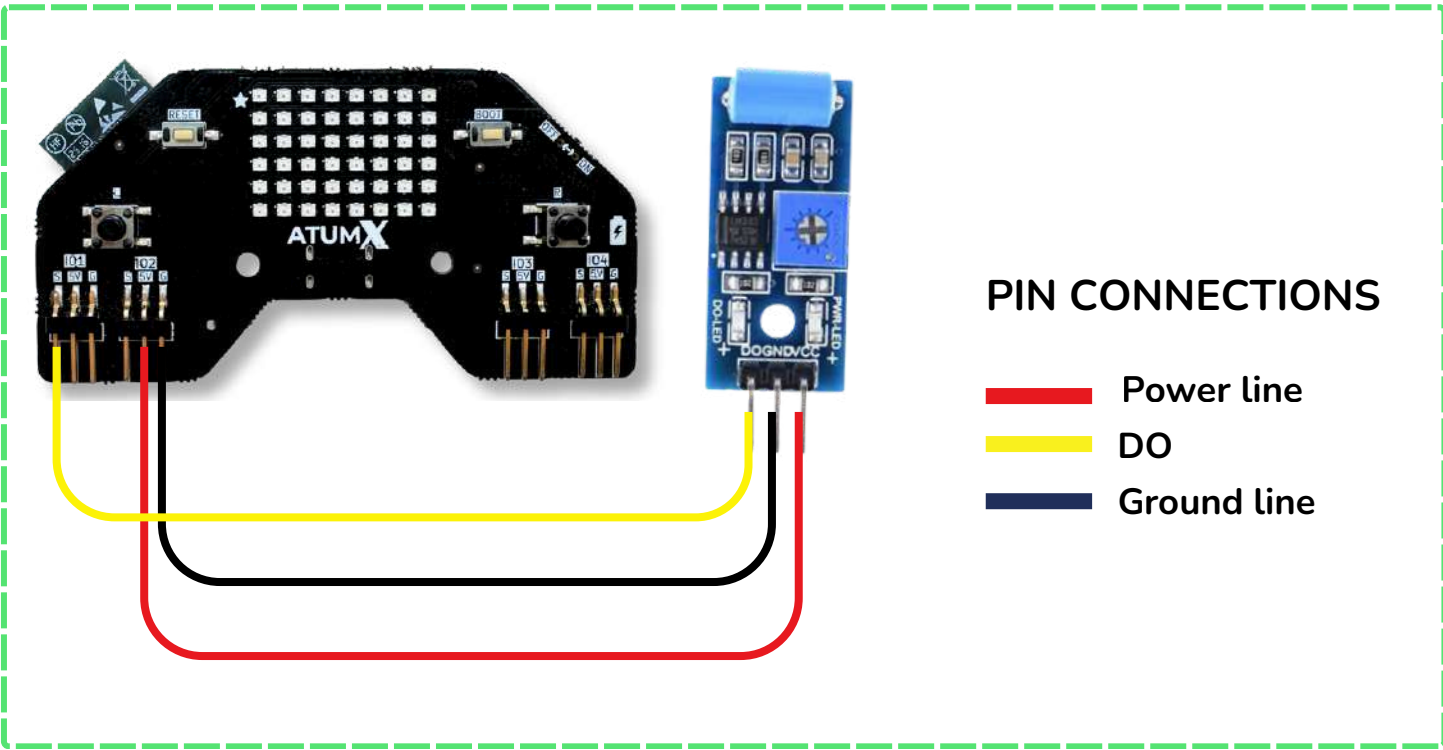
Video link on how to connect



# Pin Connection

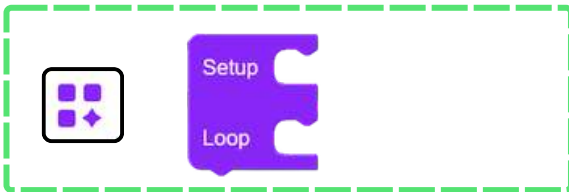
| Board                                                                   | Sensor |
|-------------------------------------------------------------------------|--------|
| In Board, connect to any 5V Pins                                        | VCC    |
| Connect to any Ground pins                                              | GND    |
| Connect to IO1 or any available GPIO Pin defined in the snippet block . | AO/DO  |

Check the IO pins you connected in the board and in the code snippet are same.





# Code explanation



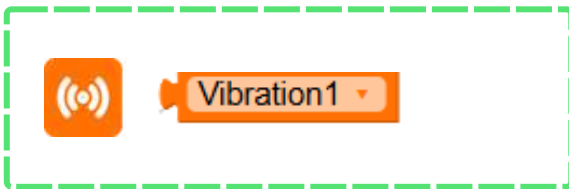
## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



## Block 2:

Use the **Setup Vibration pin** block to assign the vibration sensor to any GPIO pin on the board.



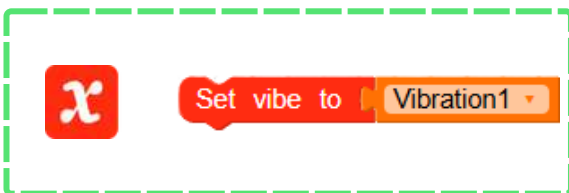
## Block 3:

This block is used to **read** the value that are measured by the vibration sensor.



## Block 4:

Create a **variable** with variable name **“vibe”**



## Block 5:

This block **stores** the measured values from the vibration sensor and is stored in the variable **vibe**.



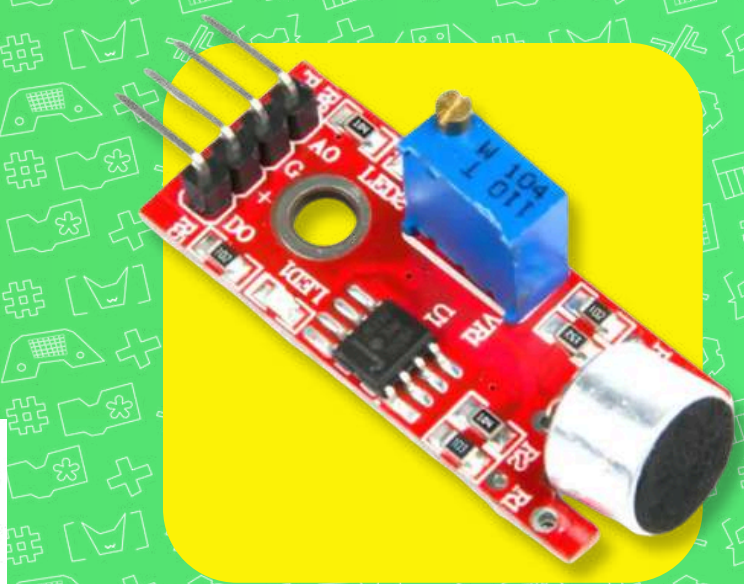
## Block 6:

Use **Serial Print** block attached with variable block to print touch sensor values on the **Serial Monitor**.

# Sound Sensor

## Models Compatible

LM393 op-amp



A Sound Sensor is used to detect sound vibrations, especially sharp sounds like claps, and convert them into electrical signals that can be read by a microcontroller or used to trigger circuits (like turning ON lights, motors, etc.).

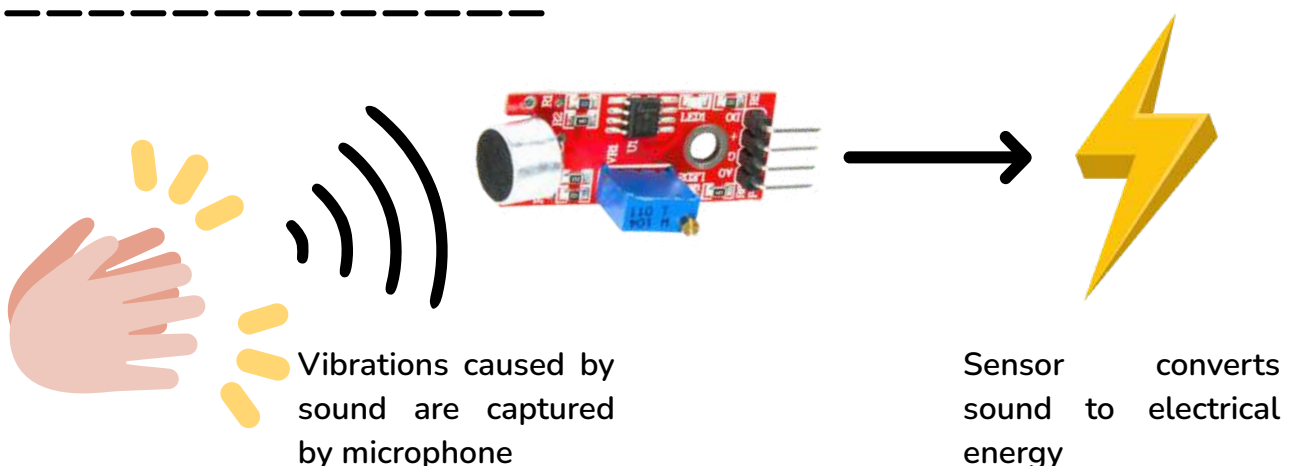
### Main Parts:

- Microphone: Detects sound waves and converts them into weak electrical signals.
- Amplifier: Strengthens the weak audio signals.
- Signal Processing Unit: Checks if the sound crosses a set threshold and outputs a clean ON/OFF pulse.

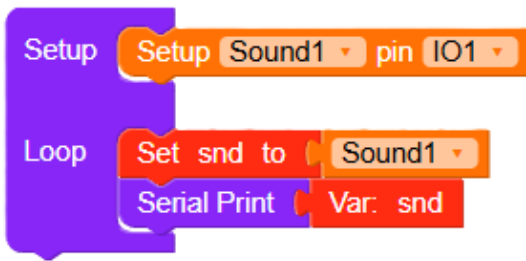
### Working Principle:

- A sound creates vibrations that the microphone converts into a small signal.
- The amplifier boosts this signal, and the processing unit compares it to a threshold.
- If the sound is loud and sharp enough, the circuit outputs a digital HIGH pulse to trigger devices like LEDs or relays.

## Sensor Working



# Block Code



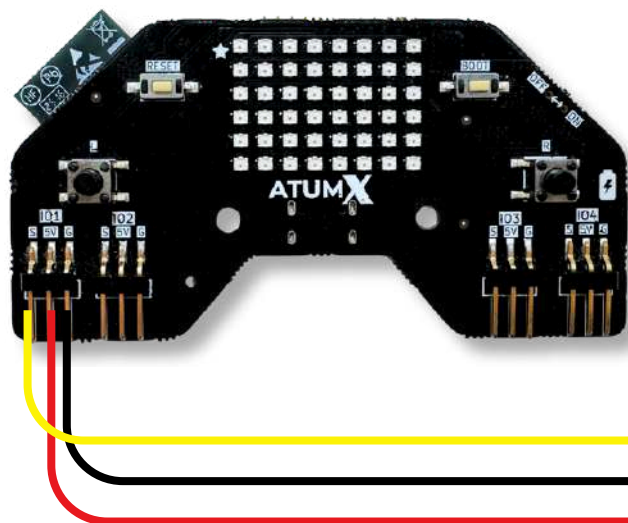
Video link on how to connect



# Pin Connection

| Board                                                                   | Sensor |
|-------------------------------------------------------------------------|--------|
| In Board, connect to any 5V Pins                                        | VCC    |
| Connect to any Ground pins                                              | GND    |
| Connect to IO1 or any available GPIO Pin defined in the snippet block . | AO/DO  |

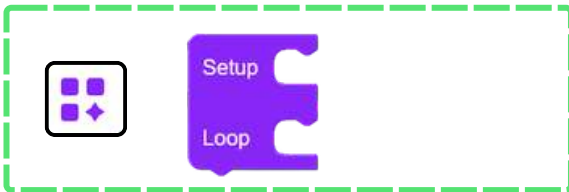
Check the IO pins you connected in the board and in the code snippet are same.



## PIN CONNECTIONS

- Power line
- DO
- Ground line

# Code explanation



## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs once at the start, while code in **Loop** runs continuously.



## Block 2:

Use the **Set Sound** block to assign the sound sensor to any **GPIO pin** on the board.



## Block 3:

This sound block is used to read and **collect the values** from the Sound Sensor.



## Block 4:

Create a **variable** with variable name **"snd"**



## Block 5:

This block **stores** the measured sound values collected by the sound sensor and stores it in a variable name **"snd"**.



## Block 6:

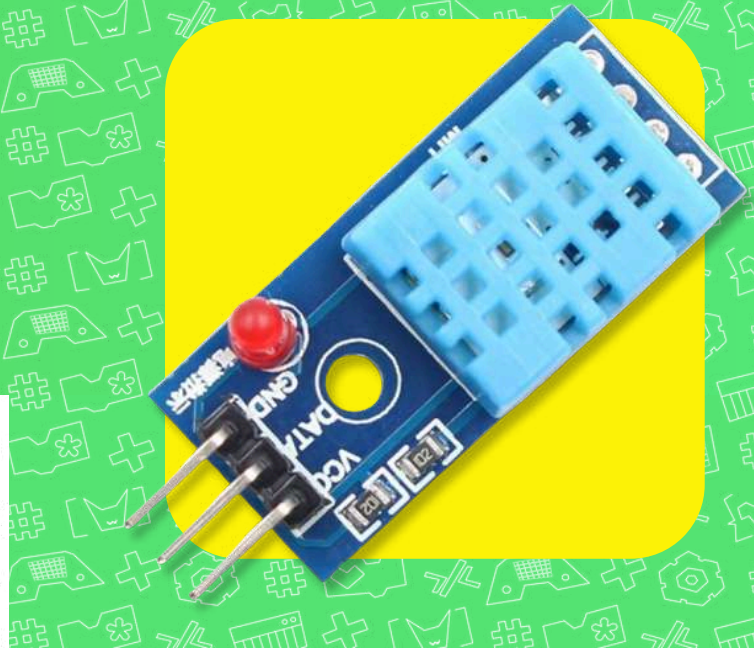
Use **Serial Print** block attached with variable block to print Sound Sensor values on the **Serial Monitor**.



# DHT11

## Models Compatible

DHT11



The DHT11 Sensor is used to measure temperature and humidity in the surrounding environment. It provides digital output signals that can be directly read by a microcontroller . The sensor contains both a humidity sensing component and a temperature measuring component inside a single module.

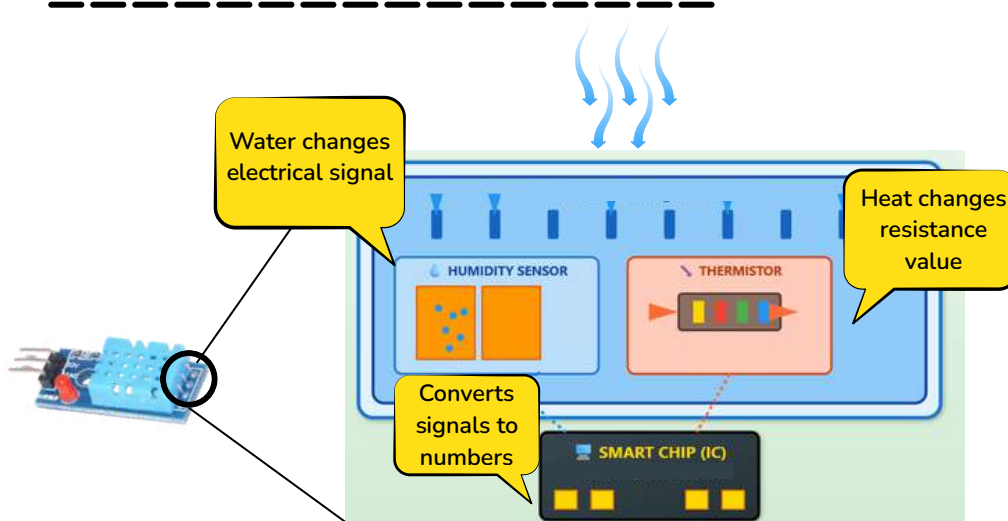
### Main Parts:

- Humidity Sensor: Measures the moisture content in the air.
- Temperature Sensor (Thermistor): Measures the surrounding temperature.
- Signal Processing Unit: Converts the readings into digital data for easy interfacing.

### Working Principle:

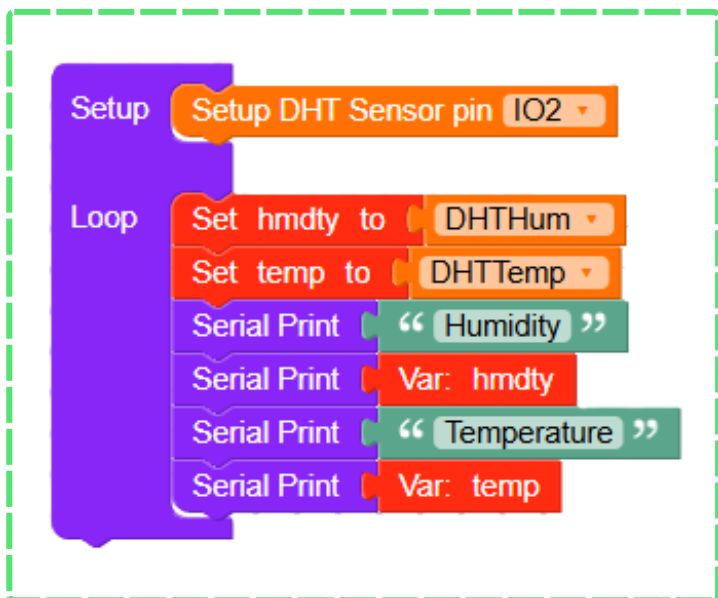
- The humidity sensor uses a moisture-holding substrate that changes electrical resistance with humidity.
- The thermistor changes resistance with temperature.
- The sensor processes both signals and sends the combined data as a digital output.

## Sensor Working



A DHT11 works by sensing air moisture through a humidity-sensitive material and temperature through an internal resistor, then sending the data digitally to a microcontroller.

# Block Code



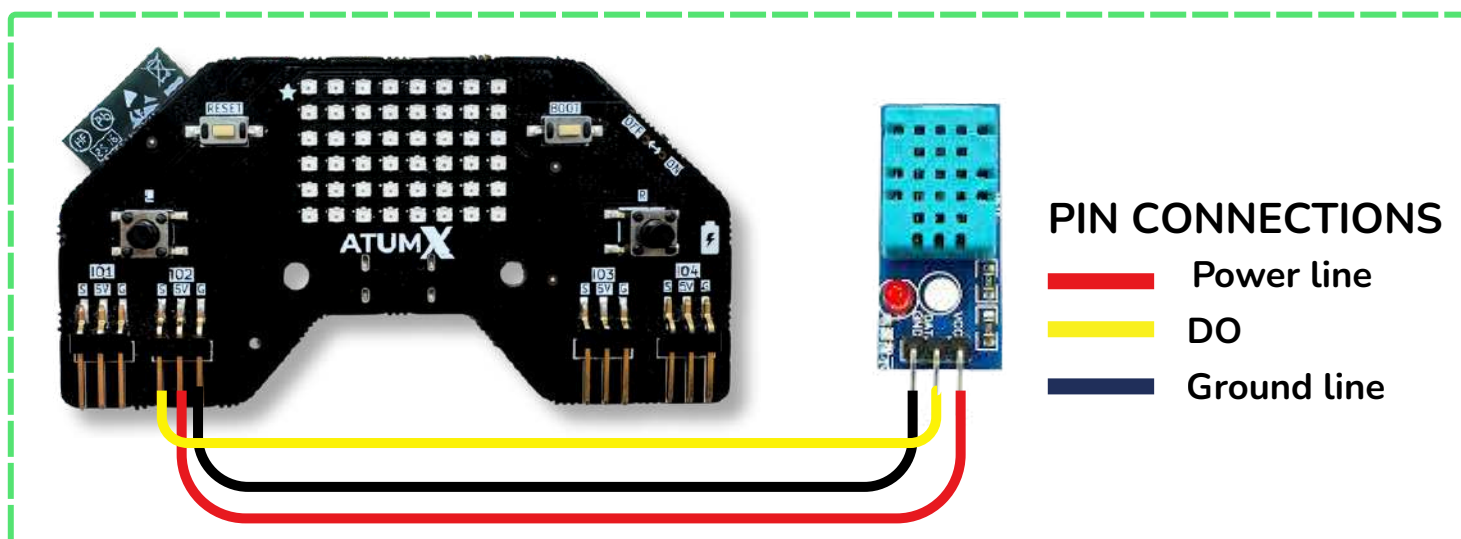
# Pin Connection

| Board                                                                   | Sensor |
|-------------------------------------------------------------------------|--------|
| In Board, connect to any 5V Pins                                        | VCC    |
| Connect to any Ground pins                                              | GND    |
| Connect to IO2 or any available GPIO Pin defined in the snippet block . | DATA   |

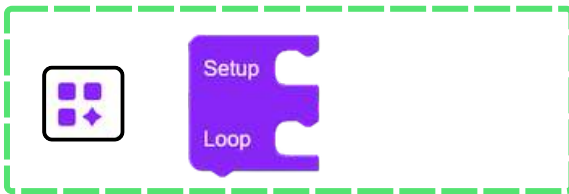
Video link on how to connect



Check the IO pins you connected in the board and in the code snippet are same.



# Code explanation



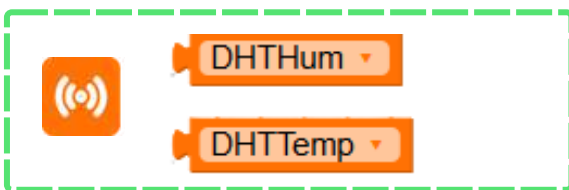
## Block 1:

From **Basics**, drag the **Setup Loop** block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



## Block 2:

Use the **Setup DHT Sensor pin** block to assign the DHT sensor to **any GPIO pin** on the board.



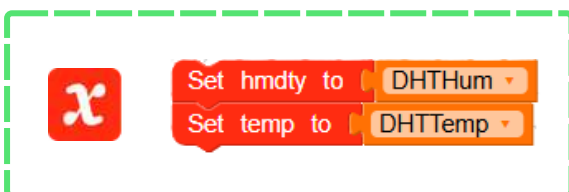
## Block 3:

These **Humidity** and **Temperature** blocks are used to read the output measured by the DHT Sensor



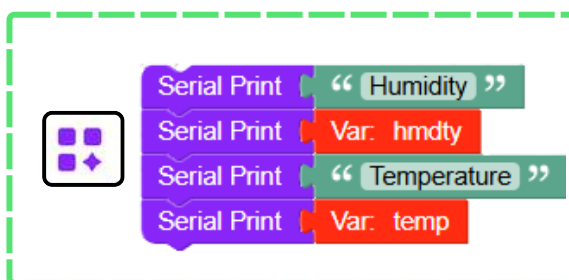
## Block 4:

Create **two variables** with variable name **“hmdty”** and **“temp”** respectively



## Block 5:

The following blocks store the temperature and humidity values in the respective variables that can be used for further logical operations and printing.



## Block 6:

Use **Serial Print** block attached with the **Humidity** and **temperature** variable blocks to print DHT Sensor values on the **Serial Monitor**.



# Relay Module - 2

## Models Compatible

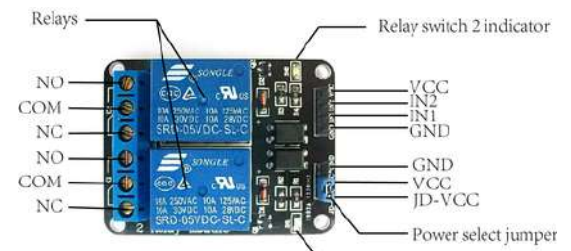
Optocoupler based relay



A 2-Relay Module lets you control two high-power devices (like bulbs or motors) using a small control signal from Arduino, ESP32, or Raspberry Pi. It works like an electronic ON/OFF switch.

### Main Parts :

- Relay Coils (×2): Create magnetism to open/close the switches.
- Transistor Driver: Boosts the weak control signal.
- Flyback Diode: Protects the circuit from voltage spikes.
- Screw Terminals: Connect the AC/DC load.



### Working:

When the control pin gets a signal, the relay coil energizes and pulls the contacts, turning the connected device ON or OFF. Removing the signal makes the relay go back to its normal state.

## Sensor Working



When signal applied to IN1, first relay coil energizes and current passes through the connected load. The other relay will not get energized





When signal applied to IN2, second relay coil energizes and current passes through the connected load. The other relay will not get energized.

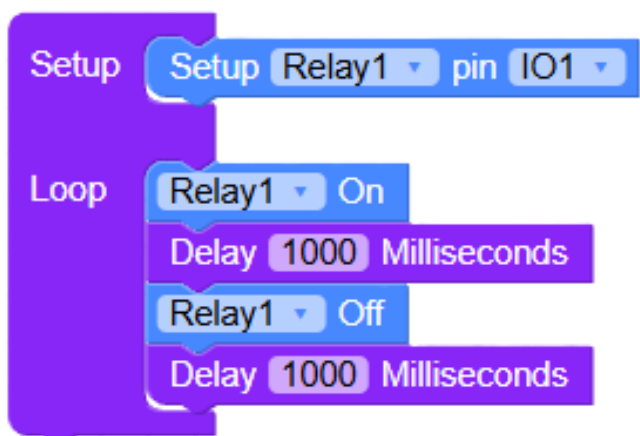
Video link on how to connect



## Pin Connection

| Board                                                                                                | Sensor |
|------------------------------------------------------------------------------------------------------|--------|
| In Board, connect to any 5V Pins                                                                     | VCC    |
| Connect to any Ground pins                                                                           | GND    |
| Connect to <b>IO1</b> and <b>IO2</b> or any <b>available GPIO Pin</b> defined in the snippet block . | IN1    |

## Block Code



Check the IO pins you connected in the board and in the code snippet are same.





## PIN CONNECTIONS

- Power line
- Relay
- Ground line

# Code explanation



Setup  
Loop

## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs once at the start, while code in **Loop** runs continuously.



Setup Relay1 pin IO1

## Block 2:

Configure the relay using this block to pins IO1 and IO2.



Relay1 On

Relay1 Off

## Block 3:

Use the **Relay** block to turn on or off the relay.



Delay 2000 Milliseconds

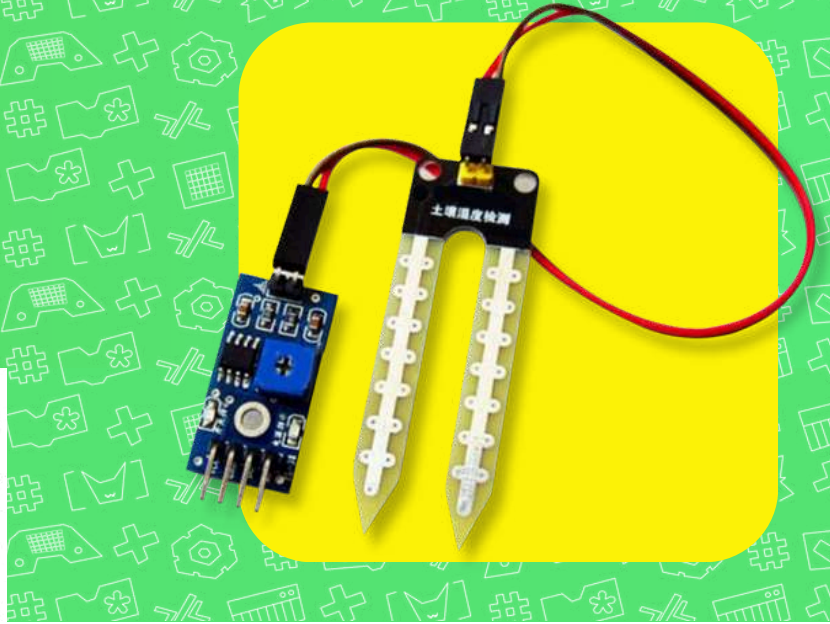
## Block 4:

Use the **Delay** block to pause the execution for a while.

# Soil moisture sensor

## Models Compatible

FC-28 with LM293,  
Capacitance based soil sensor



The Soil Moisture Sensor is used to measure how wet or dry the soil is. It helps microcontrollers like Arduino know when plants need water. The sensor provides an analog or digital output based on the moisture level in the soil.

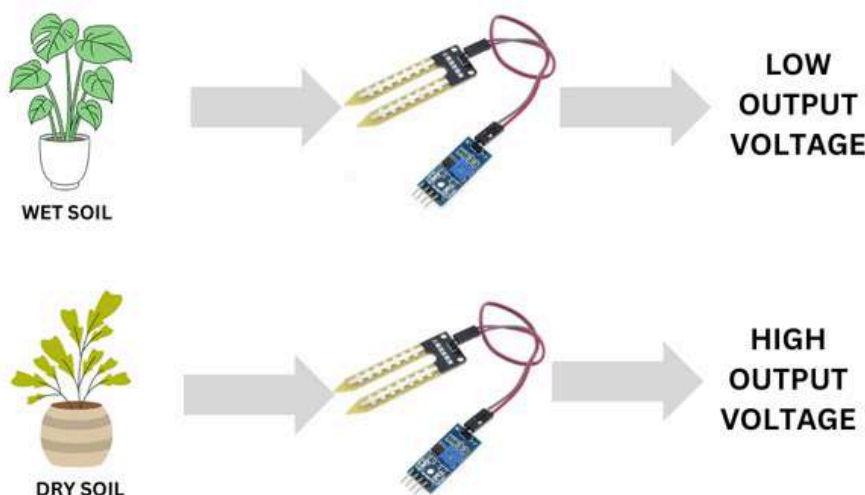
### Main Parts:

- Probes/ Electrodes: Detect the water content in the soil.
- Comparator / Op-Amp (on module): Converts the probe reading into analog or digital signals.
- Indicator LEDs: Show whether soil is wet or dry.

### Working Principle:

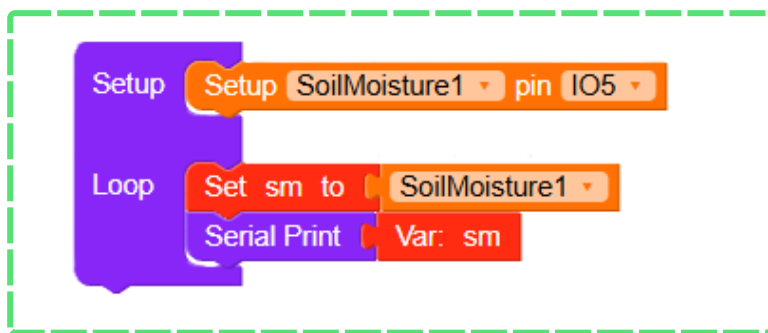
- Water in the soil increases conductivity between the probes.
- Dry soil has high resistance; wet soil has low resistance.
- The sensor reads this difference and outputs a signal that tells how much moisture is present.

## Sensor Working



A soil moisture sensor works by measuring the electrical resistance between its probes, which changes based on how much water is present in the soil.

# Block Code



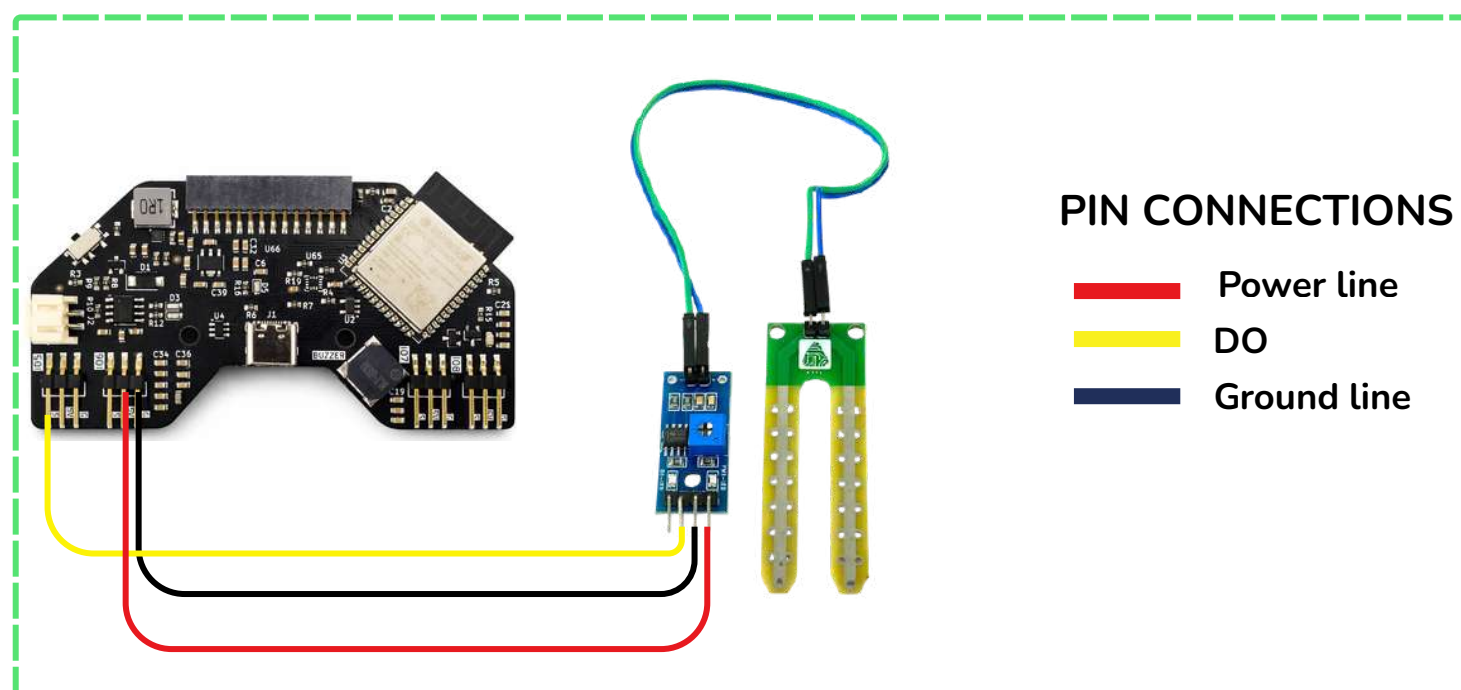
Video link on how to connect



# Pin Connection

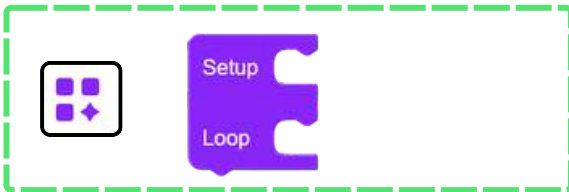
| Board                                                                   | Sensor |
|-------------------------------------------------------------------------|--------|
| In Board, connect to any 5V Pins                                        | VCC    |
| Connect to any Ground pins                                              | GND    |
| Connect to IO5 or any available GPIO Pin defined in the snippet block . | AO/DO  |

Check the IO pins you connected in the board and in the code snippet are same.





# Code explanation



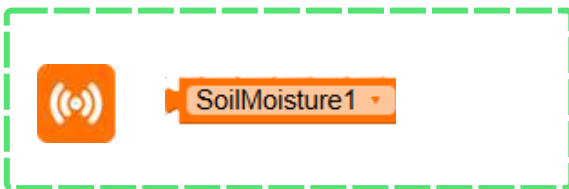
## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



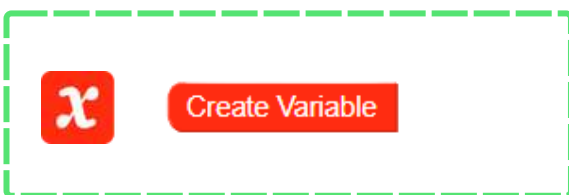
## Block 2:

Configure the soil moisture sensor pins to the IO5 pin of the **SUBO** Board.



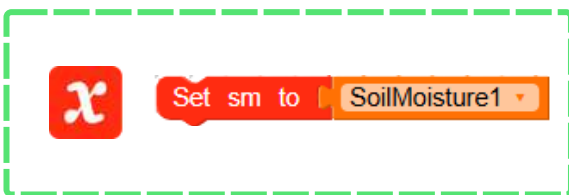
## Block 3:

This block **reads the soil wetness or dryness** values from the Soil moisture sensor.



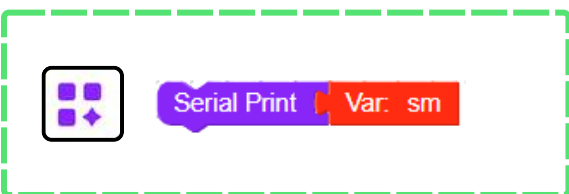
## Block 4:

Create a **variable** with variable name “**sm**” to store the values from Soil Moisture sensor.



## Block 5:

This block is used to store the values collected from the soil moisture sensor into the variable “**sm**” and can be used for further logical operations or printing.



## Block 6:

Use **Serial Print** block attached with variable block to print Soil Moisture Sensor values on the **Serial Monitor**.

# LCD Display

## Models Compatible

16x2 LCD1602 Display with I2C



The LCD (16x2) Display is used to show text, numbers, or symbols in electronic projects. It can display 16 characters per line and has 2 lines, making it useful for messages, sensor values, and menus.

### Main Parts:

- a. LCD Panel: The screen where characters appear.
- b. Controller Chip (HD44780): Converts microcontroller commands into characters on the display.
- c. Data Pins (D0–D7): Carry the information to be shown.
- d. Control Pins (RS, RW, EN): Manage how data is sent.
- e. Backlight & Contrast Control: Help make the text visible.

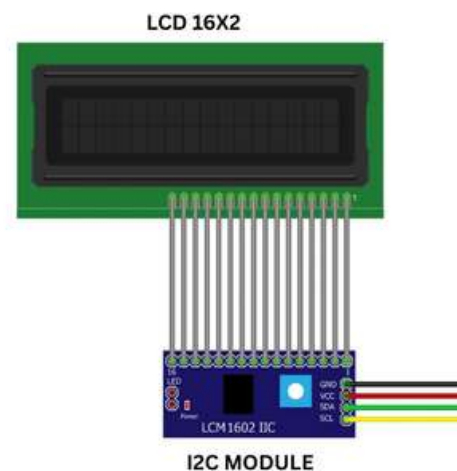
### Working Principle:

The microcontroller sends data and commands to the LCD through the data and control pins. The controller chip interprets these signals and updates the characters on the screen. Backlight and contrast settings help you clearly view the displayed text.

# Sensor Working



The board sends commands to LCD.



LCD displays the contents.

# Block Code

```
Setup
  Setup LCD SDA IO2 SCL IO1

Loop
  LCD - Print Hello
  Delay 1000 Milliseconds
  LCD - Clear
  LCD - SetCursor Column 3 Row 1
  LCD - Print Welcome
  Delay 1000 Milliseconds
  LCD - Clear
  LCD - Print This is an I2C LCD Display Scroll Direction Left
  Delay 1000 Milliseconds
  LCD - Clear
```

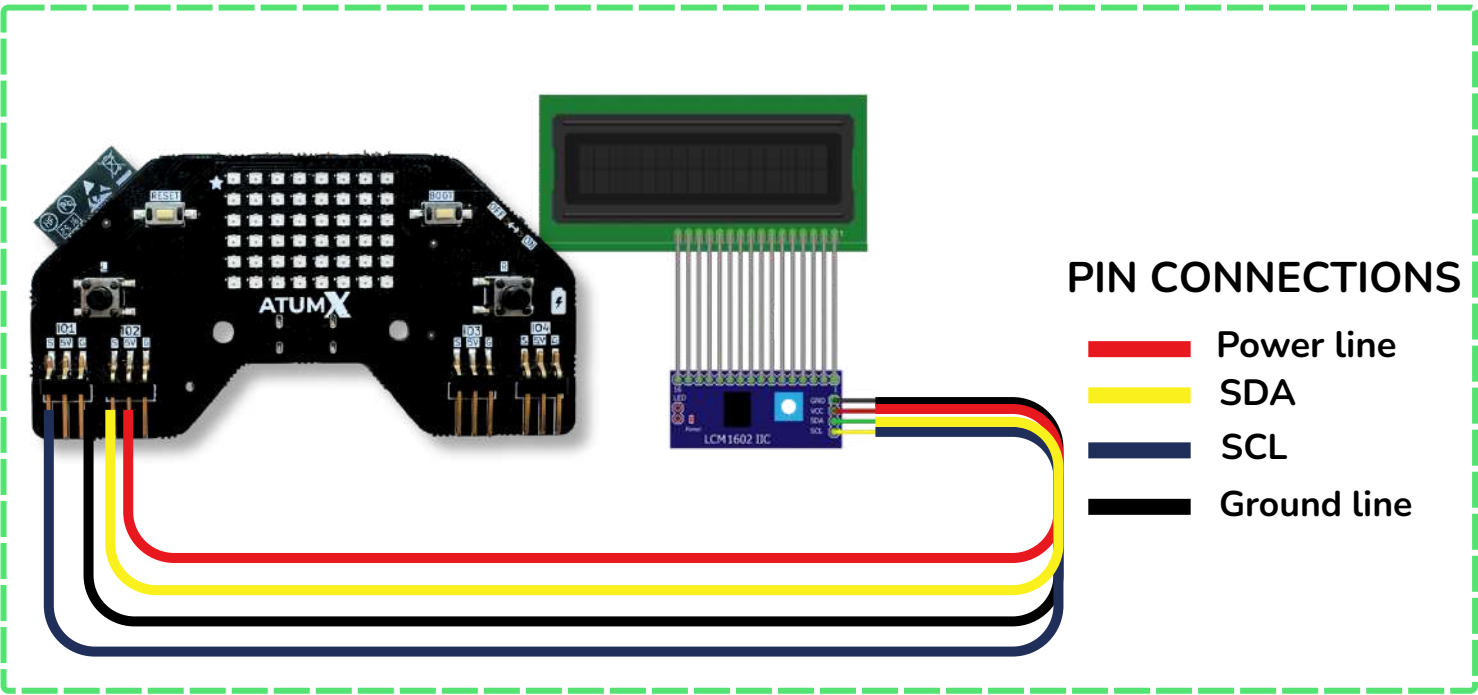
Video link on how to connect



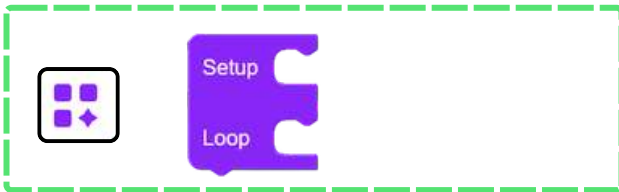
# Pin Connection

| Board                                                                         | Sensor |
|-------------------------------------------------------------------------------|--------|
| 5V                                                                            | VCC    |
| GND                                                                           | GND    |
| Connect to S(IO1) or any other available IO pin defined in the snippet block. | SCL    |
| Connect to S(IO2) or any other available IO pin defined in the snippet block. | SDA    |

Check the IO pins you connected in the board and in the code snippet are same.



# Code explanation



## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



## Block 2:

Configure the LCD pins. Select any **available GPIO** for using this block



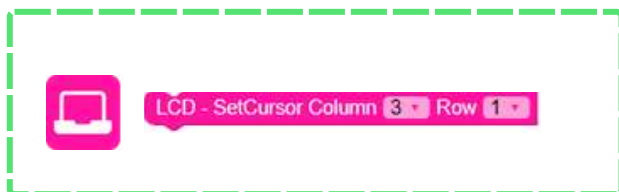
## Block 3:

This block displays **Hello** in LCD.



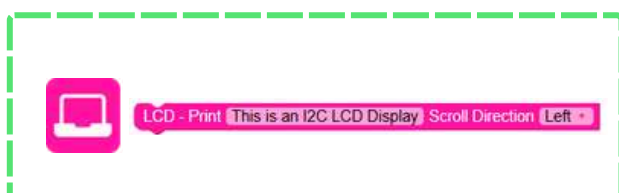
## Block 4:

Clears the LCD Screen



## Block 5:

Sets the **cursor** to a specific point on the LCD Display



## Block 6:

Displays the **text** in a **scrolling** manner.



# Colour sensor

## Models Compatible

TCS34725



The Color Sensor is used to detect the color of an object by measuring the amount of Red, Green, and Blue (RGB) light reflected from it. It sends this data to a microcontroller for color identification.

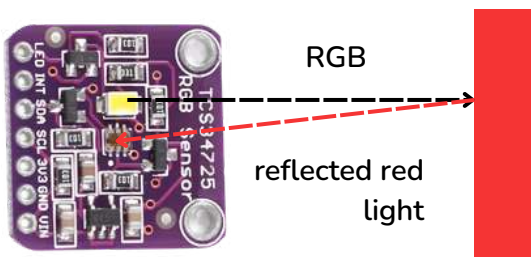
### Main Parts:

- RGB Filters: Allow only red, green, or blue light to pass.
- Photodiodes: Convert the filtered light into electrical signals.
- Onboard Converter (Frequency/ Digital Output): Converts light intensity into digital or frequency signals.
- LEDs (on module): Provide constant illumination for accurate color reading.

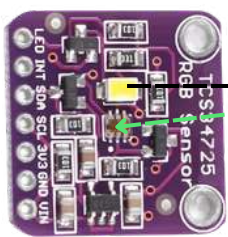
### Working Principle:

- The sensor shines light on an object and measures how much red, green, and blue light is reflected.
- Each color filter activates one set of photodiodes.
- The sensor processes these readings and sends the RGB values to the microcontroller to determine the object's color.

## Sensor Working

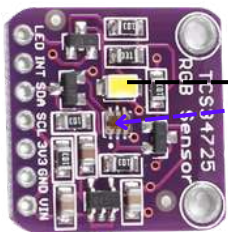


When RGB light flashes on the object, red gets reflected and other lights get absorbed. The amount of reflected red light determines the voltage produced by the sensor.



RGB

reflected  
green light



RGB

reflected  
blue light

When RGB light flashes on the object, green gets reflected and other lights get absorbed. The amount of reflected green light determines the voltage produced by the sensor.

When RGB light flashes on the object, blue gets reflected and other lights get absorbed. The amount of reflected blue light determines the voltage produced by the sensor.

Video link on how  
to connect



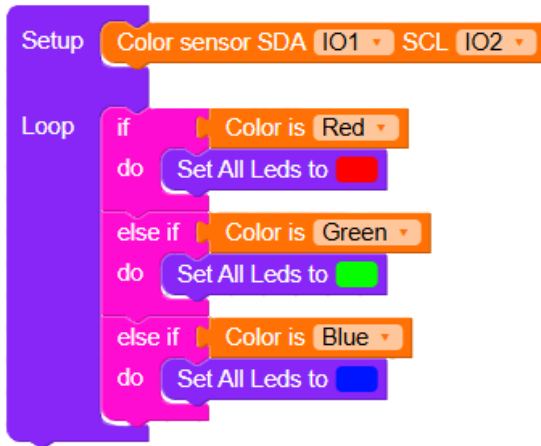
## Pin Connection

| Board                                                                                       | Sensor |
|---------------------------------------------------------------------------------------------|--------|
| 5V                                                                                          | VCC    |
| GND                                                                                         | GND    |
| Connect to <b>S(I01)</b> or any other <b>available IO pin</b> defined in the snippet block. | SDA    |
| Connect to <b>S(I02)</b> or any other <b>available IO pin</b> defined in the snippet block. | SCL    |

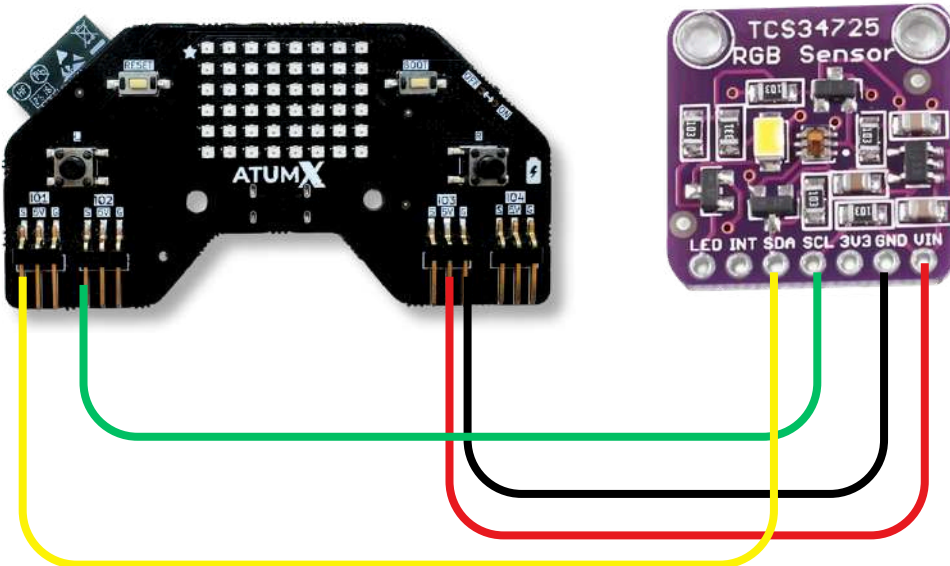
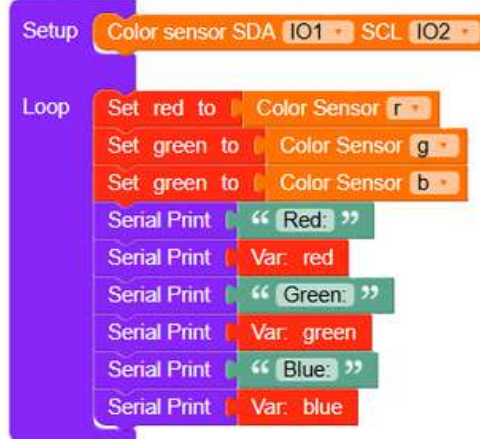
Check the IO pins you  
connected in the board and in  
the code snippet are same.

# Block Code

## Setup 1



## Setup 2

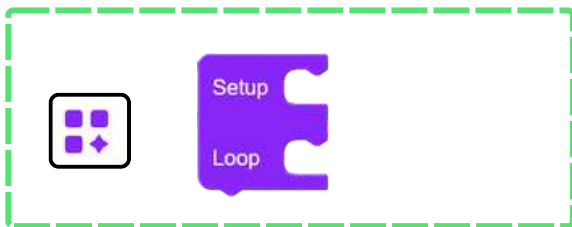


## PIN CONNECTIONS

- Power line
- SCL
- SDA
- Ground line

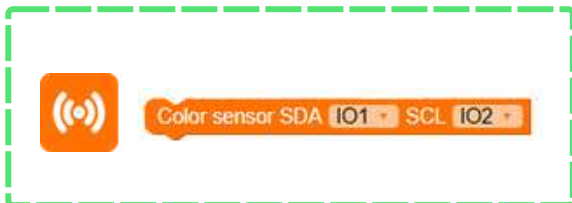


# Code explanation



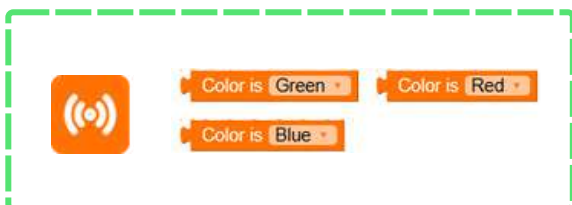
## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



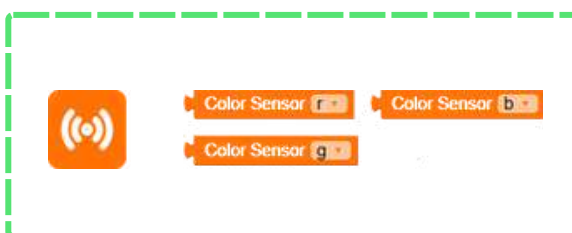
## Block 2:

Use the **Setup Color Sensor pin** to assign the Color sensor to **any GPIO pin** on the board.



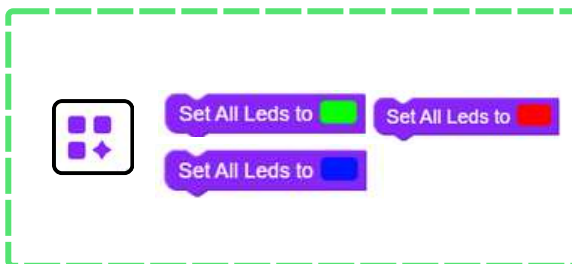
## Block 3:

These blocks are used to **detect respective colors**. They are used along with logical operations.



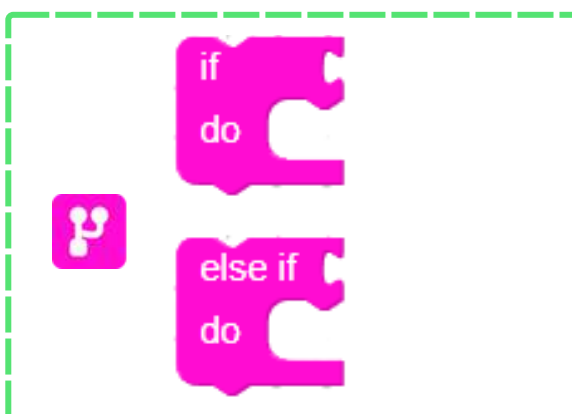
## Block 4:

These blocks are used to **collect the numerical value** of respective **colors** from the **color sensor** and can be used with loops, variables and logical operations when required.



## Block 5:

These blocks **sets the LED's** to the respective **color** selected.



## Block 6:

These blocks are used to **check for conditions** and are used along with logical operations.



# Code explanation



Create Variable

## Block 7:

Create a **variable** with variable name **red**, **green** and **blue** to store the values from **Color Sensor**.



Set red to Color is Red

Set green to Color is Green

Set blue to Color is Blue

## Block 8:

These blocks **store the values** that are collected from the **Color Sensor**.



Serial Print Var: red

Serial Print Var: green

Serial Print Var: blue

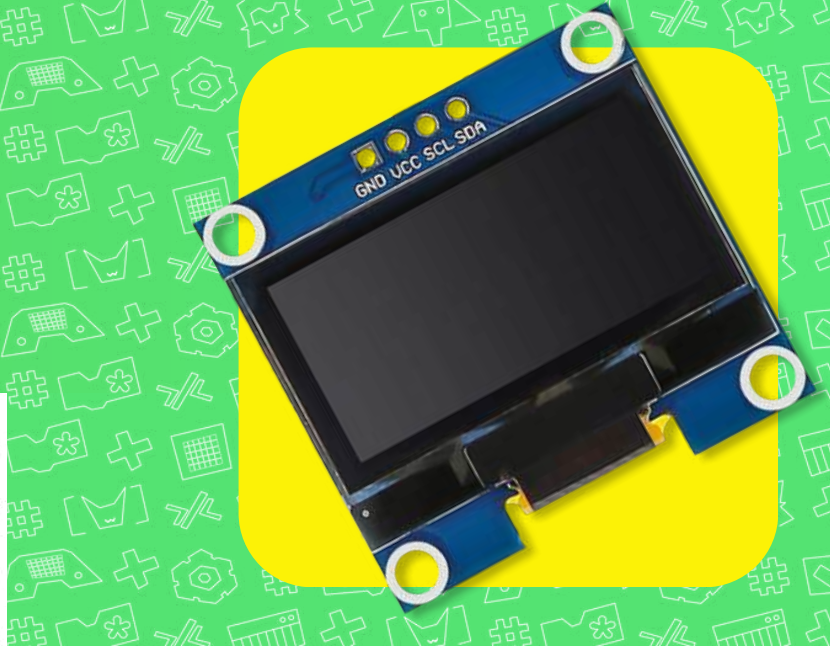
## Block 9:

These blocks use the **variable value** with the respective names and are used along with **print statements**.

# OLED Display

## Models Compatible

1.3 inch (I2C) OLED display



The OLED Display is a small screen that shows text, graphics, or symbols using tiny light-emitting pixels. It does not need a backlight, so it gives bright, clear output even in low light.

### Main Parts:

- OLED Pixels: Tiny LEDs that light up to form images or text.
- Driver/Controller Chip (like SSD1306): Converts microcontroller commands into pixel data.
- Communication Pins (I<sup>2</sup>C): Send data from the microcontroller to the display.
- Power Circuit: Powers the display and pixels.

### Working Principle:

- The microcontroller sends commands through I<sup>2</sup>C.
- The controller chip turns ON or OFF specific OLED pixels based on this data.
- These glowing pixels create the characters or graphics seen on the screen.

## OLED Working



The board sends commands to OLED.



OLED displays the contents.

# Block Code

```

Setup
  Setup OLED Display
    SDA IO2 SCL IO1

Loop
  OLED - Display text Hello
  Delay 3000 Milliseconds
  
```

Video link on how to connect



# Pin Connection

| Board                                                                                       | Sensor |
|---------------------------------------------------------------------------------------------|--------|
| 5V                                                                                          | VCC    |
| GND                                                                                         | GND    |
| Connect to <b>S(IO1)</b> or any other <b>available IO pin</b> defined in the snippet block. | SCL    |
| Connect to <b>S(IO2)</b> or any other <b>available IO pin</b> defined in the snippet block. | SDA    |

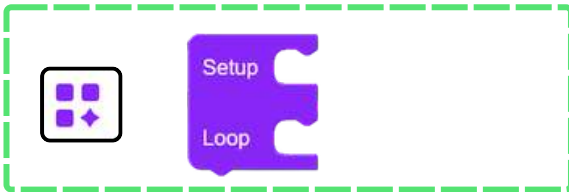
Check the IO pins you connected in the board and in the code snippet are same.



## PIN CONNECTIONS

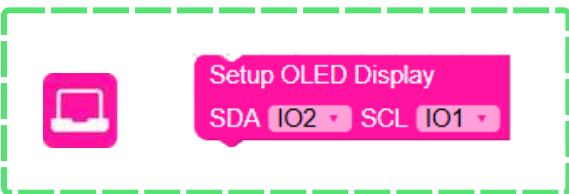
- Power line
- SDA
- SCL
- Ground line

# Code explanation



## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



## Block 2:

Configure the OLED pins. Select any **available GPIO** for using this block



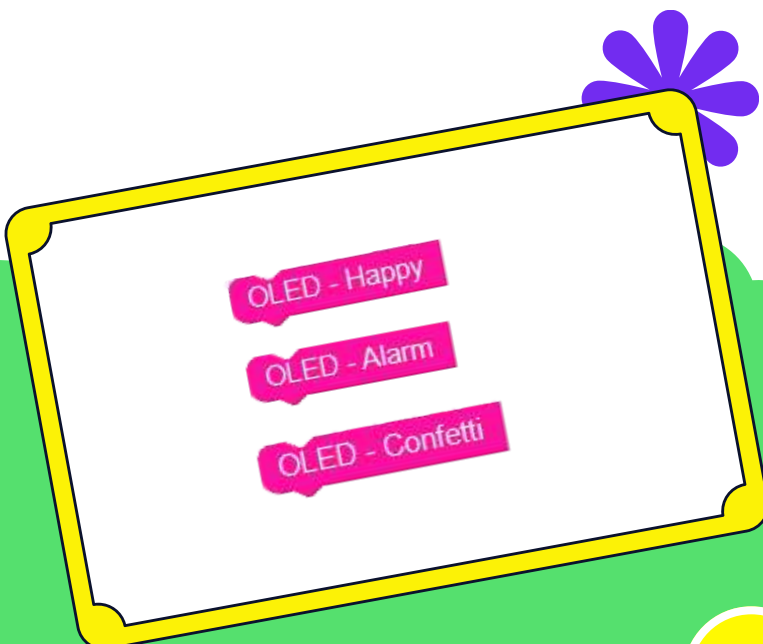
## Block 3:

This block displays **Hello** on OLED.



## Block 4:

This block delays the next block execution to 3 seconds.



try!  
me!

**Try these blocks in the code  
and have some extra fun!**



# Servo Motor 180°

## Models Compatible

Tower Pro MG90 S Micro Gear Servo Motor

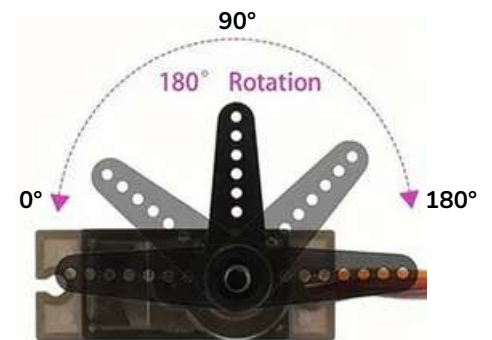
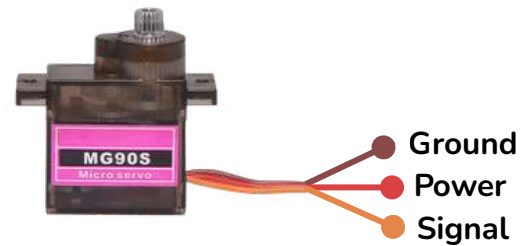


A 180° servo motor is a special type of motor that can rotate only within a fixed range — typically from 0° to 180°. You simply tell it the exact angle you want, and it turns to that position and stays there until the next command.

Inside, it contains a small DC motor connected to gears for slow, controlled movement. A built-in potentiometer continuously measures the shaft's angle and tells the control circuit whether it has reached the commanded position.

A 180° servo is controlled using PWM (Pulse Width Modulation) signals, where the width of the pulse represents the desired angle.

When you give a specific angle, the servo moves to that exact position, not beyond it. Sending the same angle again will not rotate it further — it will simply hold that angle.



Servo motor can only move from 0 to 180°

## Servo motor Working



Servo write 180°



moves to 180°

# Block Code

# Pin Connection

| Board                                                               | Sensor      |
|---------------------------------------------------------------------|-------------|
| Connect to IO1 or any S(IO) in board defined in the snippet block . | Yellow wire |
| 5V                                                                  | Red wire    |
| Ground                                                              | Brown wire  |

Video link on how to connect



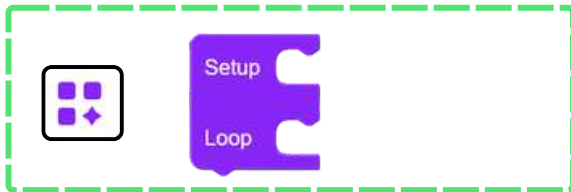
Check the IO pins you connected in the board and in the code snippet are same.



## PIN CONNECTIONS

- Power line
- Ground line
- Signal

# Code explanation



## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



## Block 2:

Use the **Attach Servo** block to connect a servo motor to a specific pin on the board



## Block 3:

Use the **Write Degrees** block to set the servo motor to a specific angle.

The servo motor moves to the particular angle that we have coded via blocks.



Servos are tiny but powerful—they're used in airplanes to move control surfaces like rudders and flaps!



# Servo Motor 360°

## Models Compatible

Tower Pro MG90 S Micro Gear Servo Motor

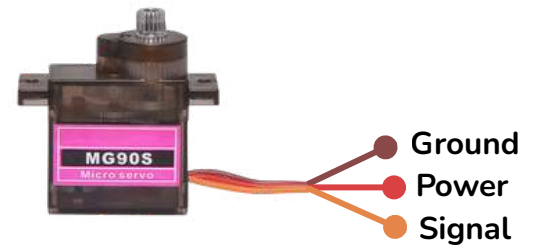


A 360° servo motor, also called a continuous rotation servo, works differently from a 180° servo. Instead of moving to a specific angle, it can rotate full 360° and continue spinning like a regular DC motor — forward or reverse.

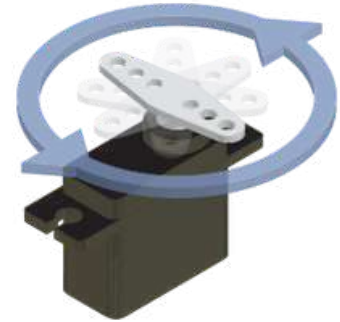
Inside, the potentiometer is replaced or modified so the servo no longer tracks position. Instead, the control circuit uses the PWM signal to control speed and direction, not angle.

A pulse near the center value stops the servo, a shorter pulse rotates it in one direction, and a longer pulse rotates it in the opposite direction.

Because it does not measure angle, you cannot tell a 360° servo to go to 90° or 150°. Instead, you instruct it to rotate clockwise or anticlockwise and at what speed.



Full 360 degrees rotation



## Servo motor Working



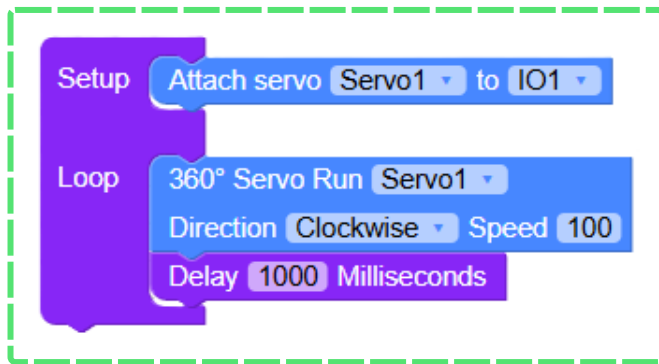
Servo write 360°



moves to 360° continuously



# Block Code



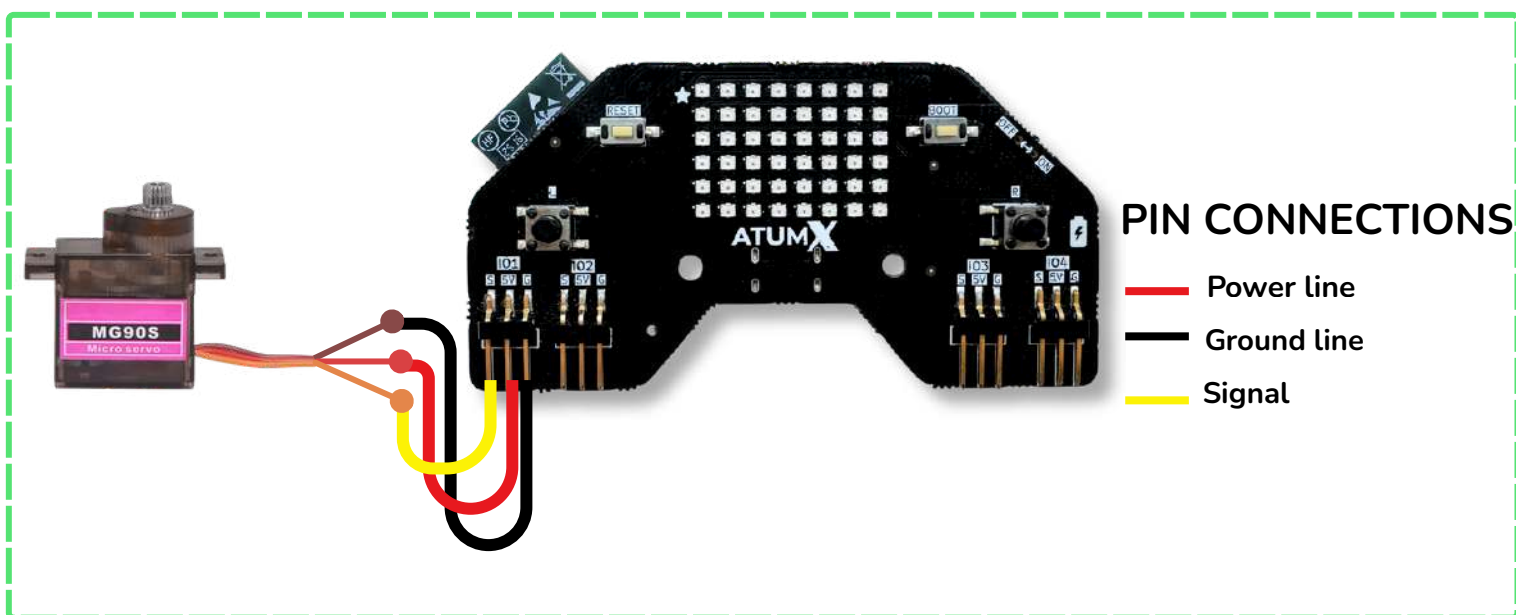
Video link on how to connect



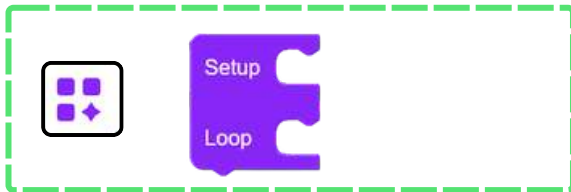
# Pin Connection

| Board                                                               | Sensor      |
|---------------------------------------------------------------------|-------------|
| Connect to IO1 or any S(IO) in board defined in the snippet block . | Yellow wire |
| 5V                                                                  | Red wire    |
| Ground                                                              | Brown wire  |

Check the IO pins you connected in the board and in the code snippet are same.



# Code explanation



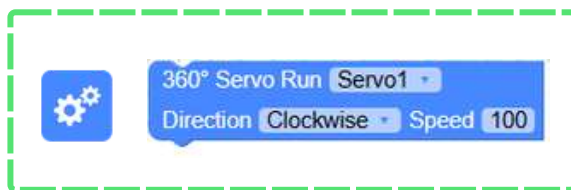
## Block 1:

From **Basics**, drag the Setup Loop block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



## Block 2:

Use the **Attach Servo** block to connect a servo motor to a specific pin on the board



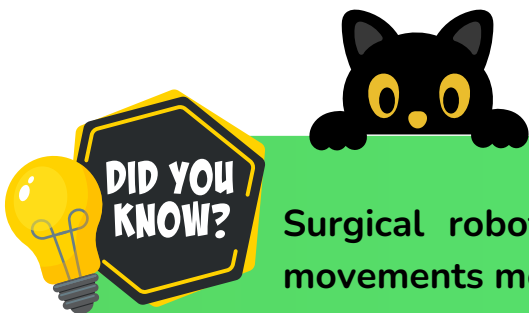
## Block 3:

Use the **Write speed and direction** block to set the servo motor to a speed and direction.



## Block 4:

Use appropriate delay to stop the servo motor.



Surgical robots use high-precision servos that can perform movements more accurate than a human hand.

# Motor Magic

## Models Compatible

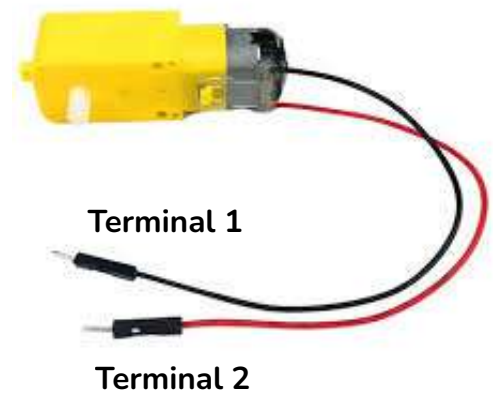
L293D / L298N Motor Driver Module



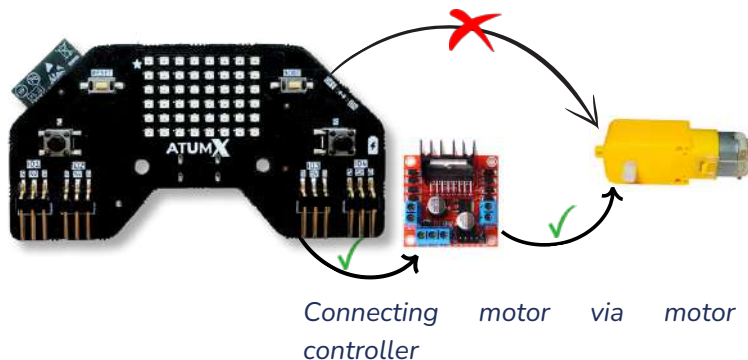
A **DC motor** (Direct Current motor) is a device that converts electrical energy into mechanical energy, producing **rotational motion**. When current flows through the motor, it creates a magnetic field that interacts with the motor's internal magnets, causing the shaft to spin. The speed and direction of rotation can be controlled by adjusting the voltage and polarity supplied to it.

### Working Principle:

A DC motor works on the principle that a current-carrying conductor placed in a magnetic field experiences a force (Lorentz force). This force causes the motor's armature to rotate, converting electrical energy into mechanical motion.



## Motor Working



A motor controller manages the power and direction of motors, acting as an intermediary between the microcontroller and the motor. It ensures precise control over speed and direction, protects the motor and microcontroller, and handles the difference in voltage and current requirements.

# Pin Connection

| Double cell battery | Motor Controller |
|---------------------|------------------|
| Red Wire            | VIN              |
| Black Wire          | GND              |

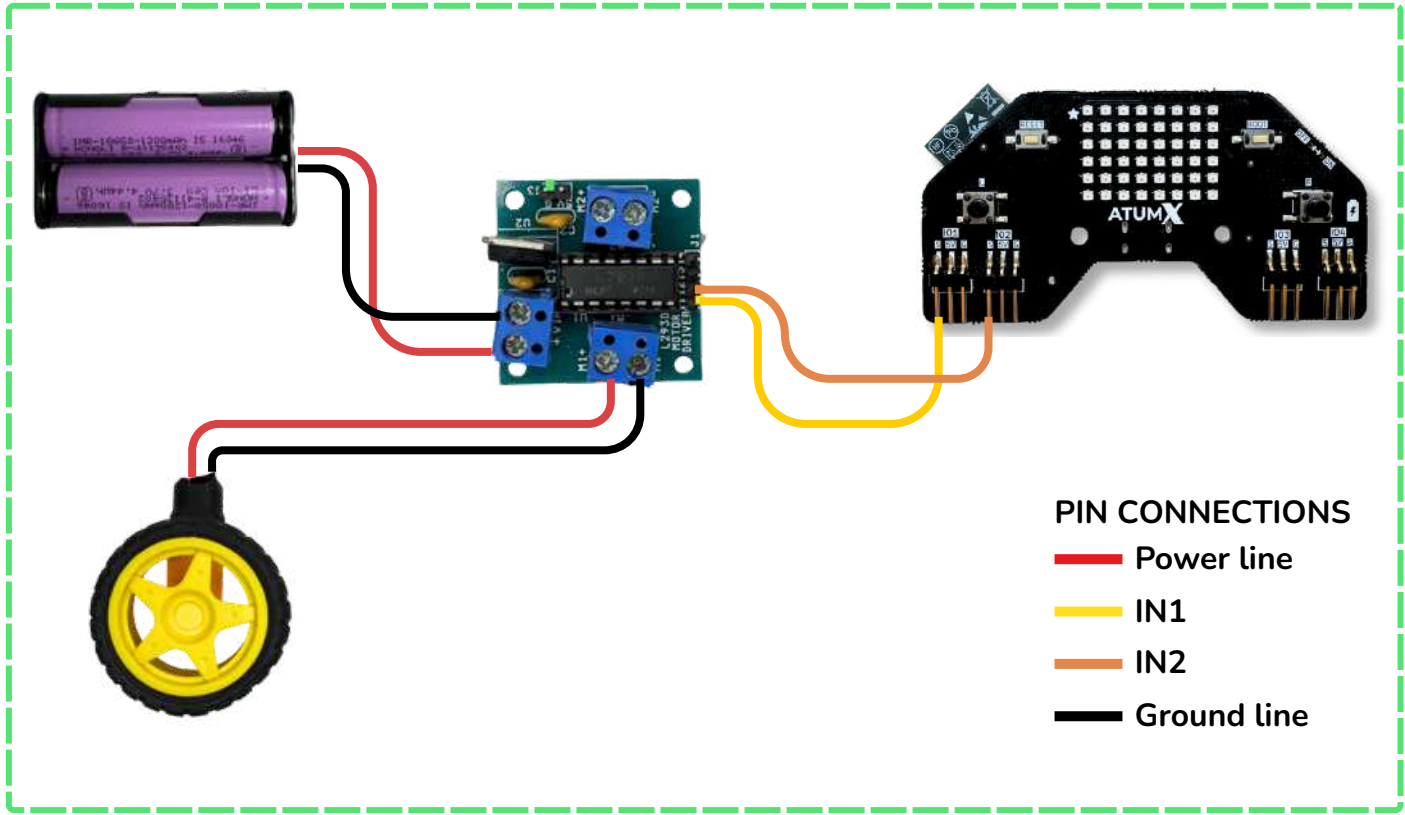
| Motor  | Motor Controller |
|--------|------------------|
| Wire 1 | M1 +             |
| Wire 2 | M1 -             |

| Board                                            | Motor Controller |
|--------------------------------------------------|------------------|
| Any GND                                          | GND              |
| IO1 or (any IO pin defined in the snippet block) | IN1              |
| IO2 or (any IO pin defined in the snippet block) | IN2              |

Video link on how to connect



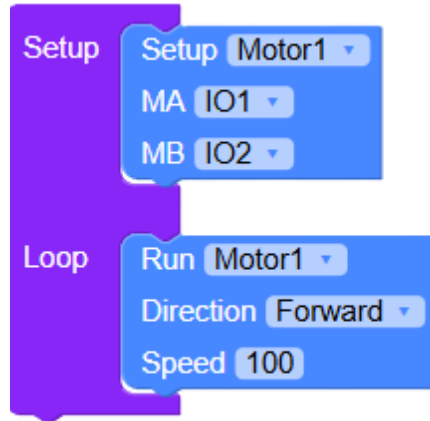
Check the IO pins you connected in the board and in the code snippet are same.



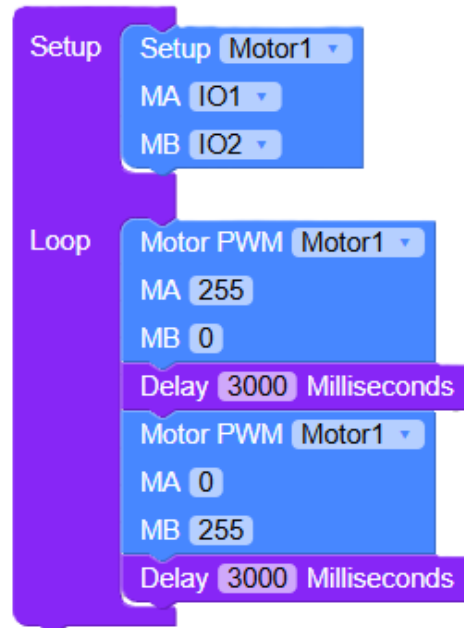


# Block Code

## Setup 1



## Setup 2

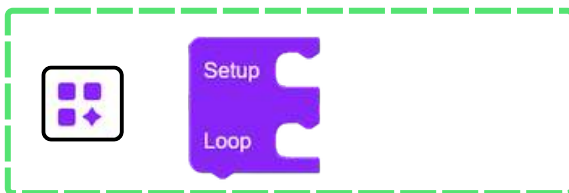


The motor will spin forward when one IN pin is HIGH and the other is LOW, and reverse when their states are swapped. If both IN pins are the same (both HIGH or both LOW), the motor will stop.

If you want to connect our board with motor without using USB, Use a JST battery as shown below

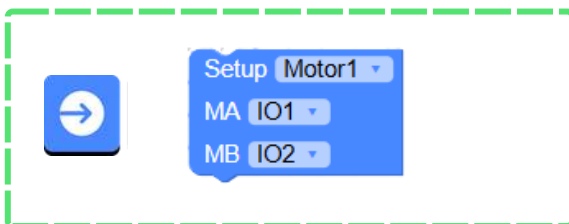


# Code explanation



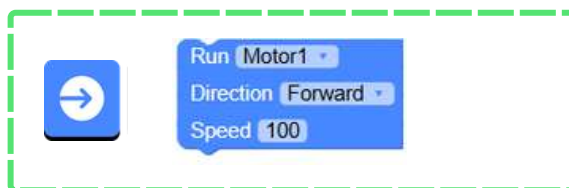
## Block 1:

From **Basics**, drag the **Setup Loop** block. Code in **Setup** runs **once** at the start, while code in **Loop** runs **continuously**.



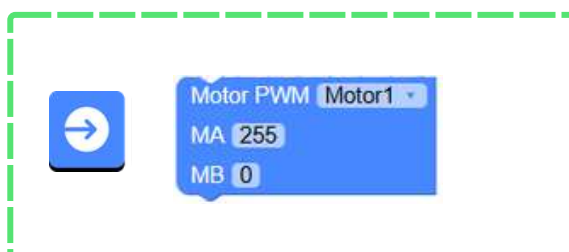
## Block 2:

Set the corresponding **MA** to **MB** pins as per your motor connections with our board



## Block 3:

Using the **Direction** block, set the desired direction **Forward/Backward** that the motor needs to run along with the speed at which you want the motors to run.



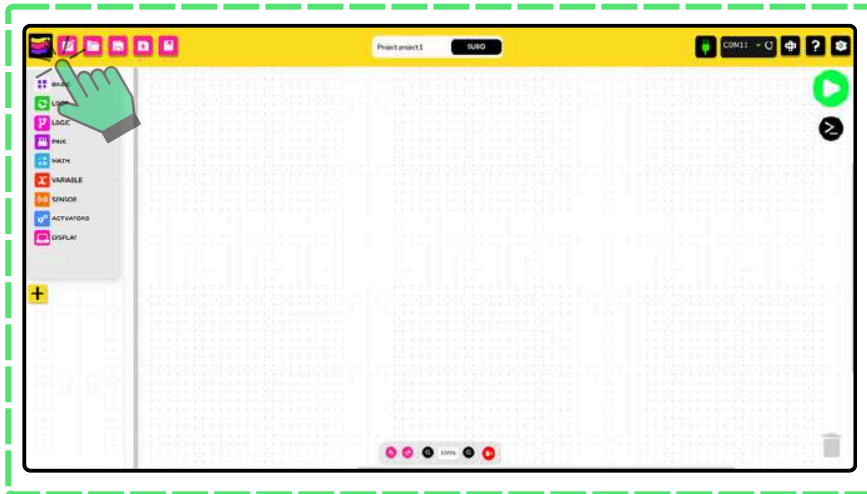
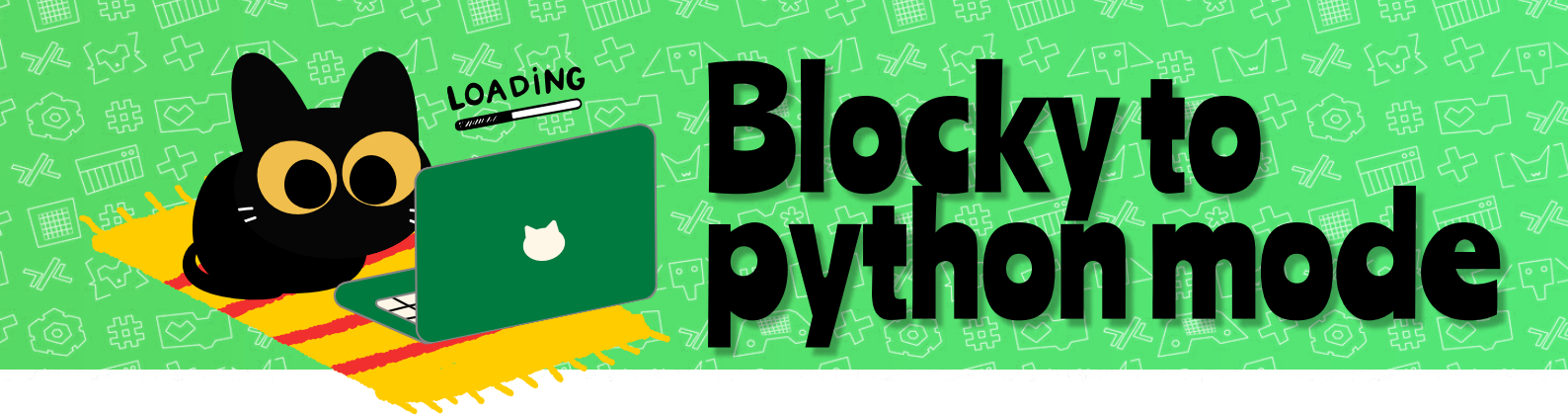
## Block 4:

Use the **Motor PWM block** to set **MA = 255** and **MB = 0** the motor runs in forward direction. Similarly when **MA = 0** and **MB = 255** the motor runs in reverse direction. This block makes the motor run in any one direction at the desired speed.



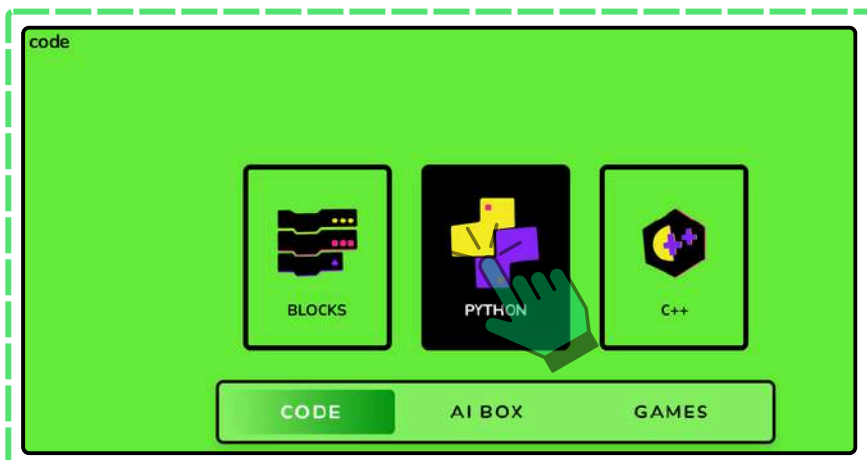
## Block 5:

Use the **Delay** block to pause the execution for a while.



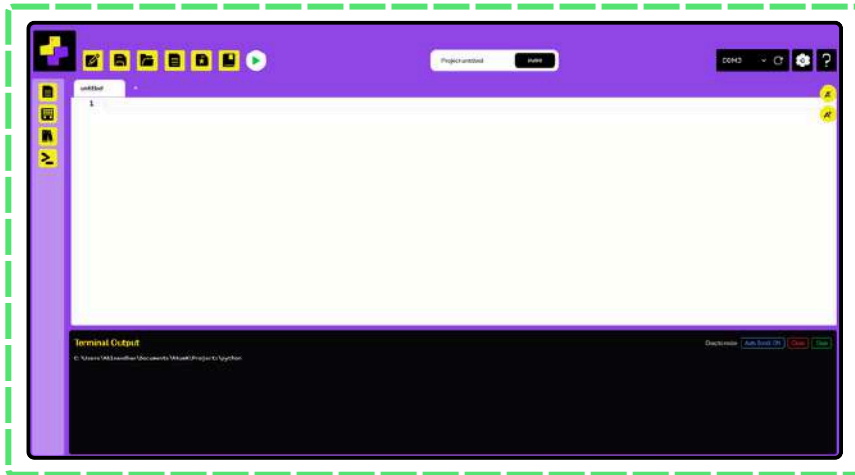
## Step 1:

Make sure that the **board is connected** to the Code app and **Click** on the Home button.



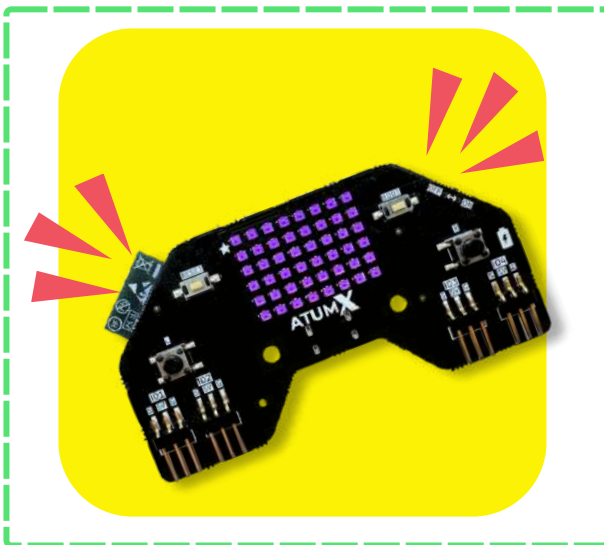
## Step 2:

Click Python in code section.



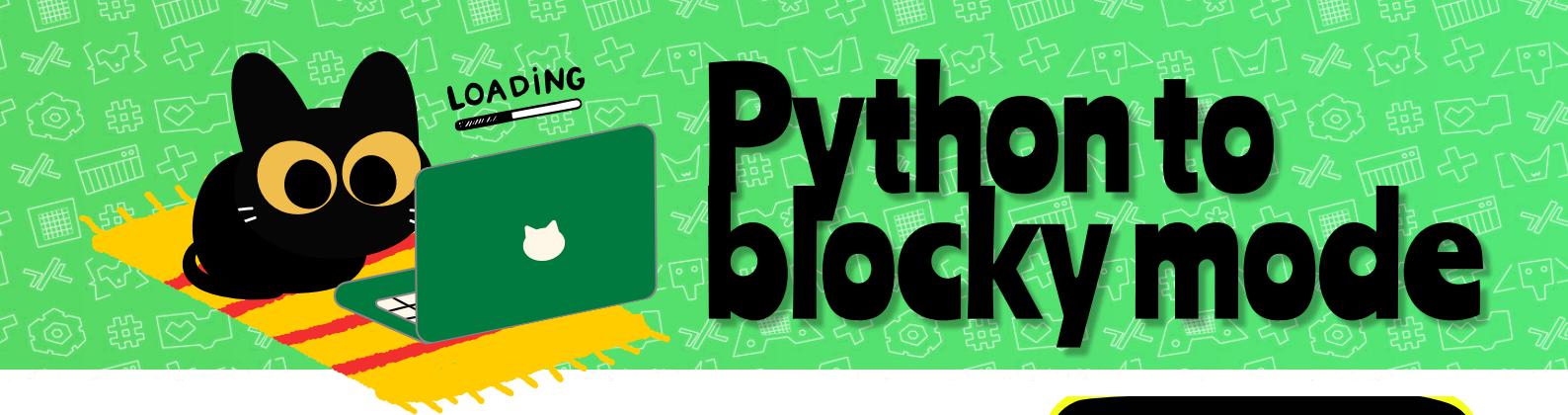
### Step 3:

The board will glow in purple color and the app is in python mode.



Board LEDs glow in purple indicating a successful switch from blockly to python mode!

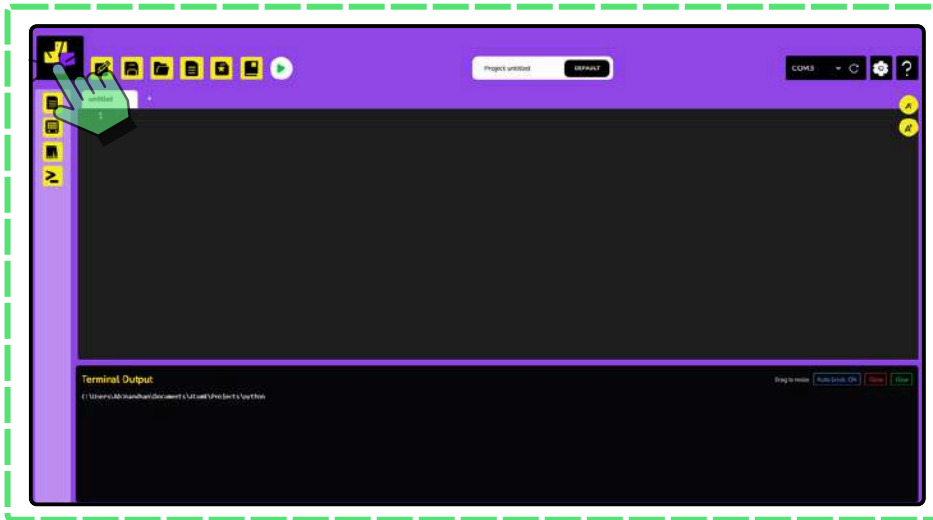




Make sure that the board is connected in wired mode

### Step 1:

Click on **home** button from python mode.



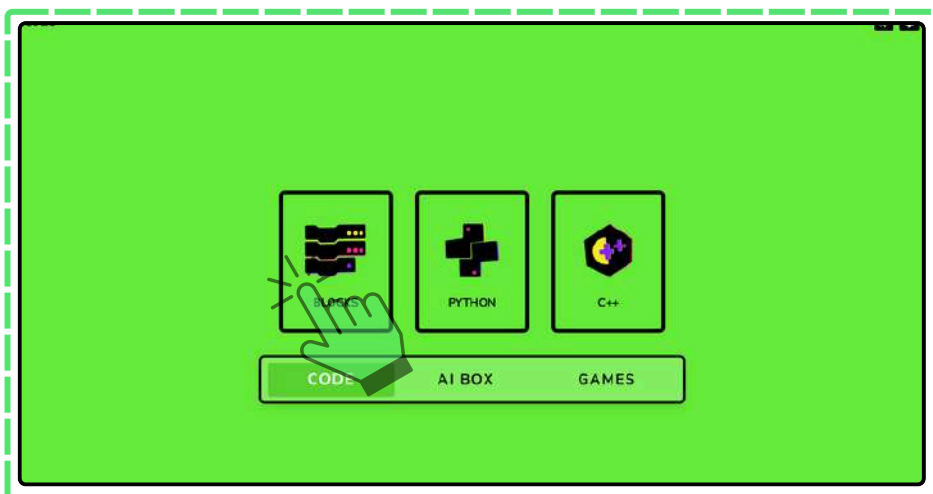
### Step 2:

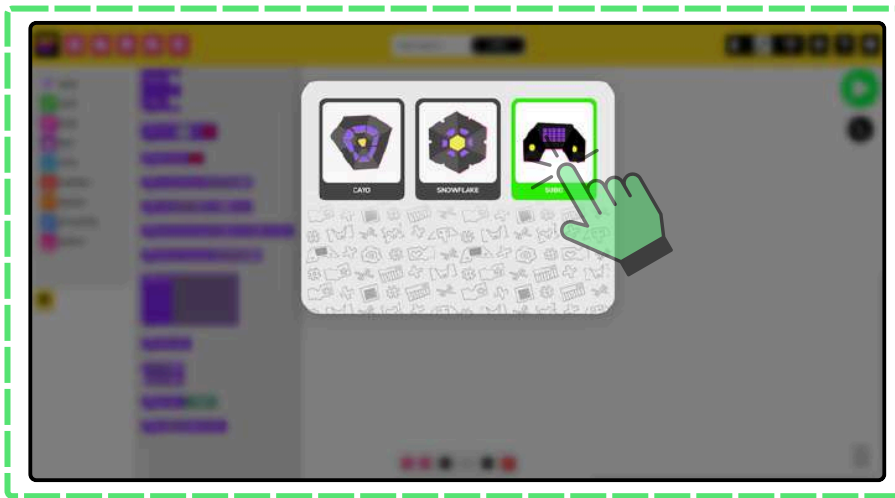
Make sure to click **Reset** button in the board after the blue light blink and **click** on **Ok**.



### Step 3:

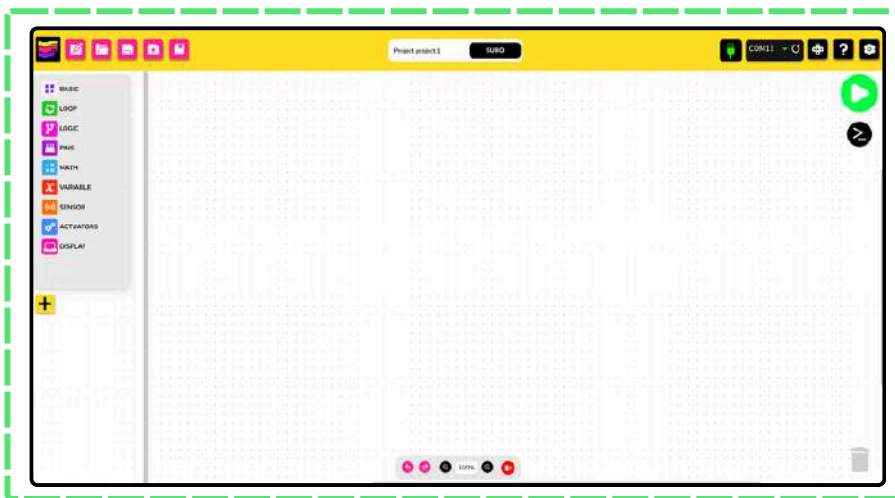
Click on **Blocks** in code section.



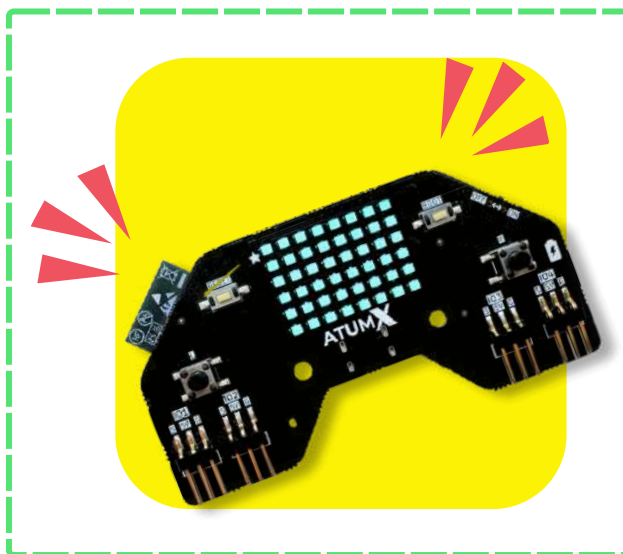


## Step 4:

Click on **Subo**.



The app is now opened and **ready** for coding.

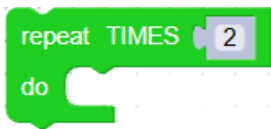


Board LEDs glow in blue indicating a successful switch from python to blockly mode!

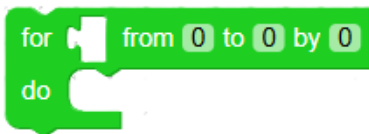
# Blocks Glossary



## LOOP



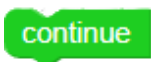
This block is used to indicate the number of times a set of actions should be **repeated automatically**, without having to add the same blocks multiple times.



This block is used to indicate a counting loop that **starts** from a set number, moves to an **end number**, **jumps by a specified value** each time, and performs the given actions at every step



This block is used to indicate an action that keeps repeating as long as a **condition is true**, and **stops** only when the **condition becomes false**



This block is used to indicate that the current step in a loop should be skipped and the next round should begin immediately without stopping

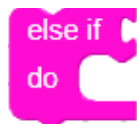
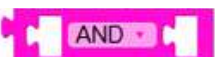
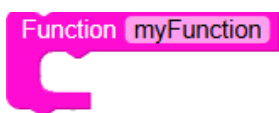
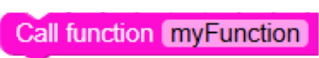


This block is used to indicate that the loop should be **stopped completely** and **exited right away**, even if all the steps **haven't been finished** yet

# Blocks Glossary



## LOGIC

|                                                                                     |                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <p>This block is used to indicate that a set of actions should only be performed when a specific <b>condition</b> is <b>true</b>, otherwise the it does nothing and moves on</p>                                       |
|    | <p>This block is used to indicate the set of actions that should be performed when the <b>If condition is false</b>, providing an alternative way to follow instead</p>                                                |
|  | <p>This block is used to indicate an <b>additional condition</b> to check when the <b>first If</b> condition is <b>false</b>, allowing <b>multiple conditions</b> to be tested one after another until one is true</p> |
|  | <p><b>Compares two values</b> using signs like = , ≠ , &lt; , &gt; , ≤ , ≥ to help the robot decide if something is equal, bigger, or smaller</p>                                                                      |
|  | <p><b>Connects two conditions</b> together <b>AND</b> means both must be true, while <b>OR</b> means at least one needs to be true for the robot to act</p>                                                            |
|  | <p>Lets you group a set of actions under one name, so you can <b>reuse</b> them anytime just by calling that name instead of rebuilding the same blocks!</p>                                                           |
|  | <p>This block is used to indicate that all the actions <b>saved</b> inside a <b>named function</b> should be executed at that point in the program</p>                                                                 |

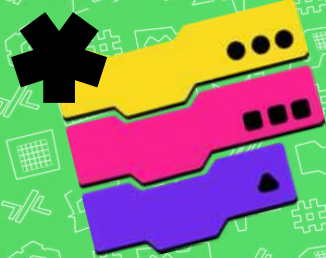


# Blocks Glossary



## PINS

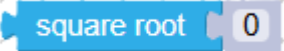
|  |                                                                                                                                                                                      |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | Checks a sensor connected to a pin and tells the robot if it's receiving a <b>HIGH (1)</b> or <b>LOW (0)</b> signal, like asking "is something there or not?"                        |
|  | Sends a <b>HIGH (ON)</b> or <b>LOW (OFF)</b> signal to a pin, like flipping a switch to turn a light, buzzer, or motor on or off                                                     |
|  | <b>Reads a range of values</b> (like 0 to 255) from a sensor, perfect for detecting things like light levels or distance that change gradually!                                      |
|  | <b>Reads a range of values</b> (like 0 to 255) from a sensor, perfect for detecting things like light levels or distance that change gradually                                       |
|  | This block is used to indicate whether a pin should be set as an <b>OUTPUT</b> (to control something) or <b>INPUT</b> (to read something), like assigning a specific job to each pin |



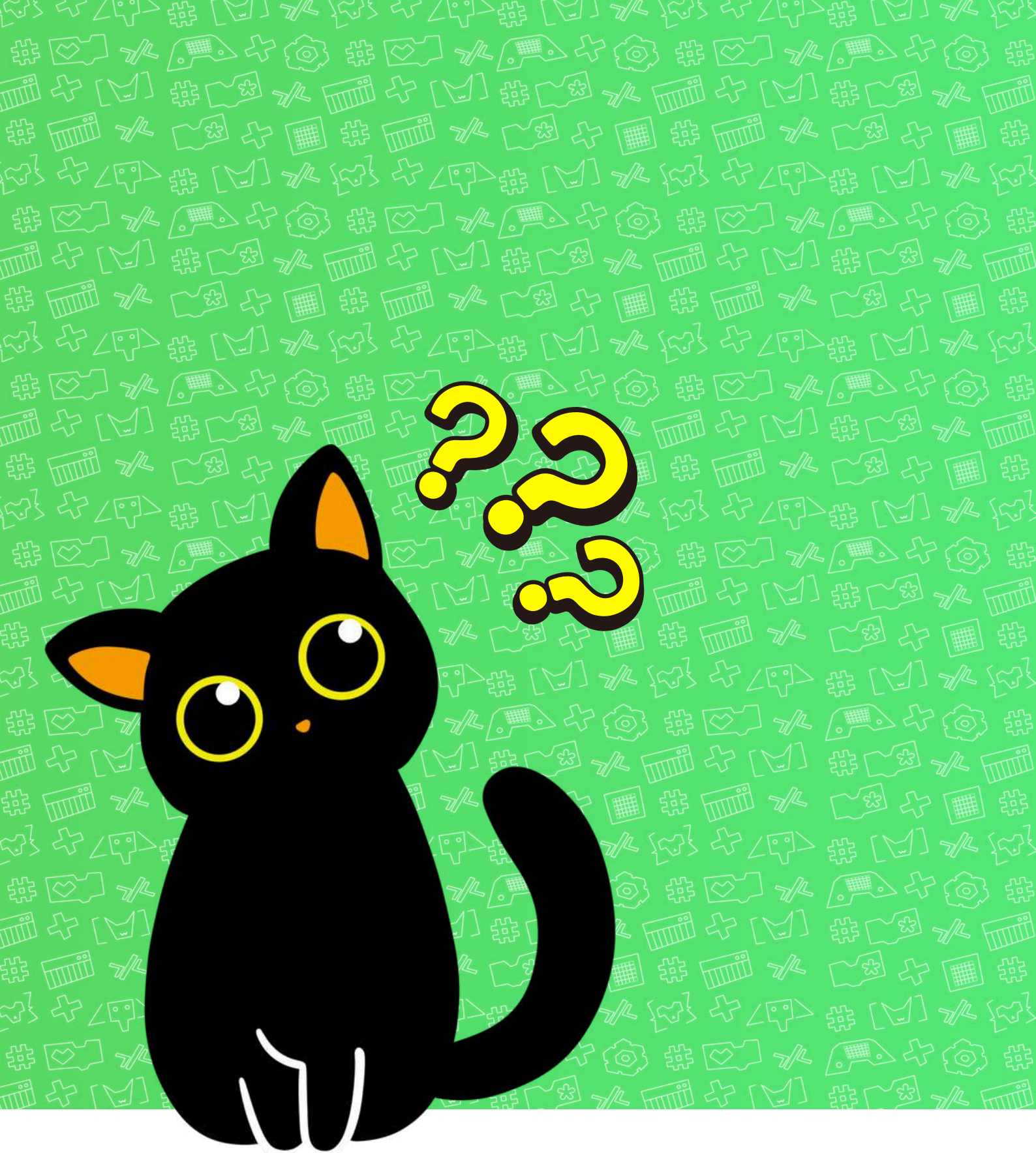
# Blocks Glossary



## MATH

|                                                                                     |                                                                                                                                      |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
|    | <p>Holds a numeric value that you can set to any number, which the robot <b>uses for counting, comparing, or calculations</b></p>    |
|    | <p><b>Performs basic math</b> between two numbers using + , - , / , × so the robot can calculate values on its own</p>               |
|  | <p><b>Divides two numbers</b> and gives back the <b>leftover number</b>, for example <math>7 \div 2 = 3</math> remainder 1</p>       |
|  | <p><b>Finds the square root</b> of a number, for example the <b>square root of 9 is 3</b>, useful for advanced math calculations</p> |
|  | <p><b>Picks a surprise random number</b> between 0 and 100</p>                                                                       |





# Common Questions

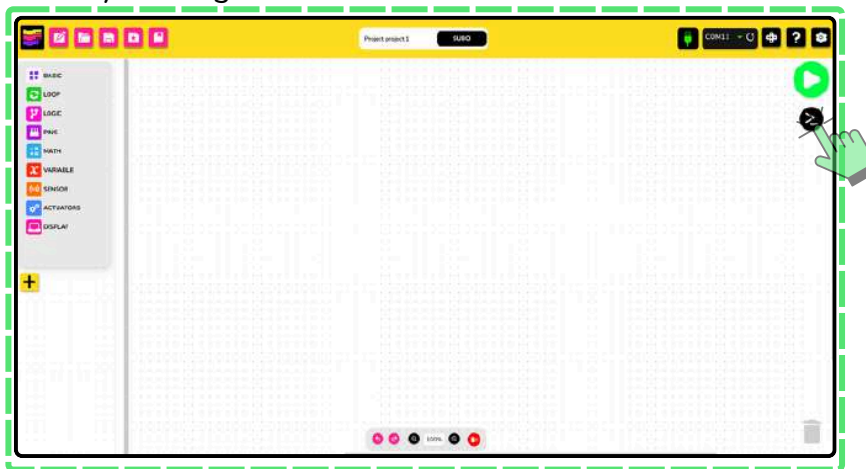
# Blockly

## 1. How to Find the Right COM Port for Your SUBO Board?

If you see multiple COM Ports, turn off Bluetooth on your PC. It removes the extra ports and helps you detect and select SUBO board on the code app.

## 2. Where are the values printed?

The readings or the values are printed on the Serial monitor. You can access the serial monitor by clicking on the icon.



## 3. Can I use SUBO board with Arduino or any other different IDE?

No. The firmware of the SUBO board gets corrupted and **warranty gets void**.

## 4. Why am I not able to rename the board when it is connected in wired mode?

The board rename feature works only when the board is connected **wirelessly**.

## 5. Any other queries?

Feel free to write to us at [info@atumx.in](mailto:info@atumx.in)



